



HarmonyOS 开发板

实验指导书

V2.1

华清远见教育集团 • 研发中心



170-9108-5953



2306275952



www.fsdev.com.cn/



support@farsight.com.cn



前言

HarmonyOS 是一款面向万物互联时代的、全新的分布式操作系统。在传统的单设备系统能力基础上，HarmonyOS 提出了基于同一套系统能力、适配多种终端形态的分布式理念，能够支持手机、平板、智能穿戴、智慧屏、车机等多种终端设备，提供全场景（移动办公、运动健康、社交通信、媒体娱乐等）业务能力。HarmonyOS 有三大特征：

1. 搭载该操作系统的设备在系统层面融为一体、形成超级终端，让设备的硬件能力可以弹性扩展，实现设备之间硬件互助，资源共享。对消费者而言，HarmonyOS 能够将生活场景中的各类终端进行能力整合，实现不同终端设备之间的快速连接、能力互助、资源共享，匹配合适的设备、提供流畅的全场景体验。

2. 面向开发者，实现一次开发，多端部署。对应用开发者而言，HarmonyOS 采用了多种分布式技术，使应用开发与不同终端设备的形态差异无关，从而让开发者能够聚焦上层业务逻辑，更加便捷、高效地开发应用。

3. 一套操作系统可以满足不同能力的设备需求，实现统一 OS，弹性部署。对设备开发者而言，HarmonyOS 采用了组件化的设计方案，可根据设备的资源能力和业务特征灵活裁剪，满足不同形态终端设备对操作系统的要求。

HarmonyOS 轻量和小系统适用于内存较小的 IOT 设备，FS_Hi3861 开发板所使用的是 WF-H861-R TA1 模组，此模组是基于 Hi3861 芯片设计，Hi3861/Hi3881 是一款高度集成的 2.4GHz Wi-Fi 芯片，集成 IEEE 802.11b/g/n 基带和 RF（Radio Frequency）电路，包括功率放大器 PA（Power Amplifier）、低噪声放大器 LNA（Low Noise Amplifier）、RF balun、天线开关以及电源管理模块等。Hi3861/Hi3881 Wi-Fi 基带实现正交频分复用（OFDM）技术，并向下兼容直接序列扩频（DSSS）、补码键控（CCK）技术，支持 IEEE 802.11b/g/n 协议。





目 录

前 言.....	I
目 录.....	i
第 1 章 HarmonyOS 之内核实验系列.....	- 1 -
1.1 HarmonyOS 之任务实验.....	- 1 -
1.2 HarmonyOS 之定时器实验.....	- 7 -
1.3 HarmonyOS 之事件实验.....	- 11 -
1.4 HarmonyOS 之互斥锁实验.....	- 14 -
1.5 HarmonyOS 之互斥型信号量实验.....	- 18 -
1.6 HarmonyOS 之同步型信号量实验.....	- 22 -
1.7 HarmonyOS 之计数型信号量实验.....	- 25 -
1.8 HarmonyOS 之消息队列实验.....	- 29 -
第 2 章 HarmonyOS 之基础实验系列.....	- 34 -
2.1 GPIO 之 KEY 按键实验	- 34 -
2.2 I2C 之 IO 扩展芯片-驱动风扇实验.....	- 38 -
2.3 I2C 之 IO 扩展芯片-驱动蜂鸣器实验.....	- 44 -
2.4 I2C 之 IO 扩展芯片-驱动 LED 灯实验.....	- 47 -
2.5 I2C 之驱动 RGB 灯实验.....	- 50 -
2.6 I2C 之温湿度传感器实验	- 54 -
2.7 I2C 之驱动三合一传感器实验	- 59 -
2.8 I2C 之驱动 0.96 寸 OLED 显示屏实验	- 63 -
2.9 I2C 之驱动 NFC 实验	- 67 -
第 3 章 HarmonyOS 之物联网实验系列.....	- 73 -
3.1 Wi-Fi 之建立网络实验.....	- 73 -
3.2 Wi-Fi 之连接网络实验.....	- 79 -
3.3 Wi-Fi 之 UDP 通信实验.....	- 82 -
3.4 Wi-Fi 之 TCP 通信实验	- 87 -
3.5 Wi-Fi 之 MQTT 通信实验	- 92 -
3.6 Wi-Fi 之 NFC 配网实验.....	- 100 -
3.7 Wi-Fi 之连接华为 IoTDA 设备接入	- 106 -
第 4 章 常见问题.....	- 118 -
4.1 在添加第三方库完成后，再次编译报错，提示“undefined reference to ****”	- 118 -
4.2 编译的时候，提示 BUILD.gn 文件错误。	- 119 -
4.3 编译的时候，提示“undefined reference to ‘hi_i2c_init’”	- 120 -
4.4 为什么我编译好之后烧写工具又检测不到板子了？	- 121 -



- 4.5 在编译开发板程序的时候，提示“/bin/sh: riscv32-unknown-elf-gcc : 未找到命令”？ - 122 -
- 4.6 为什么我的开发板上蜂鸣器一上电就开始响？- 123 -



第1章 HarmonyOS 之内核实验系列

1.1 HarmonyOS 之任务实验

1.1.1 实验目的

1. 熟悉 HarmonyOS 系统下任务相关函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.1.2 实验要求

创建两个任务，任务 1 正常执行，每隔一秒打印一次内容；任务 2 每隔 5 秒钟会恢复和挂起任务 1。

1.1.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验源码\kernel_01_task】

1.1.4 实验原理

HarmonyOS LiteOS-M 内核是面向 IoT 领域构建的轻量级物联网操作系统内核，具有小体积、低功耗、高性能的特点。其代码结构简单，主要包括内核最小功能集、内核抽象层、可选组件以及工程目录等。

HarmonyOS LiteOS-M 内核架构包含硬件相关层以及硬件无关层，如下图所示，其中硬件相关层按不同编译工具链、芯片架构分类，提供统一的 HAL（Hardware Abstraction Layer）接口，提升了硬件易适配性，满足 AIoT 类型丰富的硬件和编译工具链的拓展；其他模块属于硬件无关层，其中基础内核模块提供基础能力，扩展模块提供网络、文件系统等组件能力，还提供错误处理、调测等能力，KAL（Kernel Abstraction Layer）模块提供统一的标准接口。

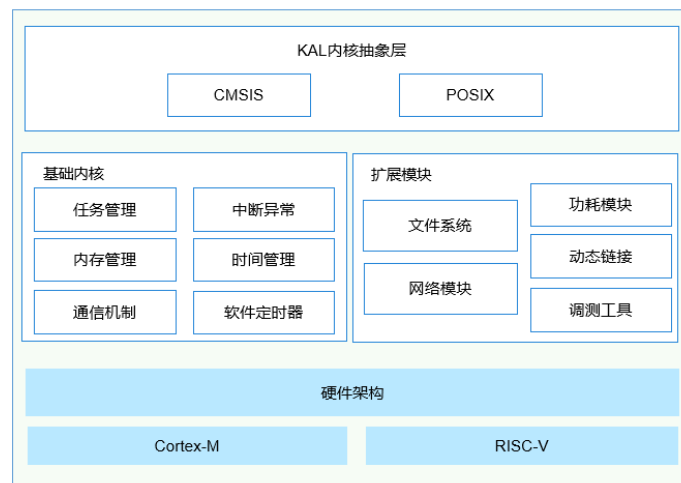




图2-1 HarmonyOS LiteOS-M 内核架构

CMSIS-RTOS2 (CMSIS-RTOS API Version2) 是 Arm®Cortex®-M 处理器的通用的 RTOS 接口。为需要 RTOS 功能的软件组件提供了标准化的 API。

从系统的角度看，任务是竞争系统资源的最小运行单元，HarmonyOS LiteOS-M 的任务模块可以给用户提供多个任务，实现任务间的切换。

1. 任务模块特性

1) 支持多任务运行。

2) 一个任务表示一个线程。

3) 抢占式任务调度机制，高优先级任务可以打断低优先级任务，低优先级任务必须在高优先级任务阻塞或结束后才能得到调度运行。

4) 相同优先级任务支持时间片轮转调度方式。

5) 共有 32 个优先级[0-31]，最高优先级为 0，最低优先级为 31。

2. 任务运行状态。

系统初始化完成后，创建的任务就可以在系统中竞争一定的资源，此时由内核进行调度，任务状态通常分为以下四种状态：

1) 就绪态 (Ready)：该任务在就绪队列中，等待 CPU 运行。

2) 运行态 (Running)：该任务正在执行。

3) 阻塞态 (Blocked)：该任务不在就绪队列中。还有任务被挂起、任务被延时、任务正在等待信号量、读写队列或者等待读写事件等，都会处于阻塞状态。

4) 退出态 (Dead)：该任务运行结束，等待系统回收资源。

3. 任务状态切换

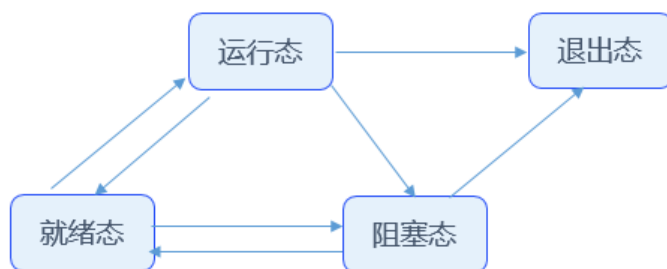


图2-2 任务状态切换

任务状态切换说明：

● 就绪态→运行态

任务创建后进入就绪态，发生任务切换时，就绪队列中最高优先级的任务被执行，从而进入运行态，同时该任务从就绪队列中移出。

● 运行态→阻塞态



正在运行的任务发生阻塞（挂起、延时、读信号量等）时，将该任务插入到对应的阻塞队列中，任务状态由运行态变成阻塞态，然后发生任务切换，运行就绪队列中最高优先级任务。

● 阻塞态→就绪态（阻塞态→运行态）

阻塞的任务被恢复后（任务恢复、延时时间超时、读信号量超时或读到信号量等），此时被恢复的任务会被加入就绪队列，从而由阻塞态变成就绪态；此时如果被恢复任务的优先级高于正在运行任务的优先级，则会发生任务切换，该任务由就绪态变成运行态。

● 就绪态→阻塞态

任务也有可能在就绪态时被阻塞（挂起），此时任务状态由就绪态变为阻塞态，该任务从就绪队列中删除，不会参与任务调度，直到该任务被恢复。

● 运行态→就绪态

有更高优先级任务创建或者恢复后，会发生任务调度，此刻就绪队列中最高优先级任务变为运行态，那么原先运行的任务由运行态变为就绪态，依然在就绪队列中。

● 运行态→退出态

运行中的任务运行结束，任务状态由运行态变为退出态。退出态包含任务运行结束的正常退出状态以及 Invalid 状态。例如，任务运行结束但是没有自删除，对外呈现的就是 Invalid 状态，即退出态。

● 阻塞态→退出态

阻塞的任务调用删除接口，任务状态由阻塞态变为退出态。

1.1.5 实验步骤

步骤1: 编写应用程序

这是需要我们在“applications/sample/wifi-iot/app”目录下新建一个目录或一套目录，用于存放自己的业务代码，首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_01_task”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_task_example.c 两个文件。下面是目录结构：

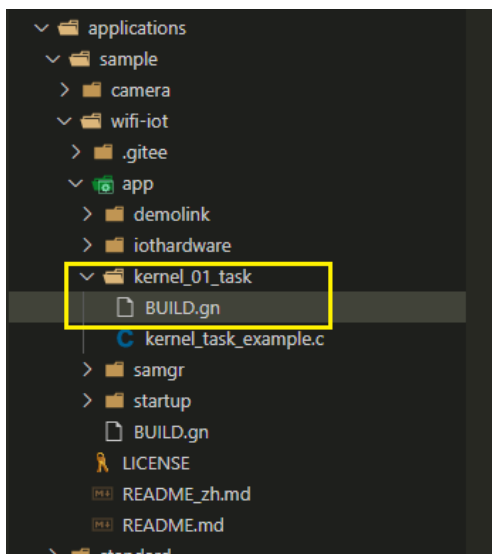


图2-3 创建目录和文件



1. 打开 “applications/sample/wifi-iot/app/kernel_01_task/kernel_task_example.c” 文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。

```

1 osThreadId_t Task1_ID;    // 任务 1 ID
2 osThreadId_t Task2_ID;    // 任务 2 ID
3 void Task1(void *argument)
4 {
5     (void)argument;
6     while (1)
7     {
8         printf("Task 1.....\n");
9         sleep(1);
10    }
11}
12 void Task2(void *argument)
13 {
14     (void)argument;
15     uint8_t num = 0;
16     while (1)
17     {
18         printf("Task 2, 开始挂起任务 1\n");
19         osThreadSuspend(Task1_ID);    // 挂起任务 1
20         sleep(5);                    // 延时 5 秒钟
21
22         printf("Task 2, 开始恢复任务 1\n");
23         osThreadResume(Task1_ID);    // 恢复任务 1
24         sleep(5);                    // 延时 5 秒钟
25     }
26 }
27 static void kernel_task_example(void)
28 {
29     printf("Enter kernel_task_example()!\n");
30
31     osThreadAttr_t options;
32     options.name = "Task1";          // 任务的名字
33     options.attr_bits = 0;           // 属性位
34     options.cb_mem = NULL;           // 堆空间地址
35     options.cb_size = 0;             // 堆空间大小
36     options.stack_mem = NULL;        // 栈空间地址
37     options.stack_size = 1024;       // 栈空间大小 单位:字节
38     options.priority = osPriorityNormal; // 任务的优先级，数据越大，优先级越高
39
40     // 创建任务 1
41     Task1_ID = osThreadNew((osThreadFunc_t)Task1, NULL, &options);

```



```

42 if (Task1_ID != NULL)
43 {
44     printf("ID = %d, Create Task1_ID is OK!\n", Task1_ID);
45 }
46
47 // 创建任务 2
48 options.name = "Task2";
49 options.priority = osPriorityNormal1;
50 Task2_ID = osThreadNew((osThreadFunc_t)Task2, NULL, &options);
51 if (Task2_ID != NULL)
52 {
53     printf("ID = %d, Create Task2_ID is OK!\n", Task2_ID);
54 }
55 }
56 SYS_RUN(kernel_task_example);
    
```

2. 编写用于将逻辑代码构建程静态库的“applications/sample/wifi-iot/app/kernel_01_task/BUILD.gn”文件。BUILD.gn 文件由三部分组成（目标、源文件、头文件路径）构成。

1) static_library 中指定业务模块的编译结果，为静态库文件 libkernel_task_example.a。

2) sources 中指定静态库.a 所依赖的.c 文件及其路径，若路径中包含“/”则表示绝对路径（此处为代码根路径），若不包含“/”则表示相对路径。

3) include_dirs 中指定 source 所需要依赖的.h 文件路径。

打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("kernel_task_example") {
2     sources = [
3         "kernel_task_example.c"
4     ]
5
6     include_dirs = [
7         "../utils/native/lite/include",
8         "../kernel/liteos_m/kal/cmsis",
9     ]
10 }
11
    
```

3. 编写“applications/sample/wifi-iot/app/BUILD.gn”文件，指定需要参与构建的特性模块。在 features 字段中增加索引，使目标模块参与编译。features 字段指定业务模块的路径和目标

1) kernel_01_task 是相对路径，指向./applications/sample/wifi-iot/app/kernel_01_task/BUILD.gn。

2) kernel_task_example 是目标，指向./applications/sample/wifi-iot/app/kernel_01_task/BUILD.gn 中的 static_library("kernel_task_example")。

打开 BUILD.gn 文件，填入一下代码：



```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         "kernel_01_task:kernel_task_example",
7     ]
8 }
9

```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

1.1.6 实验现象

重启 hi3861 模组之后，首先在任务 2 中执行挂起任务 1，等待 5 秒后，在任务 2 中执行恢复任务 1，等待 5 秒钟，此时任务 1 将每隔 1 秒打印输出一句话，等待 5 秒钟后，任务 2 将任务 1 再次挂起。

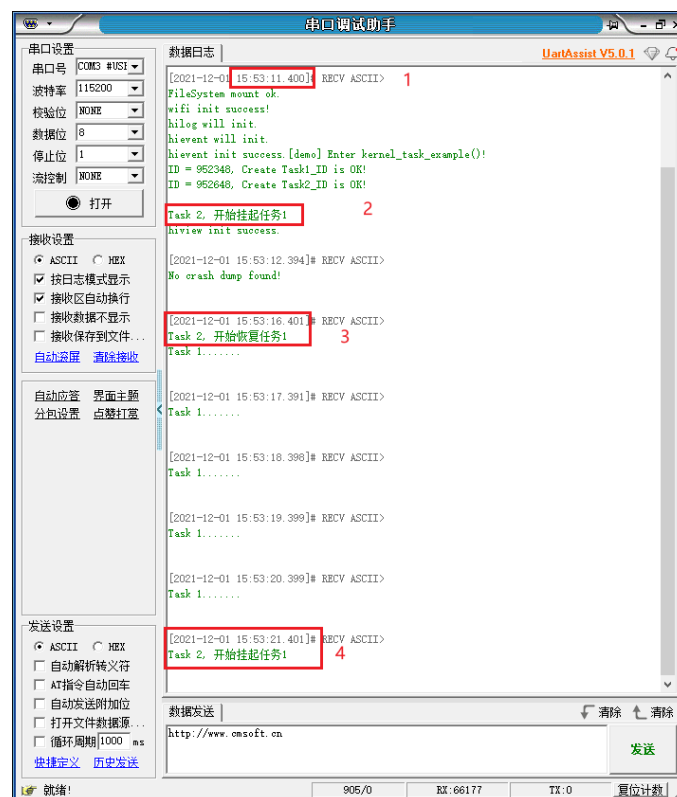


图2-4 实验串口打印结果



1.2 HarmonyOS 之定时器实验

1.2.1 实验目的

1. 熟悉 HarmonyOS 系统下软件定时器相关函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.2.2 实验要求

创建一个任务和一个软件定时器，任务 1 会每隔 1 秒钟打印输出一句话；软件定时器会每隔 3 秒钟打印输出一句话。

1.2.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\kernel_02_timer】

1.2.4 实验原理

软件定时器，是基于系统 Tick 时钟中断且由软件来模拟的定时器，当经过设定的 Tick 时钟计数值后会触发用户定义的回调函数。定时精度与系统 Tick 时钟的周期有关。

硬件定时器受硬件的限制，数量上不足以满足用户的实际需求，因此为了满足用户需求，提供更多的定时器，HarmonyOS LiteOS-M 内核提供软件定时器功能。软件定时器扩展了定时器的数量，允许创建更多的定时业务。

软件定时器功能上支持：

1. 静态裁剪：能通过宏关闭软件定时器功能。
2. 软件定时器创建。
3. 软件定时器启动。
4. 软件定时器停止。
5. 软件定时器删除。
6. 软件定时器剩余 Tick 数获取。

软件定时器是系统资源，在模块初始化的时候已经分配了一块连续的内存，系统支持的最大定时器个数由 `los_config.h` 中的 `LOSCFG_BASE_CORE_SWTMR_LIMIT` 宏配置。

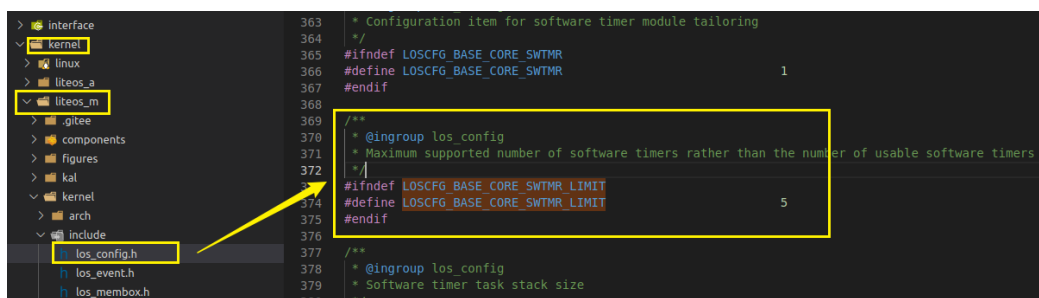


图2-5 LOSCFG_BASE_CORE_SWTMR_LIMIT 宏配置

软件定时器使用了系统的一个队列和一个任务资源，软件定时器的触发遵循队列规则，先进先出。定时短的定时器总是比定时长的靠近队列头，满足优先被触发的准则。

软件定时器以 Tick 为基本计时单位，当用户创建并启动一个软件定时器时，HarmonyOS LiteOS-M 内核会根据当前系统 Tick 时间及用户设置的定时时间间隔确定该定时器的到期 Tick 时间，并将该定时器控制结构挂入计时全局链表。

当 Tick 中断到来时，在 Tick 中断处理函数中扫描软件定时器的计时全局链表，看是否有定时器超时，若有则将超时的定时器记录下来。

Tick 中断处理函数结束后，软件定时器任务（优先级为最高）被唤醒，在该任务中调用之前记录下来的定时器的超时回调函数。

1.2.5 实验步骤

步骤1: 编写应用程序

首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_02_timer”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_timer_example.c 两个文件。

1. 打开“applications/sample/wifi-iot/app/kernel_02_timer/kernel_timer_example.c”文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。

```
1 osThreadId_t Task1_ID; // 任务 1 ID
2 osTimerId_t Timer_ID; // 定时器 ID
3 // 任务 1
4 void Task1(void *argument)
5 {
6     (void)argument;
7     while (1)
8     {
9         printf("enter Task 1.....\n");
10        sleep(1); // 1 秒
11    }
12}
13// 软件定时器回调函数
14void timer1_Callback(void *argument)
15{
```



```

16 (void)argument;
17 printf("enter timer1_Callback.....\n");
18 }
19 static void kernel_timer_example(void)
20 {
21     printf("Enter kernel_timer_example()!\n");
22
23     osThreadAttr_t taskOptions;
24     taskOptions.name = "Task1";           // 任务的名字
25     taskOptions.attr_bits = 0;           // 属性位
26     taskOptions.cb_mem = NULL;           // 堆空间地址
27     taskOptions.cb_size = 0;             // 堆空间大小
28     taskOptions.stack_mem = NULL;        // 栈空间地址
29     taskOptions.stack_size = 1024;       // 栈空间大小 单位:字节
30     taskOptions.priority = osPriorityNormal; // 任务的优先级
31     // 创建任务
32     Task1_ID = osThreadNew((osThreadFunc_t)Task1, NULL, &taskOptions);
33     if (Task1_ID != NULL)
34     {
35         printf("ID = %d, Create Task1_ID is OK!\n", Task1_ID);
36     }
37     // 创建定时器
38     Timer_ID = osTimerNew(timer1_Callback, osTimerPeriodic, NULL, NULL);
39     if (Timer_ID != NULL)
40     {
41         printf("ID = %d, Create Timer_ID is OK!\n", Timer_ID);
42         // 开始定时器, 并赋予定时器的定时值 (在 Hi3861 中, 1U=10ms, 100U=1S)
43         osStatus_t timerStatus = osTimerStart(Timer_ID, 300U);
44         if (timerStatus != osOK)
45         {
46             printf("timer is not startRun !\n");
47         }
48     }
49 }
50 SYS_RUN(kernel_timer_example);

```

2. 编写用于将逻辑代码构建程静态库的 “applications/sample/wifi-iot/app/kernel_02_timer/BUILD.gn” 文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("kernel_timer_example") {
2     sources = [
3         "kernel_timer_example.c"
4     ]
5     include_dirs = [

```



```
6    "//utils/native/lite/include",
7    "//kernel/liteos_m/kal/cmsis",
8  ]
9}}
```

3. 编写“applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```
1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         # "kernel_01_task:kernel_task_example",
7         "kernel_02_timer:kernel_timer_example",
8     ]
9 }
```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

1.2.6 实验现象

重启 hi3861 模组之后，任务 1 会每隔 1 秒钟打印输出一句话，而定时器会每隔 3 秒会进入到定时器的回调函数。

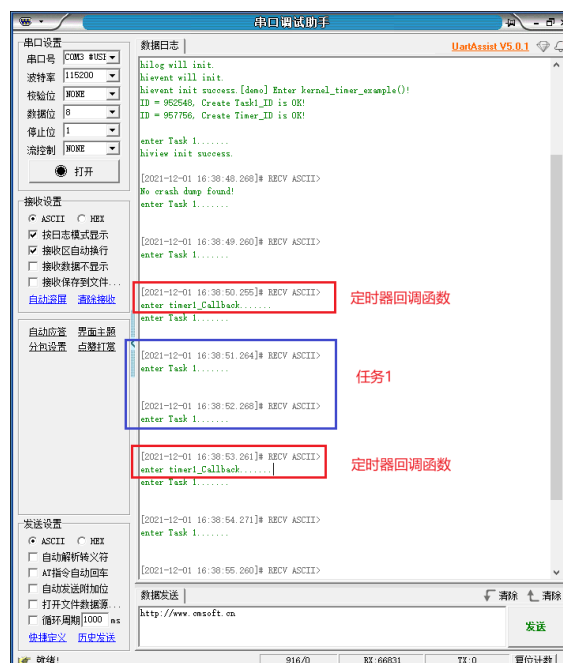


图2-6 实验串口打印结果



1.3 HarmonyOS 之事件实验

1.3.1 实验目的

1. 熟悉 HarmonyOS 系统下事件相关函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.3.2 实验要求

创建两个任务，任务 1 每隔 1 秒钟设置一个事件标志位，总共设置 3 个事件标志位；任务 2 阻塞等待是否接收到 3 个事件标志位，如果接收到了，打印输出。

1.3.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\kernel_03_event】

1.3.4 实验原理

事件（Event）是一种任务间的通信机制，可用于任务间的同步操作。事件的特点是：

1. 任务间的事件同步，可以一对多，也可以多对多。一对多表示一个任务可以等待多个事件，多对多表示多个任务可以等待多个事件。但是一次写事件最多触发一个任务从阻塞中醒来。
2. 事件读超时机制。
3. 只做任务间同步，不传输具体数据。

下面为事件的工作机制：

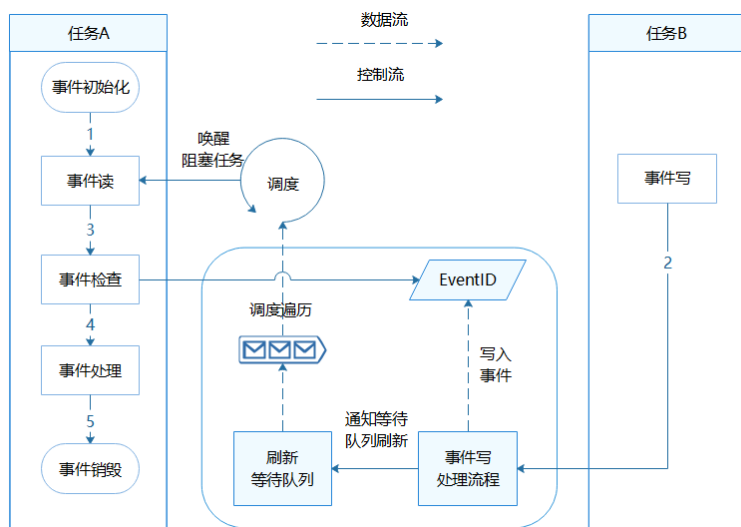


图2-7 事件工作机制



1.3.5 实验步骤

步骤1: 编写应用程序

首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_03_event”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_event_example.c 两个文件。

1. 编写程序代码。打开“applications/sample/wifi-iot/app/kernel_03_event/kernel_event_example.c”文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。

```
1 uint32_t event1_Flags = 0x00000001U; // 事件掩码 每一位代表一个事件
2 uint32_t event2_Flags = 0x00000002U; // 事件掩码 每一位代表一个事件
3 uint32_t event3_Flags = 0x00000004U; // 事件掩码 每一位代表一个事件
4
5 /**
6  * @description: 任务 1 用于发送事件
7  * @param {*}
8  * @return {*}
9  */
10 void Task1(void *argument)
11 {
12     (void)argument;
13     while (1)
14     {
15         printf("enter Task 1.....\n");
16         osEventFlagsSet(event_ID, event1_Flags); // 设置事件标记
17         printf("send eventFlag1.....\n");
18         sleep(1); // 1 秒
19         osEventFlagsSet(event_ID, event2_Flags); // 设置事件标记
20         printf("send eventFlag2.....\n");
21         sleep(1); // 1 秒
22         osEventFlagsSet(event_ID, event3_Flags); // 设置事件标记
23         printf("send eventFlag3.....\n");
24         sleep(1); // 1 秒
25     }
26 }
27 /**
28  * @description: 任务 2 用于接受事件
29  * @param {*}
30  * @return {*}
31  */
32 void Task2(void *argument)
33 {
34     (void)argument;
35     uint32_t flags = 0;
```



```

36 while (1)
37 {
38     // 永远等待事件标记触发，当接收到 event1_Flags 和 event2_Flags
39     // 和 event3_Flags 时才会执行 printf 函数
40     // osFlagsWaitAll : 全部事件标志位接收到
41     // osFlagsWaitAny: 任意一个事件标志位接收到
42     // 当只有一个事件的时候，事件的类型选择哪个都可以
43     flags = osEventFlagsWait(event_ID, event1_Flags |
44                             event2_Flags |
45                             event3_Flags, osFlagsWaitAll, osWaitForever);
46     printf("receive event is OK\n");    // 事件已经标记
47 }
48 }

```

2. 编写用于将逻辑代码构建程静态库的“applications/sample/wifi-iot/app/kernel_03_event/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("kernel_timer_example") {
2     sources = [
3         "kernel_timer_example.c"
4     ]
5
6     include_dirs = [
7         "../utils/native/lite/include",
8         "../kernel/liteos_m/kal/cmsis",
9     ]
10 }

```

编写“applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("../build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         # "kernel_01_task:kernel_task_example",
7         # "kernel_02_timer:kernel_timer_example",
8         "kernel_03_event:kernel_event_example",
9     ]
10 }

```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。



步骤3: 烧写程序

按照第1章 开发环境搭建中“烧写程序”小节的内容进行烧写程序。

1.3.6 实验现象

重启 hi3861 模组之后, 首先进入到任务 1 中, 这个时候任务 2 在等待 3 个事件全部触发的时候, 才会执行。然后在任务 1 中, 每隔 1S 中设置一个事件标志位, 发送了 3 个事件, 任务 2 接收到了 3 个事件标志位, 开始执行打印信息。

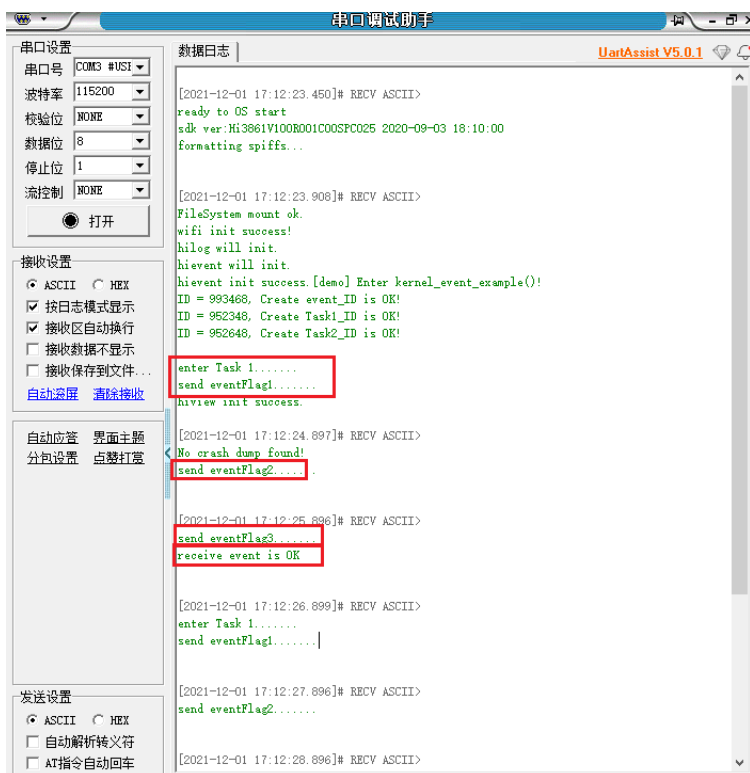


图2-8 实验串口打印结果

1.4 HarmonyOS 之互斥锁实验

1.4.1 实验目的

1. 熟悉 HarmonyOS 系统下互斥锁的通信机制。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤, 能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.4.2 实验要求

创建两个任务, 任务 1 向全局数组中存入数据, 此时在任务 2 中不能读数据; 任务 2 读全局数组中的数据, 此时在任务 1 中不能写数据。



1.4.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\kernel_04_mutex】

1.4.4 实验原理

互斥锁又称互斥型信号量，是一种特殊的二值性信号量，用于实现对共享资源的独占式处理。任意时刻互斥锁的状态只有两种，开锁或闭锁。当有任务持有时，互斥锁处于闭锁状态，这个任务获得该互斥锁的所有权。当该任务释放它时，该互斥锁被开锁，任务失去该互斥锁的所有权。当一个任务持有互斥锁时，其他任务将不能再对该互斥锁进行开锁或持有。

多任务环境下往往存在多个任务竞争同一共享资源的应用场景，互斥锁可被用于对共享资源的保护从而实现独占式访问。另外互斥锁可以解决信号量存在的优先级翻转问题。

多任务环境下会存在多个任务访问同一公共资源的场景，而有些公共资源是非共享的，需要任务进行独占式处理。互斥锁怎样来避免这种冲突呢？

用互斥锁处理非共享资源的同步访问时，如果有任务访问该资源，则互斥锁为加锁状态。此时其他任务如果想访问这个公共资源则会被阻塞，直到互斥锁被持有该锁的任务释放后，其他任务才能重新访问该公共资源，此时互斥锁再次上锁，如此确保同一时刻只有一个任务正在访问这个公共资源，保证了公共资源操作的完整性。

运行机制如下图所示：

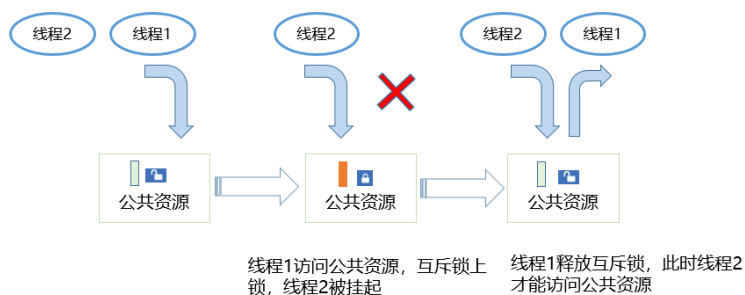


图2-9 互斥锁运行机制

1.4.5 实验步骤

步骤1: 编写应用程序

首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_04_mutex”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_mutex_example.c 两个文件。

1. 打开“applications/sample/wifi-iot/app/kernel_04_mutex/kernel_mutex_example.c”文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。



```

1 osThreadId_t Task1_ID; // 任务 1 ID
2 osThreadId_t Task2_ID; // 任务 2 ID
3 osMutexId_t Mutex_ID; // 定义互斥锁 ID
4 uint8_t buff[20] = {0}; // 定义一个共享资源
5 /**
6  * @description: 任务 1
7  * @param {*}
8  * @return {*}
9  */
10 void Task1(void *argument)
11 {
12     (void)argument;
13     printf("enter Task 1.....\n");
14     while (1)
15     {
16         osMutexAcquire(Mutex_ID, osWaitForever); // 请求互斥锁
17
18         // 操作共享数据 写数据
19         printf("write Buff Data: \n");
20         for (size_t i = 0; i < sizeof(buff); i++)
21         {
22             buff[i] = i;
23         }
24         printf("\n");
25         osMutexRelease(Mutex_ID); // 释放互斥锁
26
27         sleep(1);
28     }
29 }
30 /**
31  * @description: 任务 2
32  * @param {*}
33  * @return {*}
34  */
35 void Task2(void *argument)
36 {
37     (void)argument;
38     printf("enter Task 2.....\n");
39     while (1)
40     {
41         osMutexAcquire(Mutex_ID, osWaitForever); // 请求互斥锁
42         // 操作共享数据 读数据
43         printf("read Buff Data: \n");
44         for (size_t i = 0; i < sizeof(buff); i++)

```



```

45 {
46     printf("%d \n", buff[i]);
47 }
48 printf("\n");
49 osMutexRelease(Mutex_ID); // 释放互斥锁
50
51     sleep(1);
52 }
53 }
    
```

2. 编写用于将逻辑代码构建程静态库的“applications/sample/wifi-iot/app/kernel_04_mutex/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("kernel_mutex_example") {
2     sources = [
3         "kernel_mutex_example.c"
4     ]
5
6     include_dirs = [
7         "//utils/native/lite/include",
8         "//kernel/liteos_m/kal/cmsis",
9     ]
10 }
11
    
```

3. 编写“applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         " kernel_04_mutex:kernel_mutex_example ",
7     ]
8 }
9
    
```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。



1.4.6 实验现象

重启 hi3861 模组之后，任务 1 和任务 2 都会去访问同一块空间数据，在任务 1 中写数据时，任务 2 被挂起不能读数据，当任务 1 写数据完成释放互斥锁，任务 2 开始执行，读出共享空间中的数据，再次释放互斥锁。循环往复进行执行。

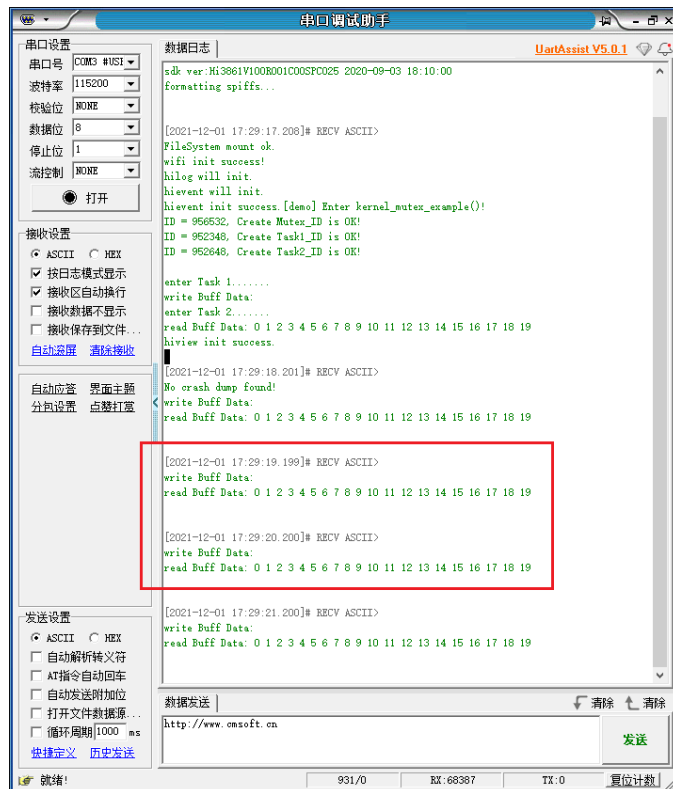


图2-10 实验串口打印结果

1.5 HarmonyOS 之互斥型信号量实验

1.5.1 实验目的

1. 熟悉 HarmonyOS 系统下信号量相关函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.5.2 实验要求

创建两个任务，任务 1 写数据时任务 2 不能读数据，任务 2 读数据时任务 1 不能写数据。

1.5.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\kernel_05_mutex_semaphore】



1.5.4 实验原理

信号量（Semaphore）是一种实现任务间通信的机制，可以实现任务间同步或共享资源的互斥访问。

一个信号量的数据结构中，通常有一个计数值用于对有效资源数的计数，表示剩下的可被使用的共享资源数，其值的含义分两种情况：

1. 表示该信号量当前不可获取，因此可能存在正在等待该信号量的任务。
2. 正值，表示该信号量当前可被获取。

当任务申请信号量时，如果申请成功，则信号量的计数值递减，如果申请失败，则挂起在该信号量的等待任务队列上，一旦有任务释放该信号量时，则等待任务队列中的任务被唤醒开始执行。

运行机制如下图所示：

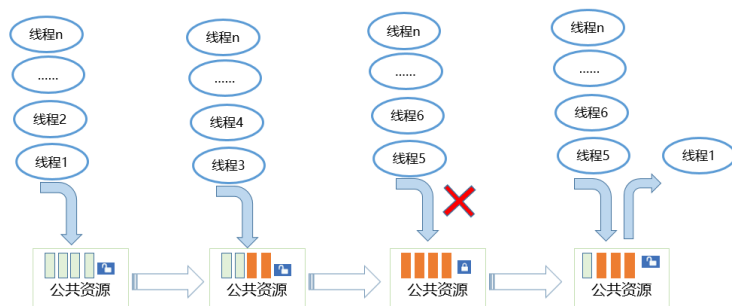


图2-11 信号量运作机制

信号量还有一些应用场景，可以用做多种场合中，比如可以实现互斥锁、同步、资源技术等功能，也能方便用于任务与任务，中断与任务的同步中。

● 互斥锁

用做互斥时，信号量在创建的时候，将最大值设置为 1，初始值设置为 1，可以实现两个任务之间的资源进行互斥。如果想要使用临界资源时，先申请信号量，使其变为 0，这样其他任务就会因为无法申请到信号量而阻塞，从而保证了临界资源的安全，在使用完临界资源之后，进行释放信号量。

1.5.5 实验步骤

步骤1: 编写应用程序

首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_05_mutex_semaphore”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_mutex_semaphore_example.c 两个文件。

1. 编写程序代码。打开“applications/sample/wifi-iot/app/kernel_05_mutex_semaphore/kernel_mutex_semaphore_example.c”文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。

```
1 uint8_t buff[20] = {0};           // 定义一个共享资源
2
3 /**
4  * @description: 任务 1
5  * @param {*}
6  */
```



```

6  * @return {*}
7  */
8 void Task1(void *argument)
9 {
10 (void)argument;
11 while (1)
12 {
13     printf("[info] enter Task 1.....\n");
14     osSemaphoreAcquire(Semaphore_ID, osWaitForever); // 请求信号量 -1
15
16     // 操作共享数据 写数据
17     printf("[info] write Buff Data: \n");
18     for (size_t i = 0; i < sizeof(buff); i++)
19     {
20         buff[i] = i;
21     }
22     printf("\n");
23     osSemaphoreRelease(Semaphore_ID); // 释放信号量 +1
24
25     sleep(1);
26 }
27 }
28 /**
29 * @description: 任务 2
30 * @param {*}
31 * @return {*}
32 */
33 void Task2(void *argument)
34 {
35 (void)argument;
36 while (1)
37 {
38     printf("enter Task 2.....\n");
39     osSemaphoreAcquire(Semaphore_ID, osWaitForever); // 请求信号量 -1
40     // 操作共享数据 读数据
41     printf("[info] read Buff Data: \n");
42     for (size_t i = 0; i < sizeof(buff); i++)
43     {
44         printf("%d \n", buff[i]);
45     }
46     printf("\n");
47     osSemaphoreRelease(Semaphore_ID); // 释放信号量 +1
48
49     sleep(1);

```



```
50 }
51 }
```

2. 编写用于将逻辑代码构建程静态库的“applications/sample/wifi-iot/app/kernel_05_mutex_semaphore/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```
1 static_library("kernel_mutex_semaphore_example") {
2     sources = [
3         "kernel_mutex_semaphore_example.c"
4     ]
5
6     include_dirs = [
7         "//utils/native/lite/include",
8         "//kernel/liteos_m/kal/cmsis",
9     ]
10 }
11
```

3. 编写“applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```
1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         "kernel_05_mutex_semaphore:kernel_mutex_semaphore_example",
7     ]
8 }
9
```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

1.5.6 实验现象

重启 hi3861 模组之后，任务 1 写数据，任务 2 读数据，这是一个顺序执行的过程。

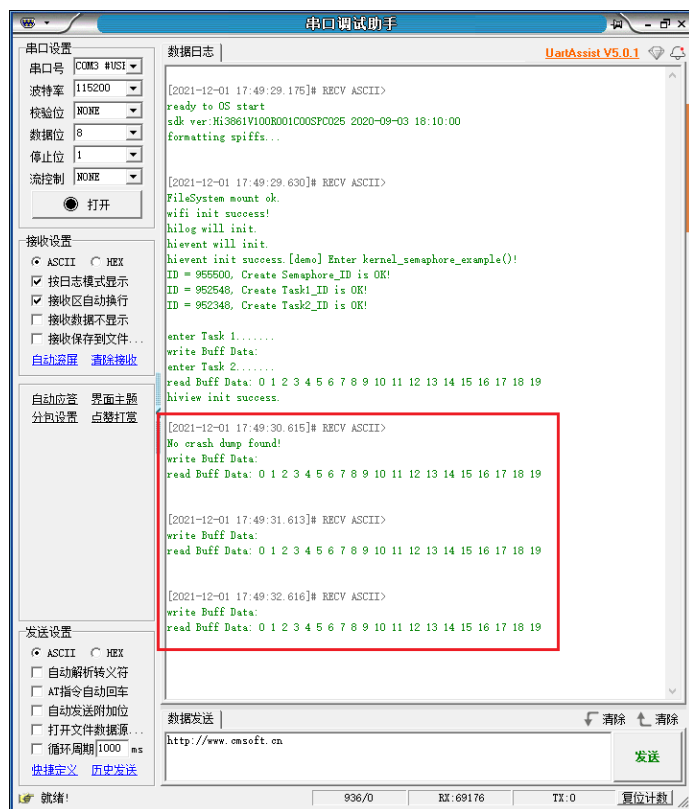


图2-12 实验串口打印结果

1.6 HarmonyOS 之同步型信号量实验

1.6.1 实验目的

1. 熟悉 HarmonyOS 系统下信号量相关函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.6.2 实验要求

创建两个任务，任务 1 请求信号量，任务 2 释放信号量。

1.6.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\kernel_06_sync_semaphore】

1.6.4 实验原理

信号量（Semaphore）是一种实现任务间通信的机制，可以实现任务间同步或共享资源的互斥访问。一个信号量的数据结构中，通常有一个计数值用于对有效资源数的计数，表示剩下的可被使用的共享



资源数，其值的含义分两种情况：

1. 表示该信号量当前不可获取，因此可能存在正在等待该信号量的任务。
2. 正值，表示该信号量当前可被获取。

当任务申请信号量时，如果申请成功，则信号量的计数值递减，如果申请失败，则挂起在该信号量的等待任务队列上，一旦有任务释放该信号量时，则等待任务队列中的任务被唤醒开始执行。

运行机制如下图所示：

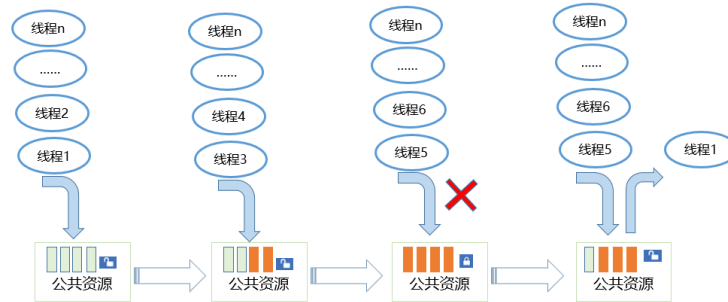


图2-13 信号量运作机制

信号量还有一些应用场景，可以用做多种场合中，比如可以实现互斥锁、同步、资源技术等功能，也能方便用于任务与任务，中断与任务的同步中。

● 任务与任务之间同步

用做同步时，信号量在创建时初始值设置为 0，任务 1 要申请信号量而阻塞，任务 2 可以释放信号量，于是任务 1 得以进入等待态和运行态，从而达到了两个任务间的同步。

1.6.5 实验步骤

步骤1: 编写应用程序

首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_06_sync_semaphore”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_sync_semaphore_example.c 两个文件。

1. 编写程序代码。打开“applications/sample/wifi-iot/app/ kernel_06_sync_semaphore/kernel_sync_semaphore_example.c”文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。

```
1 /**
2  * @description: 任务 1
3  * @param {*}
4  * @return {*}
5  */
6 void Task1(void *argument)
7 {
8     (void)argument;
9     while (1)
10    {
```



```

11 // 请求信号量 -1
12 osSemaphoreAcquire(Semaphore_ID, osWaitForever);
13 printf("Task 1 osSemaphore Acquire <=====\n");
14
15 sleep(1);
16 }
17 }
18 /**
19 * @description: 任务 2
20 * @param {*}
21 * @return {*}
22 */
23 void Task2(void *argument)
24 {
25     (void)argument;
26     while (1)
27     {
28         osSemaphoreRelease(Semaphore_ID); // 释放信号量 +1
29         printf("Task 2 osSemaphore Release =====>\n");
30         sleep(2);
31     }
32 }

```

2. 编写用于将逻辑代码构建程静态库的“applications/sample/wifi-iot/app/kernel_06_sync_semaphore/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("kernel_sync_semaphore_example") {
2     sources = [
3         "kernel_sync_semaphore_example.c"
4     ]
5
6     include_dirs = [
7         "//utils/native/lite/include",
8         "//kernel/liteos_m/kal/cmsis",
9     ]
10 }

```

3. 编写“applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         "kernel_06_sync_semaphore:kernel_sync_semaphore_example",
7     ]

```



8
9}

]

步骤2: 编译工程

按照第 1 章 开发环境搭建中“编译源码”小节的内容进行编译工程。

步骤3: 烧写程序

按照第 1 章 开发环境搭建中“烧写程序”小节的内容进行烧写程序。

1.6.6 实验现象

重启 hi3861 模组之后，任务 1 请求信号量，任务 2 释放信号量。

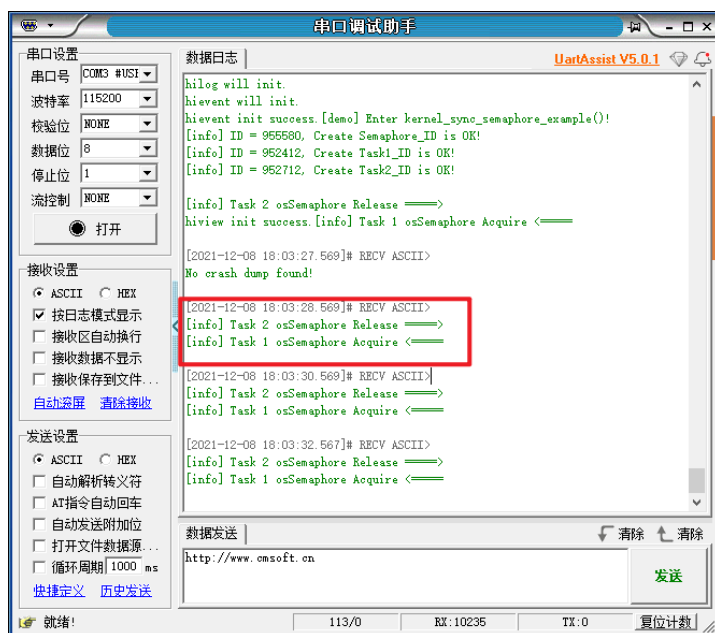


图2-14 实验串口打印结果

1.7 HarmonyOS 之计数型信号量实验

1.7.1 实验目的

1. 熟悉 HarmonyOS 系统下信号量相关函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.7.2 实验要求

创建两个任务，模拟一个停车场，停车场的容量为 10 个，任务 1 中每隔一秒释放信号量，信号量的计数值会加 1（相当于进入了一辆车）；任务 2 中每隔两秒钟请求信号量，信号量的计数值会减 1（相当于出去了 1 辆车）。



1.7.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\kernel_07_count_semaphore】

1.7.4 实验原理

信号量（Semaphore）是一种实现任务间通信的机制，可以实现任务间同步或共享资源的互斥访问。

一个信号量的数据结构中，通常有一个计数值用于对有效资源数的计数，表示剩下的可被使用的共享资源数，其值的含义分两种情况：

1. 表示该信号量当前不可获取，因此可能存在正在等待该信号量的任务。
2. 正值，表示该信号量当前可被获取。

当任务申请信号量时，如果申请成功，则信号量的计数值递减，如果申请失败，则挂起在该信号量的等待任务队列上，一旦有任务释放该信号量时，则等待任务队列中的任务被唤醒开始执行。

运行机制如下图所示：

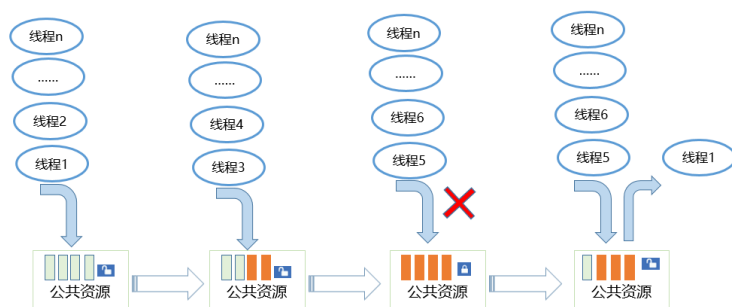


图2-15 信号量运作机制

信号量还有一些应用场景，可以用做多种场合中，比如可以实现互斥锁、同步、资源技术等功能，也能方便用于任务与任务，中断与任务的同步中。

● 计数型信号量

用做资源计数时，信号量的作用是一个特殊的计数器，可以递增或递减，但是此值不能为负值，典型的应用场景是生产者与消费者的场景中。

1.7.5 实验步骤

步骤1: 编写应用程序

首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_07_count_semaphore”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_count_semaphore_example.c 两个文件。

1. 编写程序代码。打开“applications/sample/wifi-iot/app/kernel_05_count_semaphore/kernel_count_semaphore_example.c”文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。



```

1 #define SEM_MAX_COUNT 10
2
3 /**
4  * @description: 任务1 模拟停车场的入口
5  * 每隔一秒钟，停车场就会进入一辆车
6  * @param {*}
7  * @return {*}
8  */
9 void Task1(void *argument)
10 {
11     (void)argument;
12     while (1)
13     {
14         if (osSemaphoreGetCount(Semaphore_ID) < SEM_MAX_COUNT)
15         {
16             if (osSemaphoreRelease(Semaphore_ID) == osOK) // 释放信号量 +1
17                 printf("[进入%d辆车，停车场容量: %d] 信号量+1.\n",
18                     osSemaphoreGetCount(Semaphore_ID), SEM_MAX_COUNT);
19         }
20         else
21             printf("[进入停车场失败，请等待...]\n");
22
23         sleep(1);
24     }
25 }
26 /**
27  * @description: 任务2
28  * @param {*}
29  * @return {*}
30  */
31 void Task2(void *argument)
32 {
33     (void)argument;
34     while (1)
35     {
36         if (osSemaphoreAcquire(Semaphore_ID, osWaitForever) == osOK) // 释放信号量 -1
37             printf("[出去1辆车，剩余停车场容量: %d] 信号量-1.\n",
38                 osSemaphoreGetCount(Semaphore_ID));
39         else
40             printf("[出停车场失败]\n");
41
42         sleep(2);
43     }
44 }

```



2. 编写用于将逻辑代码构建程静态库的“applications/sample/wifi-iot/app/kernel_05_count_semaphore/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```
1 static_library("kernel_count_semaphore_example") {
2     sources = [
3         "kernel_count_semaphore_example.c"
4     ]
5
6     include_dirs = [
7         "//utils/native/lite/include",
8         "//kernel/liteos_m/kal/cmsis",
9     ]
10 }
11
```

3. 编写“applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```
1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         "kernel_07_count_semaphore:kernel_count_semaphore_example",
7     ]
8 }
9
```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

1.7.6 实验现象

重启 hi3861 模组之后，刚开始的时候，当前计数置为 0，任务 1 每隔 1 秒释放一次信号量（+1）。

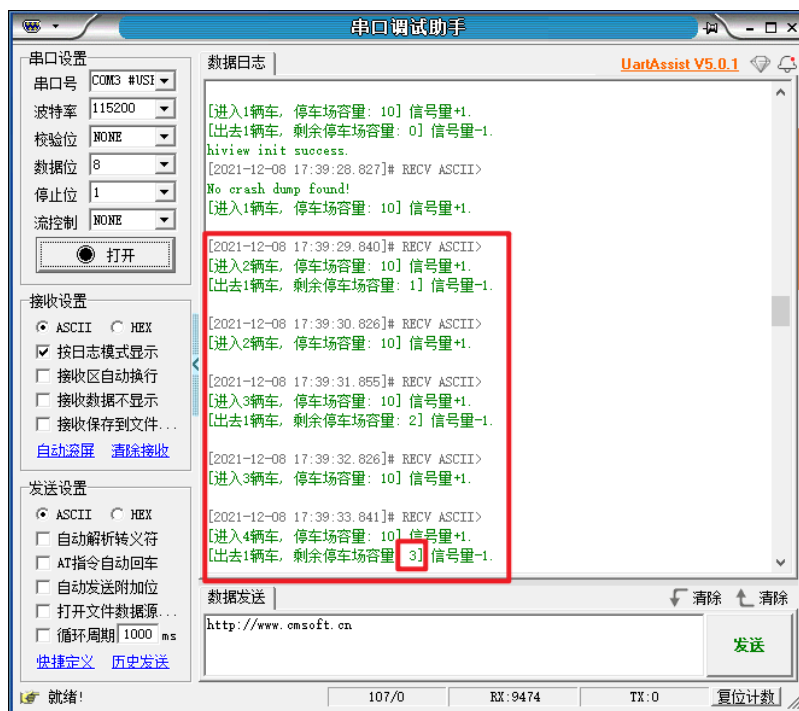


图2-16 实验串口打印结果

1.8 HarmonyOS 之消息队列实验

1.8.1 实验目的

1. 熟悉 HarmonyOS 系统下消息队列相关函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

1.8.2 实验要求

创建两个任务，任务 1 将数据放进消息队列中，任务 2 如果数据满了，会将消息队列中的数据进行出队，出队之后，打印信息。

1.8.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\kernel_08_message_queue】

1.8.4 实验原理

队列又称消息队列，是一种常用于任务间通信的数据结构。队列接收来自任务或中断的不固定长度消息，并根据不同的接口确定传递的消息是否存放在队列空间中。

任务能够从队列里面读取消息，当队列中的消息为空时，挂起读取任务；当队列中有新消息时，挂起



的读取任务被唤醒并处理新消息。任务也能够往队列里写入消息，当队列已经写满消息时，挂起写入任务；当队列中有空闲消息节点时，挂起的写入任务被唤醒并写入消息。

可以通过调整读队列和写队列的超时时间来调整读写接口的阻塞模式，如果将读队列和写队列的超时时间设置为 0，就不会挂起任务，接口会直接返回，这就是非阻塞模式。反之，如果将读队列和写队列的超时时间设置为大于 0 的时间，就会以阻塞模式运行。

消息队列提供了异步处理机制，允许将一个消息放入队列，但不立即处理。同时队列还有缓冲消息的作用，可以使用队列实现任务异步通信，队列具有如下特性：

1. 消息以先进先出的方式排队，支持异步读写。
2. 读队列和写队列都支持超时机制。
3. 每读取一条消息，就会将该消息节点设置为空闲。
4. 发送消息类型由通信双方约定，可以允许不同长度（不超过队列的消息节点大小）的消息。
5. 一个任务能够从任意一个消息队列接收和发送消息。
6. 多个任务能够从同一个消息队列接收和发送消息。
7. 创建队列时所需的队列空间，接口内系统自行动态申请内存。

运行机制如下图所示：

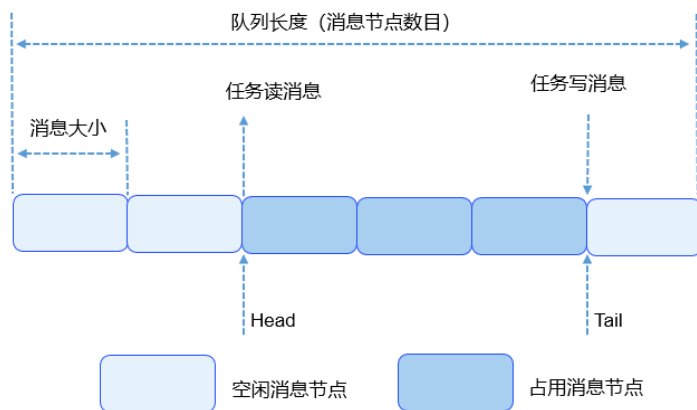


图2-17 消息队列运行机制

1.8.5 实验步骤

步骤1: 编写应用程序

首先第一步，在“applications/sample/wifi-iot/app”目录下，创建“kernel_08_message_queue”目录，然后在这个目录下，新建 BUILD.gn 和 kernel_message_queue_example.c 两个文件。

1. 编写程序代码。打开“applications/sample/wifi-iot/app/kernel_08_message_queue/kernel_message_queue_example.c”文件，填写如下代码：

注意：下面显示出主要代码，其余的请查看源码。

```
1 #define MsgQueueObjectNumber 16 // 定义消息队列对象的个数
2 typedef struct message_people
```



```

3{
4  uint8_t id;      // ID
5  uint8_t age;     // 年龄
6  char *name;      // 名字
7} msg_people_t;
8msg_people_t msg_people;
9
10/**
11 * @description: 任务 1 发送消息
12 * @param {*}
13 * @return {*}
14 */
15void Task1(void *argument)
16{
17  (void)argument;
18  osStatus_t msgStatus;
19  while (1)
20  {
21    printf("enter Task 1.....\n");
22    msg_people.id = 0;
23    msg_people.age = 16;
24    msg_people.name = "xiao_ming";
25    msgStatus = osMessageQueuePut(MsgQueue_ID, &msg_people, 0, 100);
26    if(msgStatus == osOK)
27    {
28      printf("osMessageQueuePut is ok.\n");
29    }
30    sleep(1);
31  }
32}
33/**
34 * @description: 任务 2 接收消息
35 * @param {*}
36 * @return {*}
37 */
38void Task2(void *argument)
39{
40  (void)argument;
41  osStatus_t msgStatus;
42  while (1)
43  {
44    printf("enter Task 2.....\n");
45    msgStatus = osMessageQueueGet(MsgQueue_ID, &msg_people, 0, 100);
46    if(msgStatus == osOK)

```



```

47 {
48     printf("osMessageQueueGet is ok.\n");
49     printf("Recv: id = %d, age = %d, name = %s\n",
50         msg_people.id,
51         msg_people.age,
52         msg_people.name);
53 }
54 }
55 }

```

2. 编写用于将逻辑代码构建程静态库的“applications/sample/wifi-iot/app/kernel_08_message_queue/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("kernel_message_queue_example") {
2     sources = [
3         "kernel_message_queue_example.c"
4     ]
5
6     include_dirs = [
7         "../utils/native/lite/include",
8         "../kernel/liteos_m/kal/cmsis",
9     ]
10 }
11

```

3. 编写“applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("../build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 内核相关实验
6         "kernel_08_message_queue:kernel_message_queue_example",
7     ]
8 }
9

```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

1.8.6 实验现象

重启 hi3861 模组之后，任务 1 中，将数据放进消息队列中，在任务 2 中，如果数据了，会将消息队列



中的数据进行出队，出队之后，打印信息。

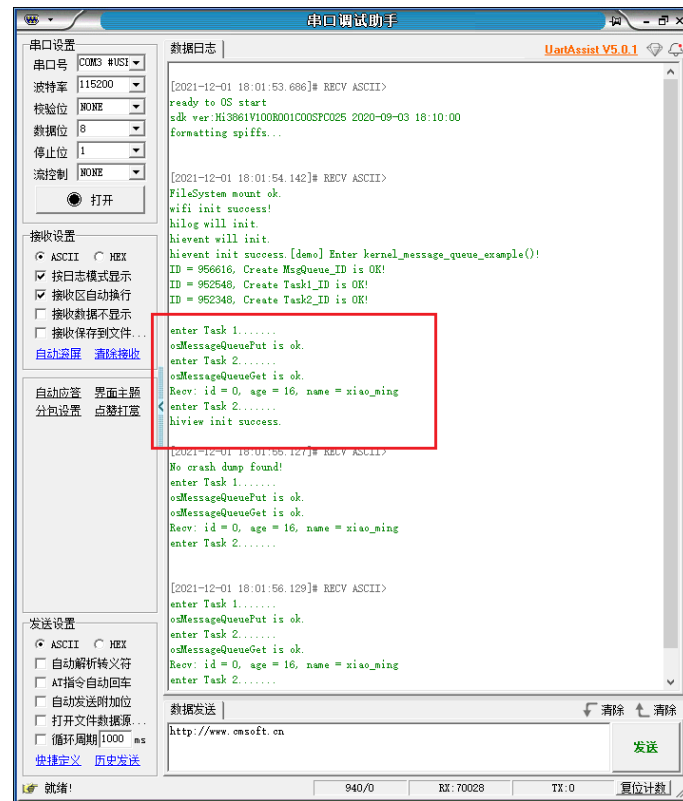


图2-18 实验串口打印结果



第2章 HarmonyOS 之基础实验系列

2.1 GPIO 之 KEY 按键实验

2.1.1 实验目的

1. 熟悉 HarmonyOS 系统下 GPIO 函数和 GPIO 中断函数的使用。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

2.1.2 实验要求

当按键按下时，在串口调试助手上显示 “HI_GPIO_VALUE_1”；

当按键松开时，在串口调试助手上显示 “HI_GPIO_VALUE_0”；

并且，当按键按下时，触发 GPIO 下降沿中断，此时在串口调试助手上显示 “key down...”。

2.1.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_01_key】

2.1.4 实验原理

在本实验中，使用 WiFi 模组的 IO 口作为按键的检测引脚，根据原理图，用户按键 KEY 接在了 WiFi 模组的 IO14 引脚上，当按键为松开状态，此时 IO14 引脚检测到高电平，所以我们在程序中设置 IO14 引脚的中断为下降沿检测时触发，并且当按键按下时，IO14 引脚为低电平，才能进入到回调函数中，打印输出 “key down...”。

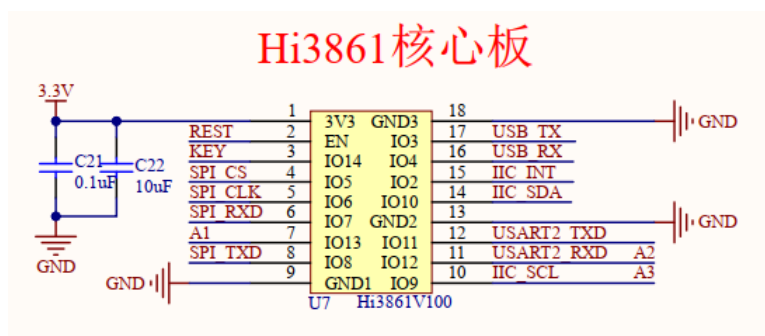


图3-1 Hi3861 WiFi 核心板

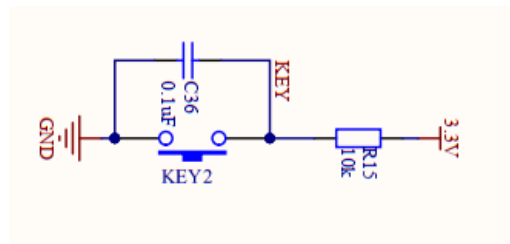


图3-2 用户按键 KEY 原理图

在使用接口函数的时候，在系统配置文件中，已经将 sdk_liteos 编译到系统镜像中，我们只需要进行在我们的例程代表中进行调用即可。

下图为头文件（路径：/device/hisilicon/hispark_pegasus/sdk_liteos/include/）：

```
[09:27:51] linux@linux:include$ ls
hi3861.h          hi_blackbox.h    hi_diag.h        hi_flash_base.h  hi_io.h          hi_mux.h         hi_sem.h         hi_tsensor.h     hi_wifi_apl.h    hks_types.h
hi3861_platform_base.h  hi_btcoex_apl.h  hi_dma.h        hi_flash.h       hi_isr.h         hi_net_apl.h     hi_shell.h       hi_types_base.h  hi_wifi_cst_apl.h
hi3861_platform.h    hi_cipher.h       hi_dumper_adapter.h  hi_fs.h         hi_list.h        hi_nv.h         hi_spi.h        hi_types.h       hi_wifi_sdp_apl.h
hi_adc.h            hi_clock.h       hi_dumper.h      hi_gpio.h        hi_lowpower.h    hi_os_stat.h    hi_stdlib.h     hi_uart.h       hi_workqueue.h
hi_any_apl.h        hi_config.h      hi_early_debug.h  hi_hook_fuc.h   hi_mdio.h        hi_partition_table.h  hi_systick.h    hi_upg_apl.h    hks_client.h
hi_at.h             hi_cpu.h         hi_efuse.h       hi_hrtimer.h    hi_mdio_types.h  hi_pwm.h        hi_task.h       hi_upg_file.h   hks_device_provision.h
hi_atomic.h         hi_crash.h       hi_errno.h       hi_i2c.h        hi_men.h         hi_reset.h      hi_time.h       hi_ver.h        hks_sereno.h
hi_blackbox_adapter.h  hi_crc.h         hi_event.h       hi_i2s.h        hi_msg.h         hi_sdio_device.h  hi_timer.h     hi_watchdog.h   hks_file_apl.h
```

图3-3 外设接口头文件

下图为源文件（路径：/device/hisilicon/hispark_pegasus/sdk_liteos/platform/drivers/）：

```
[09:39:10] linux@linux:drivers$ ls
adc  cipher  dma  flash  i2c  Makefile  module_config.mk  pwm  SConscript  sdio  spi  tsensor  uart
```

图3-4 外设接口源文件

2.1.5 实验步骤

步骤1: 编写应用程序

1. 在“/applications/sample/wifi-iot/app/”目录中创建“base_01_key”目录，在“base_01_key”目录中创建“base_key_example.c”文件和“BUILD.gn”文件。

2. 编写“base_key_example.c”文件：

注意：下面显示出主要代码，其余的请查看源码。

```
1#include <stdio.h>
2#include <unistd.h>
3
4#include "ohos_init.h"
5#include "cmsis_os2.h"
6#include "hi_io.h"
7#include "hi_gpio.h"
8#include "hal_bsp_log.h"
9
10#define KEY HI_IO_NAME_GPIO_14 // WiFi 模组的 IO14 引脚
11
12osThreadId_t Task1_ID = 0; // 任务 1 ID
```



```

13
14 /**
15  * @brief 按键中断回调函数
16  * @note 当按键按下时候才会触发
17  * @param *arg:
18  * @retval None
19  */
20 hi_void gpio_callback(hi_void *arg)
21 {
22     console_log_info("key down...");
23 }
24 /**
25  * @description: 任务 1
26  * @param {*}
27  * @return {*}
28  */
29 void Task1(void *argument)
30 {
31     (void)argument;
32     hi_gpio_value val, val_last; // GPIO 的状态值
33     console_log_info("enter Task 1.....");
34     hi_gpio_init(); // GPIO 初始化
35     hi_io_set_pull(KEY, HI_IO_PULL_UP); // 设置 GPIO 上拉
36     hi_io_set_func(KEY, HI_IO_FUNC_GPIO_14_GPIO); // 设置 IO14 为 GPIO 功能
37     hi_gpio_set_dir(KEY, HI_GPIO_DIR_IN); // 设置 GPIO 为输入模式
38     hi_gpio_register_isr_function(KEY, // KEY 按键引脚
39                                     HI_INT_TYPE_EDGE, // 下降沿检测
40                                     HI_GPIO_EDGE_FALL_LEVEL_LOW, // 低电平时触发
41                                     &gpio_callback, // 触发后调用的回调函数
42                                     NULL); // 回调函数的传参值
43     while (1)
44     {
45         hi_gpio_get_input_val(KEY, &val); // 获取 GPIO 引脚的状态
46         if(val != val_last) // 当 GPIO 状态改变的时候, 打印输出
47         {
48             console_log_info("keyValue: %s", val ? "HI_GPIO_VALUE_1" : "HI_GPIO_VALUE_0");
49             val_last = val;
50         }
51         // 200ms 一检测
52         usleep(200 * 1000); // 200ms sleep
53     }
54 }
55

```

3. 编写用于将逻辑代码构建程静态库的“/applications/sample/wifi-iot/app/base_01_key/BUILD.gn”文



件。打开 BUILD.gn 文件，填入一下代码：

```
1 static_library("base_key_example") {
2     sources = [
3         "base_key_example.c",
4     ]
5
6     include_dirs = [
7         "../utils/native/lite/include",
8         "../kernel/liteos_m/kal/cmsis",
9         "../base/iot_hardware/peripheral/interfaces/kits",
10    ]
11 }
12
13
```

4. 编写 “/applications/sample/wifi-iot/app/BUILD.gn” 文件，打开 BUILD.gn 文件，填入一下代码：

```
1 import("../build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         "base_01_key:base_key_example",
7     ]
8 }
```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

2.1.6 实验现象

当按键按下的时候，在串口调试助手上显示 “HI_GPIO_VALUE_1”；当按键松开的时候，在串口调试助手上显示 “HI_GPIO_VALUE_0”；并且，当按键按下的时候，触发 GPIO 下降沿中断，此时在串口调试助手是上显示 “key down...”。

效果图如下图所示：

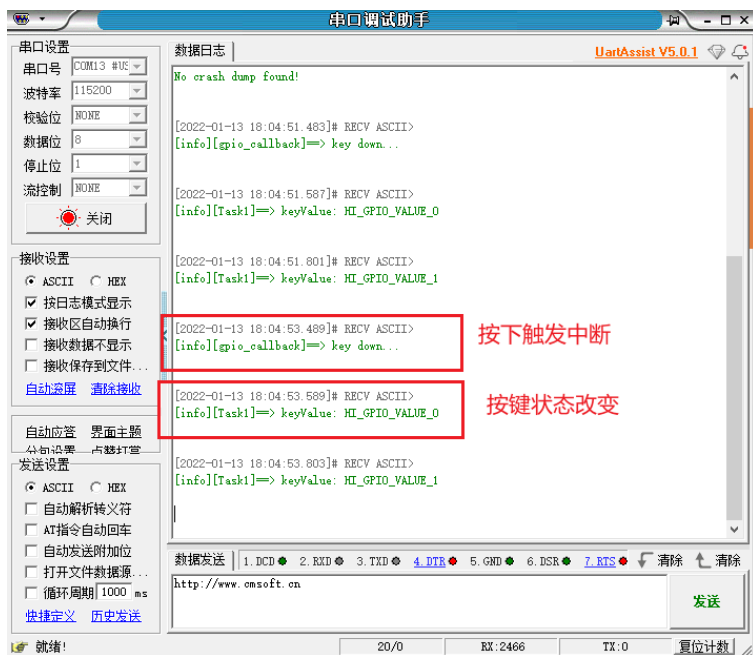


图3-5 串口调试助手打印

2.2 I2C 之 IO 扩展芯片-驱动风扇实验

2.2.1 实验目的

1. 学会 I2C 总线与 pcf8574 芯片进行通信。
2. 熟悉 HarmonyOS 系统下 I2C 通信和相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.2.2 实验要求

使用 I2C 与 PCF8574 芯片进行通信，实现风扇转 5 秒，停 5 秒的现象。

2.2.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：**【程序源码\基础实验\base_02_i2c_pcf8574_fan】**

2.2.4 实验原理

在本实验中，使用 IO 扩展 PCF8574 芯片进行扩展 Hi3861 的引脚，Hi3861 与 PCF8574 芯片之间使用的 I2C 总线进行通信。在 Hi3861 开发板中使用的是小型直流风扇，根据原理图，可以得知当 IO 扩展芯片的 P0 引脚为高电平时，风扇为打开状态；当 IO 扩展芯片的 P0 引脚为低电平时，风扇为关闭状态。下面给出了硬件连接图：

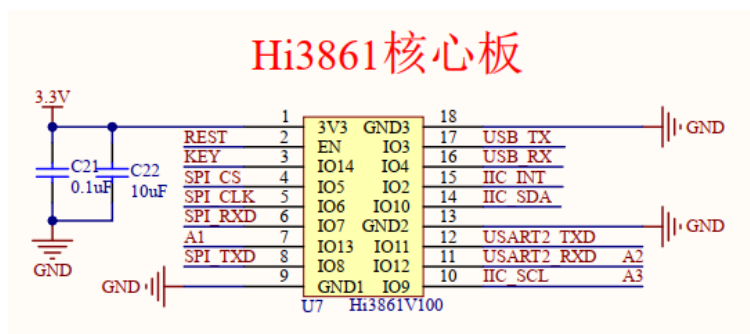


图3-6 Hi3861 核心板电路图

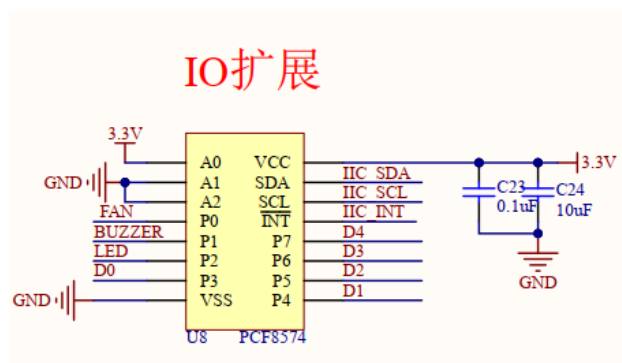


图3-7 PCF8574 芯片电路原理图

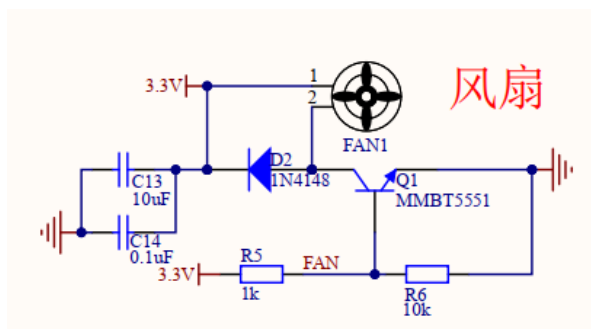
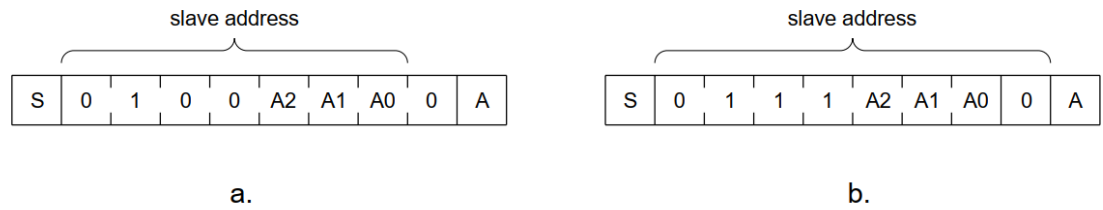


图3-8 风扇电路原理图

1. 根据 PCF8574 芯片手册，找到所需要的数据。
- 1) 找到 I2C 的 7 位从机地址为 0x21，如下图所示：



(a) PCF8574.
(b) PCF8574A.

Fig.9 PCF8574 and PCF8574A slave addresses.

图3-9 PCF8574 芯片的从机地址

2) 写 pcf8574 端口的数据，如下图所示：

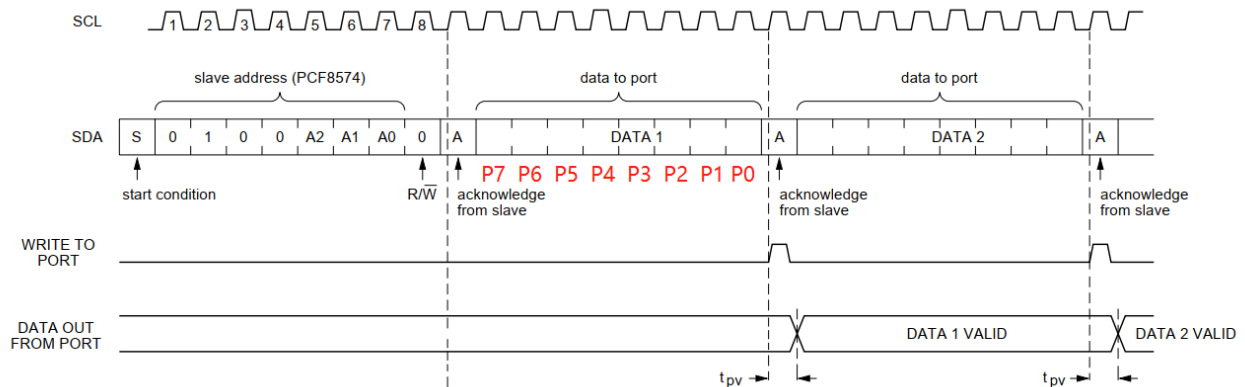


图3-10 PCF8574 芯片的写模式

3) 读 PCF8574 芯片端口的数据，如下图所示：

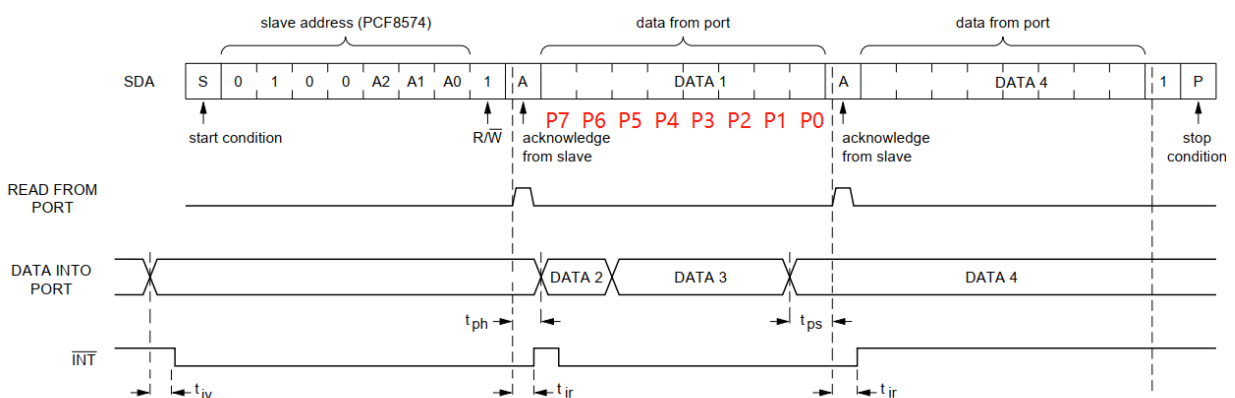


图3-11 PCF8574 芯片的读模式



2.2.5 实验步骤

步骤1: 编写 pcf8574 驱动

注意：下面显示出主要代码，其余的请查看源码。

1. 打开 “//device/hisilicon/hispark_pegasus/sdk_liteos/build/config/usr_config.mk” 文件，在此文件中，将 “CONFIG_I2C_SUPPORT” 取消注释，并加上 “=y”。

```
30  #
31  # BSP Settings
32  #
33  CONFIG_I2C_SUPPORT=y
```

图3-12 打开 I2C 的支持包

注意：比如今后可能还会用到 SPI、PWM、DMA 等，都需要在这个文件中进行修改。

2. 在 “base/iot_hardware/”目录下创建 bsp 目录，然后在 bsp 目录下创建 src 目录和 include 目录，然后在 src 目录下创建 “hal_bsp_pcf8574.c”文件和在 include 目录下创建 “hal_bsp_pcf8574.h”文件。下面为目录格式：

```
bsp
├── include
│   └── hal_bsp_pcf8574.h
└── src
    └── hal_bsp_pcf8574.c
```

3. 在 hal_bsp_pcf8574.c 文件中，编写驱动程序：

```
1 // 写 PCF8574 各个引脚的状态值
2 uint32_t PCF8574_Write(const uint8_t send_data)
3 {
4     uint32_t result = 0;
5     uint8_t buffer[2] = {send_data};
6
7     hi_i2c_data i2c_data = { 0 };
8     i2c_data.send_buf = buffer;
9     i2c_data.send_len = 1;
10
11     result = hi_i2c_write(PCF8574_I2C_ID, PCF8574_I2C_ADDR, &i2c_data);
12     if (result != HI_ERR_SUCCESS)
13     {
14         console_log_error("I2C PCF8574 Write result is 0x%x!!!", result);
15         return result;
16     }
17     return HI_ERR_SUCCESS;
18 }
```



```

19 // 读 PCF8574 各个引脚的状态值
20 uint32_t PCF8574_Read(uint8_t *recv_data)
21 {
22     uint32_t result = 0;
23     hi_i2c_data i2c_data = {0};
24     i2c_data.receive_buf = recv_data;
25     i2c_data.receive_len = 1;
26
27     result = hi_i2c_read(PCF8574_I2C_ID, PCF8574_I2C_ADDR, &i2c_data);
28     if (result != HI_ERR_SUCCESS)
29     {
30         console_log_error("I2C PCF8574 Read result is 0x%x!!!", result);
31         return result;
32     }
33
34     return HI_ERR_SUCCESS;
35 }

```

步骤2: 编写应用程序

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 base_02_i2c_pcf8574_fan 目录，在此目录中创建 base_fan_example.c 文件和 BUILD.gn 文件。
2. 编写 base_fan_example.c 文件:

```

1 #include <stdio.h>
2 #include <unistd.h>
3
4 #include "ohos_init.h"
5 #include "cmsis_os2.h"
6 #include "iot_i2c.h"
7 #include "iot_gpio.h"
8 #include "hal_bsp_pcf8574.h"
9 #include "hal_bsp_structAll.h"
10 #include "hal_bsp_log.h"
11
12 tn_pcf8574_io_t pcf8574_io = {0}; // IO 扩展芯片的引脚
13 osThreadId_t Task1_ID = 0; // 任务 1 ID
14 /**
15  * @description: 任务 1
16  * @param {*}
17  * @return {*}
18  */
19 void Task1(void *argument)
20 {
21     (void)argument;

```



```

22 console_log_info("enter Task 1.....");
23 while (1)
24 {
25     pcf8574_io.bit.p0 = 1;           // 风扇打开
26     PCF8574_Write(pcf8574_io.all);
27     sleep(5); // sleep for 5 seconds
28
29     pcf8574_io.bit.p0 = 0;           // 风扇关闭
30     PCF8574_Write(pcf8574_io.all);
31     sleep(5); // sleep for 5 seconds
32 }
33 }

```

3. 编写用于将逻辑代码构建程静态库的“/applications/sample/wifi-iot/app/base_02_i2c_pcf8574_fan/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("base_fan_example") {
2     sources = [
3         "base_fan_example.c",
4         "../base/iot_hardware/bsp/src/hal_bsp_pcf8574.c"
5     ]
6
7     include_dirs = [
8         "../utils/native/lite/include",
9         "../kernel/liteos_m/kal/cmsis",
10        "../base/iot_hardware/peripheral/interfaces/kits",
11        "../base/iot_hardware/bsp/include"
12    ]
13 }

```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("../build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         "base_02_i2c_pcf8574_fan: base_fan_example",
7     ]
8 }

```

步骤3: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤4: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。



2.2.6 实验现象

重启开发板，此时风扇会转 5 秒，停 5 秒。

2.3 I2C 之 IO 扩展芯片-驱动蜂鸣器实验

2.3.1 实验目的

1. 学会 I2C 总线与 pcf8574 芯片进行通信。
2. 熟悉 HarmonyOS 系统下 I2C 通信和相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.3.2 实验要求

使用 I2C 与 PCF8574 芯片进行通信，实现蜂鸣器响 5 秒，停 5 秒。

2.3.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_03_i2c_pcf8574_buzzer】

2.3.4 实验原理

在本实验中，使用 IO 扩展 PCF8574 芯片进行扩展 Hi3861 的引脚，Hi3861 与 PCF8574 芯片之间使用的 I2C 总线进行通信。在 Hi3861 开发板中使用的蜂鸣器为有源蜂鸣器，根据原理图，可以得知当 IO 扩展芯片的 P1 引脚为高电平时，蜂鸣器为打开状态；当 IO 扩展芯片的 P1 引脚为低电平时，蜂鸣器为关闭状态。下面给出了硬件连接图：

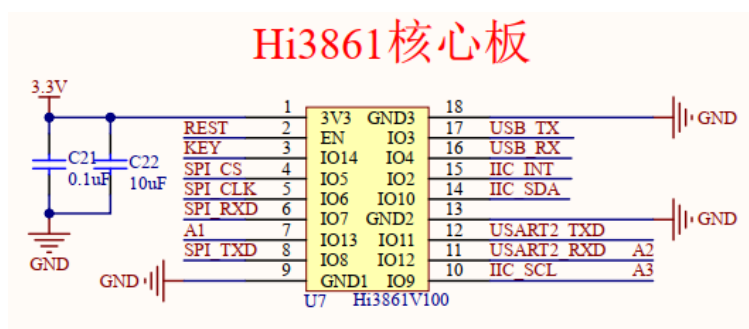


图3-13 Hi3861 核心板原理图

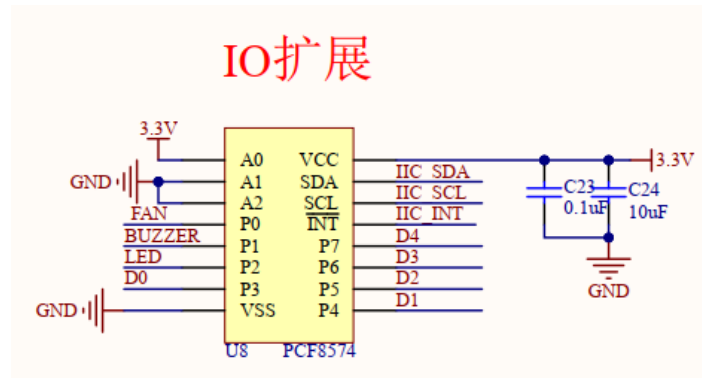


图3-14 IO 扩展芯片原理图

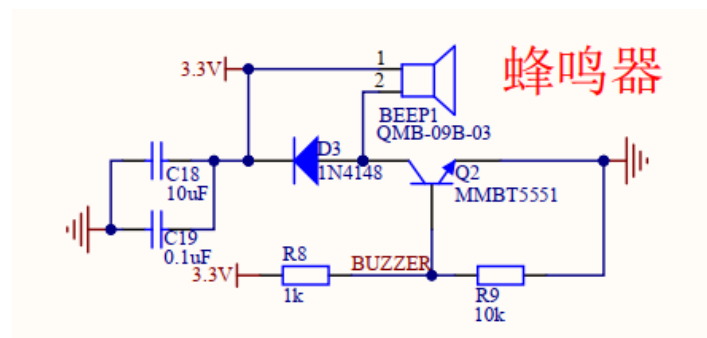


图3-15 蜂鸣器的原理图

2.3.5 实验步骤

步骤1: 编写应用程序

注意：下面显示出主要代码，其余的请查看源码。

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 base_03_i2c_pcf8574_buzzer 目录，在此目录中创建 base_buzzer_example.c 文件和 BUILD.gn 文件。

2. 编写 base_buzzer_example.c 文件：

```
1 tn_pcf8574_io_t pcf8574_io = {0}; // IO 扩展芯片的引脚
2 osThreadId_t Task1_ID = 0; // 任务 1 ID
3
4 /**
5  * @description: 任务 1
6  * @param {*}
7  * @return {*}
8  */
9 void Task1(void *argument)
10 {
11     (void)argument;
12     console_log_info("enter Task 1.....");
13     while (1)
14     {
```



```

15  pcf8574_io.bit.p1 = 1;      // 打开蜂鸣器
16  PCF8574_Write(pcf8574_io.all);
17  sleep(5);
18
19  pcf8574_io.bit.p1 = 0;      // 关闭蜂鸣器
20  PCF8574_Write(pcf8574_io.all);
21  sleep(5);
22  }
23 }

```

3. 编写用于将逻辑代码构建静态库的“/applications/sample/wifi-iot/app/base_03_i2c_pcf8574_buzzer/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("base_buzzer_example") {
2     sources = [
3         base_buzzer_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_pcf8574.c"
5     ]
6
7     include_dirs = [
8         "//utils/native/lite/include",
9         "//kernel/liteos_m/kal/cmsis",
10        "//base/iot_hardware/peripheral/interfaces/kits",
11        "//base/iot_hardware/bsp/include"
12    ]
13 }

```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         "base_03_i2c_pcf8574_buzzer: base_buzzer_example",
7     ]
8 }

```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

2.3.6 实验现象

重启开发板，此时蜂鸣器会每隔 5 秒响一次。



2.4 I2C 之 IO 扩展芯片-驱动 LED 灯实验

2.4.1 实验目的

1. 学会 I2C 总线与 pcf8574 芯片进行通信。
2. 熟悉 HarmonyOS 系统下 I2C 通信和相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.4.2 实验要求

使用 I2C 与 PCF8574 芯片进行通信，实现 LED 灯每隔一秒闪烁一次。

2.4.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_04_i2c_pcf8574_led】

2.4.4 实验原理

在本实验中，使用 IO 扩展 PCF8574 芯片进行扩展 Hi3861 的引脚，Hi3861 与 PCF8574 芯片之间使用的 I2C 总线进行通信。根据原理图，可以得知当 IO 扩展芯片的 P2 引脚为高电平时，LED 灯为熄灭状态；当 IO 扩展芯片的 P2 引脚为低电平时，LED 灯为点亮状态。下面给出了硬件连接图：

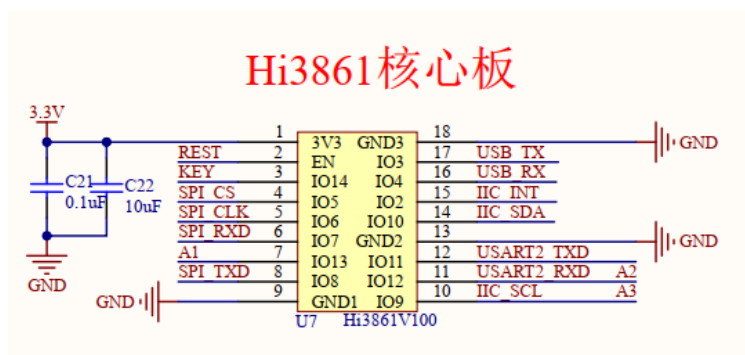


图3-16 Hi3861 核心板原理图



IO扩展

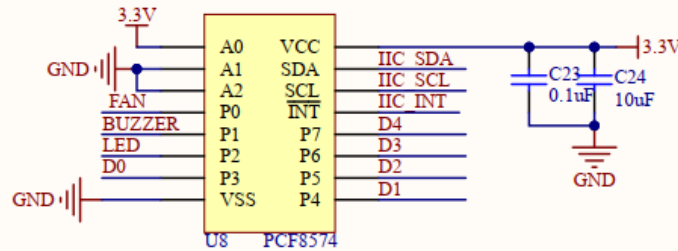


图3-17 IO 扩展原理图

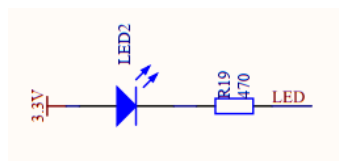


图3-18 LED 灯原理图

2.4.5 实验步骤

注意：下面显示出主要代码，其余的请查看源码。

步骤1: 编写应用程序

1. 在“/applications/sample/wifi-iot/app/”目录中创建 base_04_i2c_pcf8574_led 目录，在此目录中创建 base_led_example.c 文件和 BUILD.gn 文件。

2. 编写 base_led_example.c 文件：

```
1 tn_pcf8574_io_t pcf8574_io = {0}; // IO 扩展芯片的引脚
2 osThreadId_t Task1_ID = 0; // 任务 1 ID
3
4 /**
5  * @description: 任务 1
6  * @param {*}
7  * @return {*}
8  */
9 void Task1(void *argument)
10 {
11     (void)argument;
12     console_log_info("enter Task 1.....");
13     while (1)
14     {
15         pcf8574_io.bit.p2 = 1; // LED 灯熄灭
16         PCF8574_Write(pcf8574_io.all);
17         sleep(1);
```



```

18
19   pcf8574_io.bit.p2 = 0;    // LED 灯熄灭
20   PCF8574_Write(pcf8574_io.all);
21   sleep(1);
22 }
23 }

```

3. 编写用于将逻辑代码构建程静态库的“/applications/sample/wifi-iot/app/base_04_i2c_pcf8574_led/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("base_led_example") {
2     sources = [
3         base_led_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_pcf8574.c"
5     ]
6
7     include_dirs = [
8         "//utils/native/lite/include",
9         "//kernel/liteos_m/kal/cmsis",
10        "//base/iot_hardware/peripheral/interfaces/kits",
11        "//base/iot_hardware/bsp/include"
12    ]
13 }

```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         "base_04_i2c_pcf8574_led: base_led_example",
7     ]
8 }

```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

2.4.6 实验现象

重启开发板，进行查看结果：LED 灯会每隔 1 秒进行闪烁。



2.5 I2C 之驱动 RGB 灯实验

2.5.1 实验目的

1. 学会 I2C 总线与 AW2013 芯片进行通信。
2. 熟悉 HarmonyOS 系统下 I2C 通信和相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.5.2 实验要求

RGB 灯的三种颜色开始循环点亮。

2.5.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_05_i2c_aw2013】

2.5.4 实验原理

在本实验中，使用 I2C 总线与 RGB 灯芯片 AW2013 进行通信，AW2013 可以控制 RGB 三色灯的亮灭程度。下面给出了硬件连接图：



图3-19 Hi3861 核心板原理图

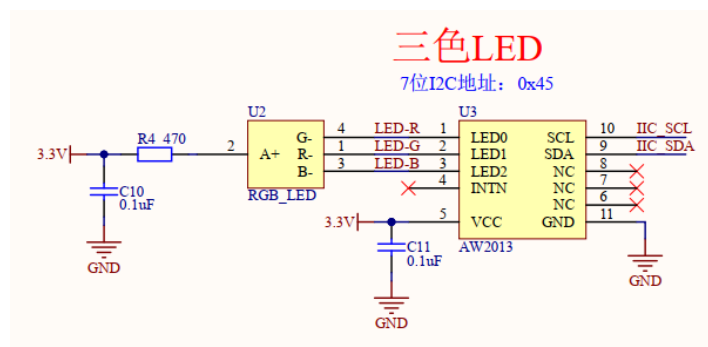


图3-20 RGB 灯电路原理图

1. 根据 AW2013 芯片手册，找到所需要的数据。
- 1) 可以找到 AW2013 芯片的从机地址，如下图所示：

6.2 I2C Address

The default I2C device address (7-bit) of AW2013 is 45h followed by the R/W bit(Read=1/Write=0), composites an slave address byte:

Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Device Address: 45h							R/W

图3-21 AW2013 芯片的从机地址

- 2) AW2013 芯片的 I2C 写时序，如下图所示：

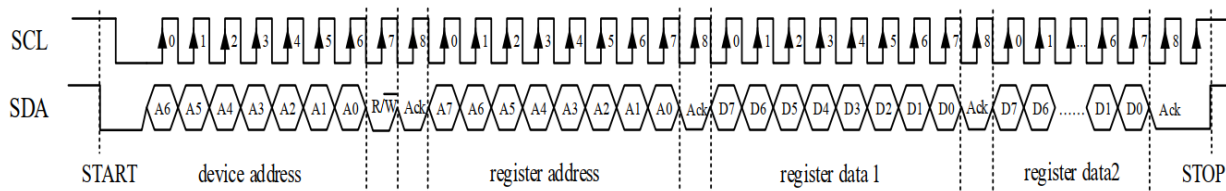


Figure 5 I2C write cycle, multiple registers are written

图3-22 AW2013 的写时序图

- 3) AW2013 芯片的读时序，如下图所示：

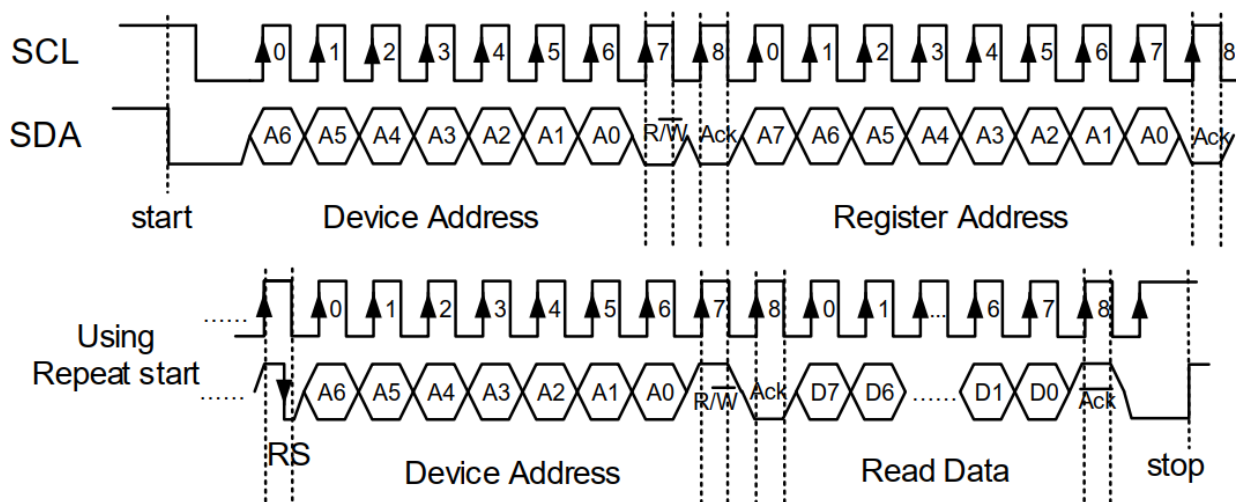


图3-23 AW2013 的读时序图

- 4) AW2013 芯片寄存器



Address	Name	Function	Default
00h	Soft Reset register, RSTR	Soft reset control	33h
01h	Global Control Register, GCR	Set Global control bits	00h
02h	Interrupt Status Register, ISR	Report interrupt status	00h
30h	LED Control Register, LCTR	Enable LED channels	00h
31h~33h	LED Mode Control Register, LCFG	Set working mode	00h
34~36h	PWM Setting Register, PWMx	Set brightness level	00h
37/3A/3Dh	LED Timing Control Register 0, LEDxT0	Set T1&T2 Timing	00h
38/3B/3Eh	LED Timing Control Register 1, LEDxT1	Set T3&T4 Timing	00h
39/3C/3Fh	LED Timing Control Register 2, LEDxT2	Set T0 and Repeat times	00h
77h	I2C address control register, IADR	Modify the device address for I2C bus	45h

图3-24 AW2013 的控制寄存器

2.5.5 实验步骤

注意：下面显示出主要代码，其余的请查看源码。

步骤1: 编写 AW2013 的驱动

1. 在“base/iot硬件/”目录下创建 bsp 目录，然后在 bsp 目录下创建 src 目录和 include 目录，然后在 src 目录下创建“hal_bsp_aw2013.c”文件和在 include 目录下创建“hal_bsp_aw2013.h”文件。下面为目录格式：

```

bsp
├── include
│   └── hal_bsp_aw2013.h
└── src
    └── hal_bsp_aw2013.c
    
```

2. 编写 aw2013 的驱动程序，具体驱动代码可在“/base/iot硬件/bsp/src/hal_bsp_aw2013.c”文件中进行查看。

步骤2: 编写应用程序

1. 在“/applications/sample/wifi-iot/app/”目录中创建 base_05_i2c_aw2013 目录，在此目录中创建 base_05_i2c_aw2013_example.c 文件和 BUILD.gn 文件。

2. 编写 base_05_i2c_aw2013_example.c 文件：

```

1 void Task1(void *argument)
2 {
3     while (1)
4     {
5         AW2013_Control_Red(255);
6         AW2013_Control_Green(0);
7         AW2013_Control_Blue(0);
8         usleep(100000); // 100 ms
    
```



```

9
10     AW2013_Control_Red(0);
11     AW2013_Control_Green(255);
12     AW2013_Control_Blue(0);
13     usleep(100000); // 100 ms
14
15     AW2013_Control_Red(0);
16     AW2013_Control_Green(0);
17     AW2013_Control_Blue(255);
18     usleep(100000); // 100 ms
19 }
20 }
```

3. 编写用于将逻辑代码构建静态库的“/applications/sample/wifi-iot/app/base_05_i2c_aw2013/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("base_aw2013_example") {
2     sources = [
3         "base_aw2013_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_aw2013.c"
5     ]
6
7     include_dirs = [
8         "//utils/native/lite/include",
9         "//kernel/liteos_m/kal/cmsis",
10        "//base/iot_hardware/peripheral/interfaces/kits",
11        "//base/iot_hardware/bsp/include"
12    ]
13 }
```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         "base_05_i2c_aw2013:base_aw2013_example",
7     ]
8 }
```

步骤3: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤4: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。



2.5.6 实验现象

烧写完成之后，重启开发板，此时查看开发板中的 RGB 灯的状态。如下图所示：

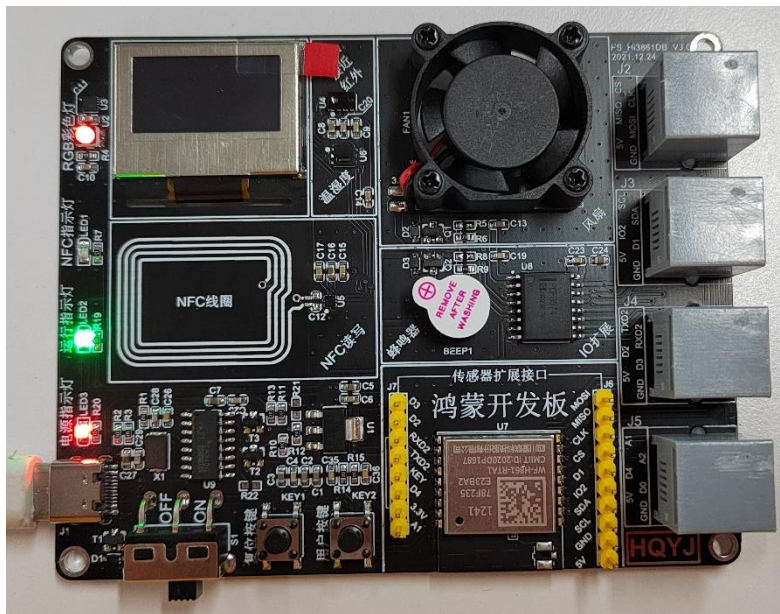


图3-25 RGB 三种颜色轮询闪烁

2.6 I2C 之温湿度传感器实验

2.6.1 实验目的

1. 学会 I2C 总线与 SHT20 芯片进行通信。
2. 熟悉 HarmonyOS 系统下 I2C 通信和相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.6.2 实验要求

在串口调试助手中显示当前温湿度的值。

2.6.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_06_i2c_sht20】

2.6.4 实验原理

在本实验中，使用 I2C 总线与 SHT20 温湿度传感器进行通信，读取当前空气温湿度的数据。下面给出了硬件连接图：

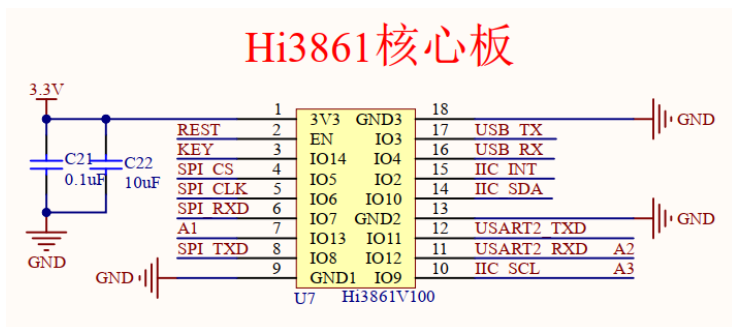


图3-26 Hi3861 核心板原理图

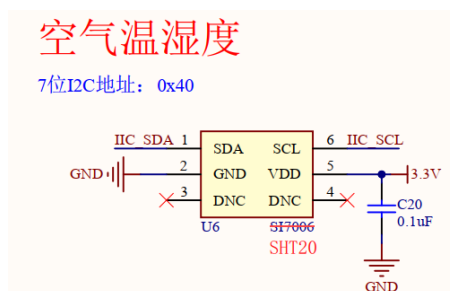


图3-27 SHT20 空气温湿度传感器原理图

1. 根据 SHT20 芯片手册，找到所需要的数据。

1) 找到 SHT20 的从机地址，可以得出 7 位 I2C 地址为 0x40，如下图所示：

5.3 Sending a Command

After sending the Start condition, the subsequent I²C header consists of the 7-bit I²C device address '1000'000' and an SDA direction bit (Read R: '1', Write W: '0'). The sensor indicates the proper reception of a byte by pulling the SDA pin low (ACK bit) after the falling edge of the 8th SCL clock. After the issue of a measurement command ('1110'0011' for temperature, '1110'0101' for relative humidity), the MCU must wait for the measurement to complete. The basic commands are summarized in Table 6.

图3-28 I2C 从机地址的描述

2) SHT20 芯片的写时序和读时序，先向芯片写命令，根据 AD 的分辨率得出采样时间，进行等待时间，然后再读取数据。

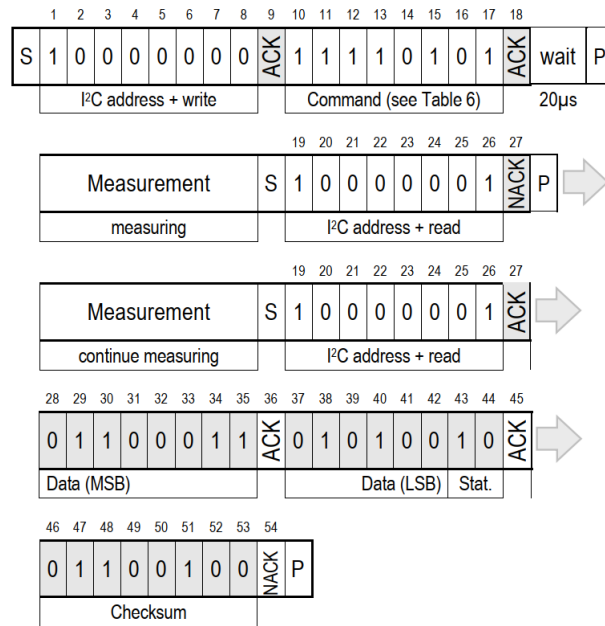


图3-29 写时序和写时序

3) AD 采样的延时时间计算，如下图所示，SHT20 芯片默认配置：湿度检测分辨率为 12 位，温度检测分辨率位 14 位。然后再看左侧的图标可以看到最大的延时时间：湿度检测最大延时时间为 29ms，湿度检测最大延时时间为 85ms。

Resolution	RH typ	RH max	T typ	T max	Units
14 bit			66	85	ms
13 bit			33	43	ms
12 Bit	22	29	17	22	ms
11 bit	12	15	9	11	ms
10 bit	7	9			ms
8 bit	3	4			ms

Table 7 Measurement times for RH and T measurements at different resolutions. Typical values are recommended for calculating energy consumption while maximum values shall be applied for calculating waiting times in communication.

Please note: I²C communication allows for repeated Start conditions (S) without closing prior sequence with Stop condition (P) – compare Figures 15, 16 and 18. Still, any sequence with adjacent Start condition may alternatively be closed with a Stop condition.

default and is not recommended for use. Please use Soft Reset instead – it contains OTP Reload.

Bit	# Bits	Description / Coding	Default		
7, 0	2	Measurement resolution	'00'		
				RH	T
		'00'		12 bit	14 bit
		'01'		8 bit	12 bit
		'10'		10 bit	13 bit
		'11'		11 bit	11 bit
6	1	Status: End of battery ¹³ '0': VDD > 2.25V '1': VDD < 2.25V	'0'		
3, 4, 5	3	Reserved			
2	1	Enable on-chip heater	'0'		
1	1	Disable OTP Reload	'1'		

图3-30 SHT20 传感器的配置

2.6.5 实验步骤

注意：下面显示出主要代码，其余的请查看源码。

步骤1: 编写 SHT20 的驱动

1. 在 “base/iot_hardware/”目录下创建 bsp 目录，然后在 bsp 目录下创建 src 目录和 include 目录，然后



在 src 目录下创建“hal_bsp_sht20.c”文件和在 include 目录下创建“hal_bsp_sht20.h”文件。下面为目录格式：

```

bsp
├── include
│   └── hal_bsp_sht20.h
└── src
    └── hal_bsp_sht20.c
    
```

2. 编写 SHT20 的驱动程序，具体驱动代码可在“/base/iot_hardware/bsp/src/hal_bsp_sht20.c”文件中进行查看。

步骤2: 编写应用程序

1. 在“/applications/sample/wifi-iot/app/”目录中创建 base_06_i2c_sht20 目录，在此目录中创建 base_sht20_example.c 文件和 BUILD.gn 文件。

2. 编写 base_sht20_example.c 文件：

```

1  void Task1(void *argument)
2  {
3  float temperature = 0, humidity = 0;
4  while (1)
5  {
6      SHT20_ReadData(&temperature, &humidity); // 读取温度值和湿度值
7      console_log_info("temperature = %.2f  humidity = %.2f",
8                      temperature,
9                      humidity); // 将检测的结果打印到串口条数助手中
10     sleep(1); // 1s
11 }
12 }
    
```

3. 编写用于将逻辑代码构建程静态库的“/applications/sample/wifi-iot/app/base_06_i2c_sht20/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("base_sht20_example")
2 {
3     sources = [
4         "base_sht20_example.c",
5         "//base/iot_hardware/bsp/src/hal_bsp_sht20.c"
6     ]
7
8     include_dirs = [
9         "//utils/native/lite/include",
10        "//kernel/liteos_m/kal/cmsis",
11        "//base/iot_hardware/peripheral/interfaces/kits",
12        "//base/iot_hardware/bsp/include"
13    ]
14 }
    
```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：



```
1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         " base_06_i2c_sht20:base_sht20_example",
7     ]
8 }
```

步骤3: 编译工程

按照[第1章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤4: 烧写程序

按照[第1章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

2.6.6 实验现象

烧写完成之后，重启开发板，然后打开串口调试助手，可以看到开发板输出打印的数据。



图3-31 串口调试助手的显示结果



2.7 I2C 之驱动三合一传感器实验

2.7.1 实验目的

1. 学会 I2C 总线与 AP3216C 芯片进行通信。
2. 熟悉 HarmonyOS 系统下 I2C 通信和相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.7.2 实验要求

使用 I2C 去驱动 AP3216C 芯片，读取光照强度值、物体接近检测值、人体红外检测值，然后使用串口调试助手将这三个信息每隔一秒中打印出来。

2.7.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_07_i2c_ap3216c】

2.7.4 实验原理

在本实验中，使用 I2C 总线与 AP3216C 三合一传感器进行通信，读取当前的光照强度、红外、接近的数据。下面给出了硬件连接图：



图3-32 Hi3861 核心板原理图

环境光/接近/红外传感

7位I2C地址: 0x1E

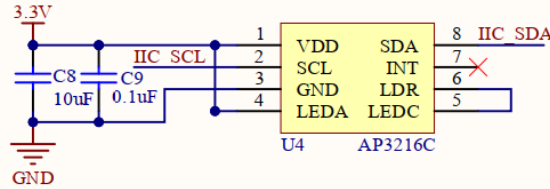


图3-33 AP3216C 传感器原理图

1. 根据 AP3216C 芯片手册，找出我们所需要的数据。

1) 7 位 I2C 从机地址，如下图所示：

I²C Slave Address

The slave addresses have 7 bits. A read/write bit should be appended to the slave address by the master device to properly communicate with the device. The slave address of this device is 0x1E.

图3-34 芯片的 I2C 从机地址

2) 写时序

· I²C Write Protocol (type 2):

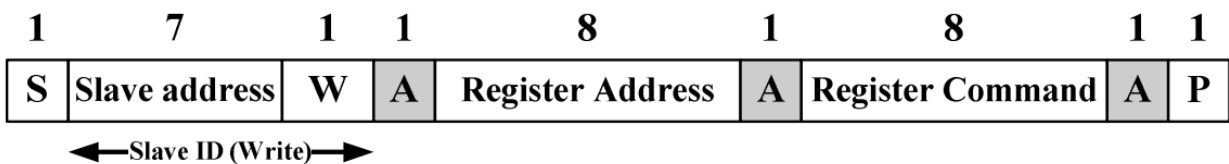


图3-35 I2C 通信的写命令时序

3) 读时序

· I²C Read (Combined format) Protocol:

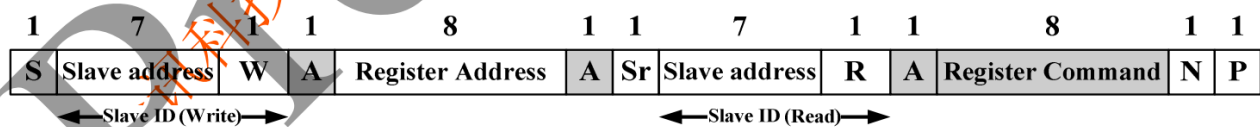


图3-36 I2C 通信的读寄存器数据时序

2.7.5 实验步骤

注意：下面显示出主要代码，其余的请查看源码。



步骤1: 编写 AP3216C 的驱动

1. 在“base/iot_hardware/”目录下创建 bsp 目录，然后在 bsp 目录下创建 src 目录和 include 目录，然后在 src 目录下创建“hal_bsp_ap3216c.c”文件和在 include 目录下创建“hal_bsp_ap3216c.h”文件。下面为目录格式：

```

bsp
├── include
│   └── hal_bsp_ap3216c.h
└── src
    └── hal_bsp_ap3216c.c
    
```

2. 编写 AP3216C 的驱动程序，具体驱动代码可在“/base/iot_hardware/bsp/src/hal_bsp_ap3216c.c 中进行查看。

步骤2: 编写应用程序

1. 在“/applications/sample/wifi-iot/app/”目录中创建 base_07_i2c_ap3216c 目录，在此目录中创建 base_ap3216c_example.c 文件和 BUILD.gn 文件。

2. 编写 base_ap3216c_example.c 文件：

```

1 void Task1(void *argument)
2 {
3     uint16_t ir = 0, als = 0, ps = 0;    // 红外传感器 接近传感器 光照强度传感器
4
5     while (1)
6     {
7         AP3216C_ReadData(&ir, &als, &ps);
8         console_log_info("ir = %d    als = %d    ps = %d", ir, als, ps);
9         sleep(1); // 1s
10    }
11}
    
```

3. 编写用于将逻辑代码构建程静态库的“/applications/sample/wifi-iot/app/base_07_i2c_ap3216c/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("base_ap3216c_example") {
2     sources = [
3         "base_ap3216c_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_ap3216c.c"
5     ]
6
7     include_dirs = [
8         "//utils/native/lite/include",
9         "//kernel/liteos_m/kal/cmsis",
10        "//base/iot_hardware/peripheral/interfaces/kits",
11        "//base/iot_hardware/bsp/include"
12    ]
    }
    
```



```
13 ]
14 }
```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```
1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         "base_07_i2c_ap3216c: base_ap3216c_example",
7     ]
8 }
```

步骤3: 编译工程

按照第1章 开发环境搭建中“编译源码”小节的内容进行编译工程。

步骤4: 烧写程序

按照第1章 开发环境搭建中“烧写程序”小节的内容进行烧写程序。

2.7.6 实验现象

烧写完成之后，重启开发板，打开串口调试助手，在串口调试助手中可以看到采集到的信息。



图3-37 串口调试助手打印的信息



2.8 I2C 之驱动 0.96 寸 OLED 显示屏实验

2.8.1 实验目的

1. 学会 I2C 总线与 ssd1306 芯片进行通信。
2. 熟悉 HarmonyOS 系统下 I2C 通信和相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.8.2 实验要求

在 OLED 显示屏上显示一个模拟时钟。

2.8.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_08_i2c_ssd1306】

2.8.4 实验原理

在本实验中，使用 I2C 总线与 SSD1306 进行通信，控制在 0.96 寸显示屏上显示数据。下面给出了硬件连接图：



图3-38 Hi3861 核心板原理图

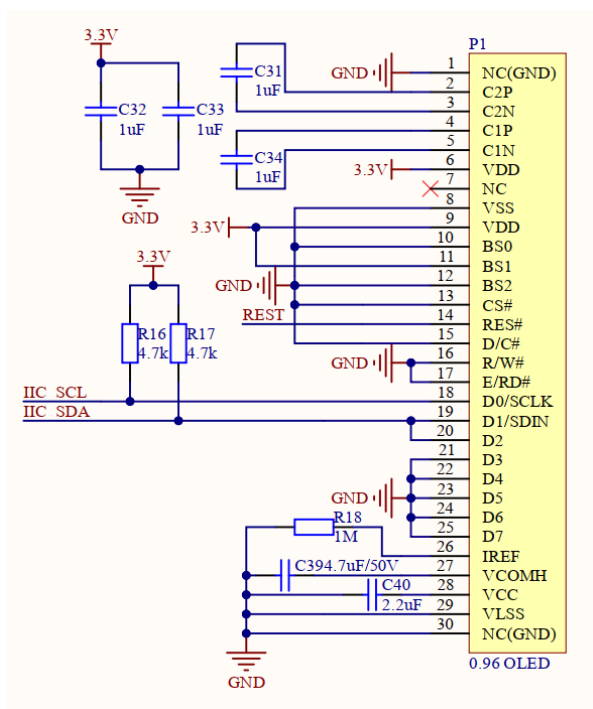


图3-39 0.96 寸 OLED 显示屏的电路原理图

1. 根据 ssd1306 的芯片手册，可以得到驱动的相关数据。

1) I2C 通信的从机地址，如下图所示，可以得知，当 D/C#引脚接 VDD（3.3V）时，从机地址为 0x3D，否则从机地址为 0x3C

The I²C slave address is 0111100b'. If the customer ties D/C# (pin 15) to VDD, the I²C slave address will be 0111101b'.

图3-40 I2C 通信的从机地址

2.8.5 实验步骤

注意：下面显示出主要代码，其余的请查看源码。

步骤1: 编写 SSD1306 芯片驱动

1. 在 “base/iot_hardware/”目录下创建 bsp 目录，然后在 bsp 目录下创建 src 目录和 include 目录，然后在 src 目录下创建 “hal_bsp_ssd1306.c”文件和在 include 目录下创建 “hal_bsp_ssd1306.h”文件。下面为目录格式：

```
bsp
├── include
│   └── hal_bsp_ssd1306.h
└── src
    └── hal_bsp_ssd1306.c
```

2. 编写 AP3216C 的驱动程序，具体驱动代码可在 “/base/iot_hardware/bsp/src/hal_bsp_ap3216c.c 中进行查看。



步骤2: 编写应用程序

1. 在“/applications/sample/wifi-iot/app/”目录中创建 base_08_i2c_ssd1306 目录，在此目录中创建 base_ssd1306_example.c 文件和 BUILD.gn 文件。

2. 编写 base_ssd1306_example.c 文件：

```
1 void Task1(void *argument)
2 {
3     uint8_t displayBuff[20] = {0};
4     uint8_t hour = 10, min = 30, sec = 0;
5     SSD1306_CLS(); // 清屏
6     SSD1306_ShowStr(0, 0, " Analog Clock ", 16);
7     SSD1306_ShowStr(0, 3, " 2021-12-24 ", 16);
8
9     while (1)
10    {
11        sec++;
12        if (sec > 59){
13            sec = 0;
14            min++;
15        }
16        if (min > 59){
17            min = 0;
18            hour++;
19        }
20        if (hour > 23)
21            hour = 0;
22
23        memset(displayBuff, 0, sizeof(displayBuff));
24        sprintf(displayBuff, " %02d:%02d:%02d ", hour, min, sec);
25        SSD1306_ShowStr(0, 2, displayBuff, 16);
26        sleep(1); // 1 s
27    }
28 }
```

3. 编写用于将逻辑代码构建程静态库的“/applications/sample/wifi-iot/app/base_08_i2c_ssd1306/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```
1 static_library("base_ssd1306_example") {
2     sources = [
3         base_ssd1306_example.c",
4         "../base/iot_hardware/bsp/src/hal_bsp_ssd1306.c"
5     ]
6
7     include_dirs = [
8         "../utils/native/lite/include",
```



```

9     "//kernel/liteos_m/kal/cmsis",
10    "//base/iot_hardware/peripheral/interfaces/kits",
11    "//base/iot_hardware/bsp/include"
12  ]
13}

```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         # 硬件基础相关实验
6         "base_08_i2c_ssd1306: base_ssd1306_example",
7     ]
8 }

```

步骤3: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤4: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

2.8.6 实验现象

烧写完成之后，重启开发板，可以看到在 OLED 显示屏上显示一个模拟时钟，如下图所示：

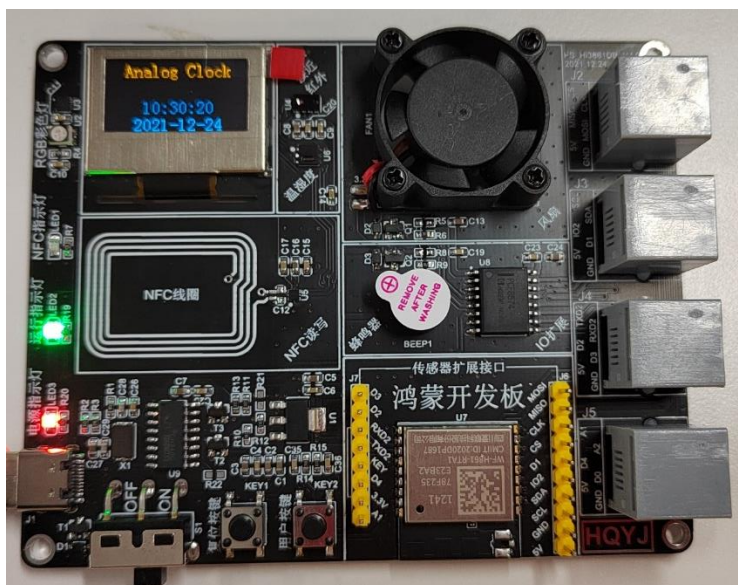


图3-41 OLED 显示模拟时钟



2.9 I2C 之驱动 NFC 实验

2.9.1 实验目的

1. 学习如何调用 I2C 总线驱动 NFC 标签进行读写数据。
2. 熟悉 HarmonyOS 系统下 I2C 相关函数的使用。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

2.9.2 实验要求

向 NFC 标签中写入数据，并且读取 NFC 标签中的数据。

2.9.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\base_09_i2c_nfc】

2.9.4 实验原理

通过使用手机上的“NFC 标签助手”APP 进行写入标签数据，写入成功之后，复位开发板，将 NFC 标签中的数据读取出来，然后进行解析。

测试案例：

1. 使用 NFC 标签助手写入文本数据，文本数据内容“123456”。
2. 在串口调试助手中查看打印的信息：

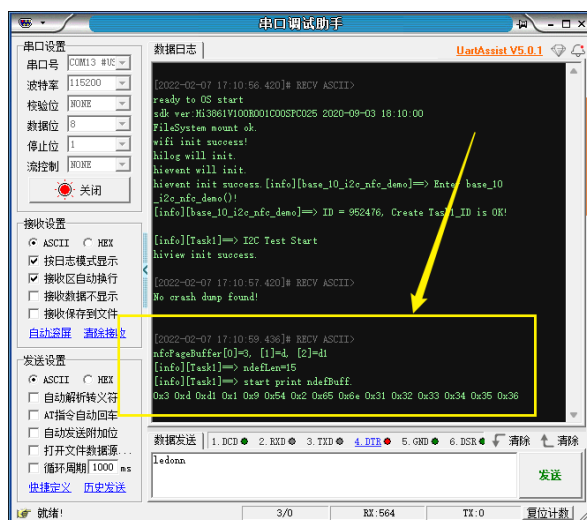


图3-42 查看串口调试助手的数据

3. 解析数据：



0x3	0xd	0xd1	0x1	0x9	0x54	0x2	0x65	0x6e	0x31	0x32	0x33	0x34	0x35	0x36
协议头	NDEF协议的长	消息控制标志	数据类型长度	数据类型长度	数据类型长度	编码类型长度	编码类型en		1	2	3	4	5	6

图3-43 NDEF 协议

2.9.5 实验步骤

步骤1: 编写应用程序

1. 在“/applications/sample/wifi-iot/app/”目录中创建 base_09_i2c_nfc 目录，在此目录中创建 base_nfc_example.c 文件和 BUILD.gn 文件。

2. 编写 base_nfc_example.c 文件：

注意：下面显示出主要代码，其余的请查看源码。

```

1 void Task1(void *argument)
2 {
3     uint8_t ret, num = 0;
4     uint8_t ndefLen = 0;           // ndef 包的长度
5     uint8_t ndef_Header = 0;      // ndef 消息开始标志位-用不到
6     uint32_t result_code = 0;    // 函数的返回值
7
8     nfc_Init();
9     console_log_info("I2C Test Start");
10    sleep(3);
11
12    // 读整个数据的包头部分，读出整个数据的长度
13    if (result_code = NT3HReadHeaderNfc(&ndefLen, &ndef_Header) != true)
14    {
15        console_log_error("NT3HReadHeaderNfc is failed. result_code = %d", result_code);
16        return ;
17    }
18
19    ndefLen += NDEF_HEADER_SIZE;   // 加上头部字节
20
21    console_log_info("ndefLen=%d", ndefLen);
22
23    uint8_t *ndefBuff = (uint8_t *)malloc(ndefLen + 1);
24    if(ndefBuff == NULL)
25    {
26        console_log_error("ndefBuff malloc is Falied!");
27        return ;
28    }
29

```



```

30 if(result_code = get_NDEFDataPackage(ndefBuff, ndefLen) != HI_ERR_SUCCESS)
31 {
32     console_log_error("get_NDEFDataPackage is failed. result_code = %d", result_code);
33     return ;
34 }
35
36 console_log_info("start print ndefBuff.");
37 for (size_t i = 0; i < ndefLen; i++)
38 {
39     printf("0x%x ", ndefBuff[i]);
40 }
41
42 while (1)
43 {
44
45     sleep(1);
46 }
47 }
    
```

3. 编写用于将逻辑代码构建程静态库的“/applications/sample/wifi-iot/app/base_09_i2c_nfc/BUILD.gn”文件。打开 BUILD.gn 文件，填入一下代码：

```

1 static_library("base_nfc_example") {
2     sources = [
3         "//base/iot_hardware/bsp/nfc/NT3H.c",
4         "//base/iot_hardware/bsp/nfc/nfc.c",
5         "//base/iot_hardware/bsp/nfc/ndef/rtd/nfcForum.c",
6         "//base/iot_hardware/bsp/nfc/ndef/rtd/rtdText.c",
7         "//base/iot_hardware/bsp/nfc/ndef/rtd/rtdUri.c",
8         "//base/iot_hardware/bsp/nfc/ndef/ndef.c",
9         "//base/iot_hardware/bsp/src/hal_bsp_nfc.c",
10        "base_nfc_example.c",
11    ]
12
13    cflags = [ "-Wno-unused-variable" ]
14    cflags += [ "-Wno-unused-but-set-variable" ]
15    cflags += [ "-Wno-unused-parameter" ]
16
17    include_dirs = [
18        "//utils/native/lite/include",
19        "//kernel/liteos_m/kal/cmsis",
20        "//base/iot_hardware/peripheral/interfaces/kits",
21        "//base/iot_hardware/bsp/include",
22
23        "//base/iot_hardware/bsp/nfc/ndef",
    
```



```
24     "//base/iot_hardware/bsp/nfc/ndef/rtd/",
25     "//base/iot_hardware/bsp/nfc"
26 ]
27 }
```

4. 编写“/applications/sample/wifi-iot/app/BUILD.gn”文件，打开 BUILD.gn 文件，填入一下代码：

```
1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5         "base_09_i2c_nfc:base_nfc_example"
6     ]
7 }
```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

步骤4: 烧写标签

1. 在手机端下载并打开“NFC 标签助手 APP”，点击“NFC 标签助手 APP”的“写入”选项。



图3-44

2. 然后点击“文本”，添加一条记录。



图3-45 添加文本类型的标签

3. 在输入框中输入数据“123456”，然后点击“确定”。



图3-46 添加文本类型记录

4. 此时会弹出“靠近 NFC 标签”的提示，然后将手机靠近开发板中的 NFC 区域。



图3-47 靠近 NFC 标签

5. 烧写标签完成之后，我们可以点击“读取”选项卡，然后手机靠近 NFC 标签进行读取。



图3-48 读取 NFC 标签中的数据

6. 读取成功之后，会在此窗口中显示出当前 NFC 标签中的数据。此时我们点击“详细数据”，查看详细数据。



图3-49 详细数据

2.9.6 实验现象

烧写完成之后，重启开发板，在串口调试助手中可以看到，所打印的信息，如下图所示：

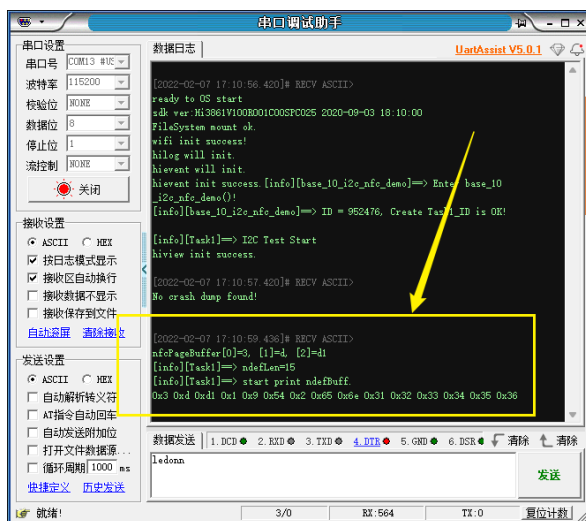


图3-50 串口调试助手的数据



第3章 HarmonyOS 之物联网实验系列

3.1 Wi-Fi 之建立网络实验

3.1.1 实验目的

1. 学习如何建立一个 WiFi 热点。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

3.1.2 实验要求

建立好 WiFi 热点之后，手机可以连接这个节点，然后在串口调试助手中打印出设备的连接信息。

3.1.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\network_01_wifi_ap】

3.1.4 实验原理

当模组单独处于 AP 模式下，可以被认为是一个无法访问外网的局域网 WiFi 路由器节点，它可以接受各类设备的无线连接请求。并可以和无线连接终端设备进行 TCP、UDP 连接，实现数据传输。在局域物联网的设计中可以承担数据的收发节点的作用。

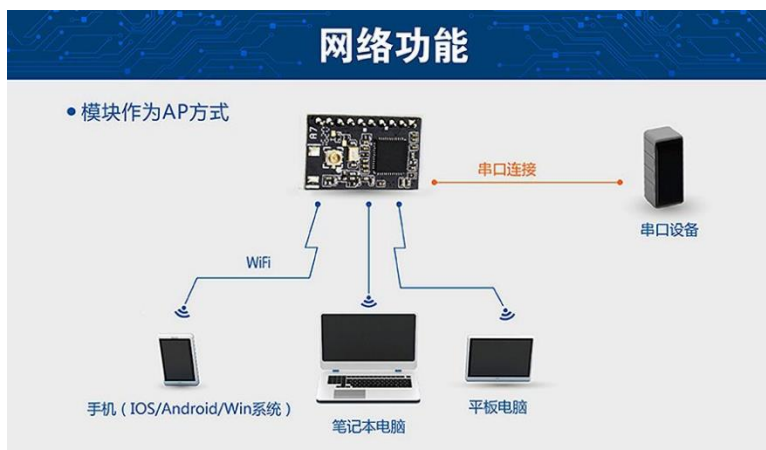


图4-1 AP 模式

3.1.5 实验步骤

步骤1: 编写 WiFi 初始化函数

1. 在 “base/iot_hardware/” 目录下创建 bsp 目录，然后在 bsp 目录下创建 src 目录和 include 目录，然后在 src 目录下创建 “hal_bsp_wifi.c” 文件和在 include 目录下创建 “hal_bsp_wifi.h” 文件。下面为目录格式：



```
.
├── include
│   ├── hal_bsp_ap3216c.h
│   ├── hal_bsp_aw2013.h
│   ├── hal_bsp_pcf8574.h
│   ├── hal_bsp_sht20.h
│   ├── hal_bsp_ssd1306_fonts.h
│   ├── hal_bsp_ssd1306.h
│   ├── hal_bsp_structAll.h
│   └── hal_bsp_wifi.h
└── src
    ├── hal_bsp_ap3216c.c
    ├── hal_bsp_aw2013.c
    ├── hal_bsp_pcf8574.c
    ├── hal_bsp_sht20.c
    ├── hal_bsp_ssd1306.c
    └── hal_bsp_wifi.c
```

2. 在 hal_bsp_wifi.c 文件中，编写驱动程序：

注意：下面显示出主要代码，其余的请查看源码。

```
1 WifiErrorCode createWifihotspots(const char *ssid, const char *psk)
2 {
3     WifiErrorCode result;
4     printf("[info] Start initialization of WiFi hotspots\n");
5     // 使能 WiFi
6     result = EnableWifi();
7     if(result != WIFI_SUCCESS)
8     {
9         printf("[error] Enable WiFi failed\n");
10        return result;
11    }
12    // 判断 WiFi 是否激活
13    result = IsWifiActive();
14    if(result != WIFI_STA_ACTIVE)
15    {
16        printf("[error] WiFi activation failed\n");
17        return result;
18    }
19    // 设置指定的热点信息
20    HotspotConfig hotspotConfig = {0};
21    strcpy(hotspotConfig.ssid, ssid);
22    strcpy(hotspotConfig.preSharedKey, psk);
23    hotspotConfig.securityType = WIFI_SEC_TYPE_PSK;
24    hotspotConfig.band = HOTSPOT_BAND_TYPE_2G;
```



```

25 hotspotConfig.channelNum = 5;
26 result = SetHotspotConfig(&hotspotConfig);
27 if(result != WIFI_SUCCESS)
28 {
29     printf("[error] Failed to set WiFi hotspot information\n");
30     return result;
31 }
32 // 开启 WiFi 热点模式
33 result = EnableHotspot();
34 if(result != WIFI_SUCCESS)
35 {
36     printf("[error] Failed to enable wifi hotspot mode\n");
37     return result;
38 }
39 // 检查 WiFi 热点是否激活
40 result = IsHotspotActive();
41 if(result != WIFI_HOTSPOT_ACTIVE)
42 {
43     printf("[error] WiFi hotspot activation failed\n");
44     return result;
45 }
46 g_lwip_netif = netifapi_netif_find(SELECT_WLAN_PORT);
47 //启动 DHCP
48 if (g_lwip_netif)
49 {
50     dhcp_start(g_lwip_netif);
51     printf("[info] began to dhcp\n");
52 }
53 //等待 DHCP
54 for (;;)
55 {
56     if (dhcp_is_bound(g_lwip_netif) == ERR_OK)
57     {
58         printf("[info] <-- DHCP state:OK -->\n");
59         break;
60     }
61     printf("[info] <-- DHCP state:Inprogress -->\n");
62     sleep(1);
63 }
64 printf("[info] WiFi hotspot initialized successfully\n");
65 return WIFI_SUCCESS;
66 }

```

3. 在 hal_bsp_ap3216c.h 文件中，编写外部接口：



```

1 #ifndef __HAL_BSP_WIFI_H
2 #define __HAL_BSP_WIFI_H
3
4 #include "cmsis_os2.h"
5 #include "wifi_error_code.h"
6
7 /**
8  * @brief WIFI 创建 wifi 热点
9  * @param ssid WiFi 名称
10 * @param psk WiFi 密码
11 * @return Returns {WIFI_SUCCESS} 成功;
12 *         Returns {other} 失败.
13 */
14 WifiErrorCode createWifihotspots(const char *ssid, const char *psk);
15
16 #endif

```

步骤2: 编写应用程序

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 network_01_wifi_ap 目录，在此目录中创建 network_wifi_ap_example.c 文件和 BUILD.gn 文件。

2. 编写 network_wifi_ap_example.c 文件：

注意：下面显示出主要代码，其余的请查看源码。

```

1 // 当 有设备连接热点时 触发的回调函数
2 static void connectingWifihotspotsCallback(StationInfo *info)
3 {
4     if (info == NULL)
5         printf("[error] HotspotStaJoin:info is null\n");
6     else
7         printf("[info] New Sta Join\n");
8 }
9 // 当 有设备断开热点时 触发的回调函数
10 static void disconnectWifihotspotsCallback(StationInfo *info)
11 {
12     if (info == NULL)
13         printf("[error] HotspotStaLeave:info is null\n");
14     else
15         printf("[info] HotspotStaLeave: macAddress=%02X:%02X:%02X:%02X:%02X:%02X, reason=%d\n",
16             info->macAddress[0],
17             info->macAddress[1],
18             info->macAddress[2],
19             info->macAddress[3],
20             info->macAddress[4],

```



```

21         info->macAddress[5],
22         info->disconnectedReason);
23 }
24 // 当 热点的状态发生改变时的回调函数
25 static void changeWiFiHotspotStateCallback(int state)
26 {
27     printf("[info] HotspotStateChanged:state is %d\n", state);
28     if (state == WIFI_HOTSPOT_ACTIVE)
29         printf("[info] wifi hotspot active\n");
30     else
31         printf("[info] wifi hotspot noactive\n");
32 }
33 void Task1(void *argument)
34 {
35     WifiErrorCode result;
36     sleep(3);
37     // 注册 WiFi 事件的回调函数
38     WifiEvent wifiEvent = {0};
39     wifiEvent.OnHotspotStaJoin = connectingWifihotspotsCallback;
40     wifiEvent.OnHotspotStaLeave = disconnectWifihotspotsCallback;
41     wifiEvent.OnHotspotStateChanged = changeWiFiHotspotStateCallback;
42     result = RegisterWifiEvent(&wifiEvent);
43     if(result != WIFI_SUCCESS)
44     {
45         printf("[error] Failed to register WiFi hotspot callback function\n");
46         return ;
47     }
48     createWifihotspots("FS_Hi3861_AP", "fs123456");
49     while (1)
50     {
51         sleep(1); // 1 s
52     }
53 }

```

3. 编写 BUILD.gn 文件

```

1 static_library("network_wifi_ap_example") {
2     sources = [
3         "network_wifi_ap_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_wifi.c"
5     ]
6     include_dirs = [
7         "//utils/native/lite/include",
8         "//kernel/liteos_m/kal/cmsis",

```



```

9      "//base/iot_hardware/peripheral/interfaces/kits",
10     "//foundation/communication/wifi_lite/interfaces/wifiservice",
11     "//base/iot_hardware/bsp/include"
12 ]
13 }
14

```

4. 修改 “/applications/sample/wifi-iot/app/BUILD.gn” 文件，将这句话取消注释，其余的全部注释。

```

# 网络相关实验
"network_01_wifi_ap:network_wifi_ap_example",
# "network_02_wifi_sta:network_wifi_sta_example",
# "network_03_wifi_udp:network_wifi_udp_example",
# "network_04_wifi_tcp:network_wifi_tcp_example",
# "network_05_wifi_mqtt:network_wifi_mqtt_example",

```

图4-2 更改 BUILD.gn 文件

步骤3: 编译工程

按照第1章 开发环境搭建中“编译源码”小节的内容进行编译工程。

步骤4: 烧写程序

按照第1章 开发环境搭建中“烧写程序”小节的内容进行烧写程序。

3.1.6 实验现象

打开串口调试助手，进行查看结果，此时需要使用手机连接 Hi3861 的热点信息，然后会在串口调试助手中看到，有设备连接。如下图所示：

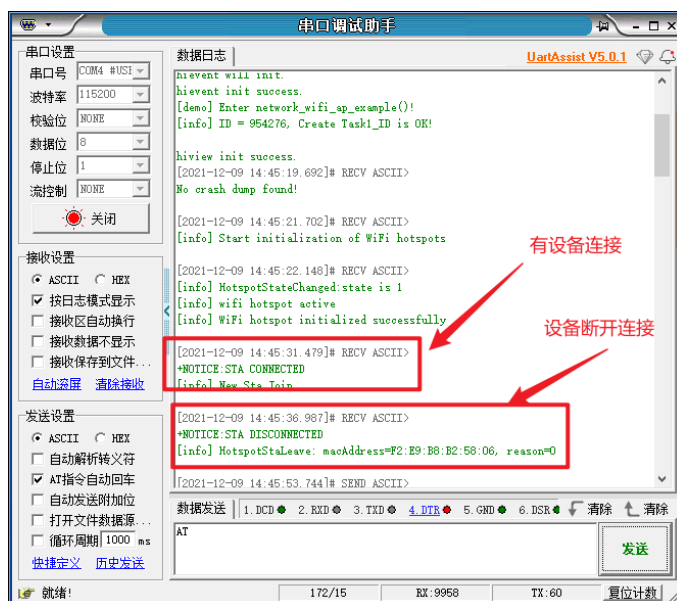


图4-3 实验串口打印结果



3.2 Wi-Fi 之连接网络实验

3.2.1 实验目的

1. 学习如何连接一个 WiFi 热点
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

3.2.2 实验要求

连接附近的 WiFi 热点，然后在串口调试助手打印出设备的连接信息。

3.2.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\network_02_wifi_sta】

3.2.4 实验原理

当模组单独处于 STA 模式下，模块通过无线路由器连接互联网、手机或电脑，并可以和无线连接终端设备进行 TCP、UDP 连接，实现数据传输。通过互联网实现对设备的远程控制。

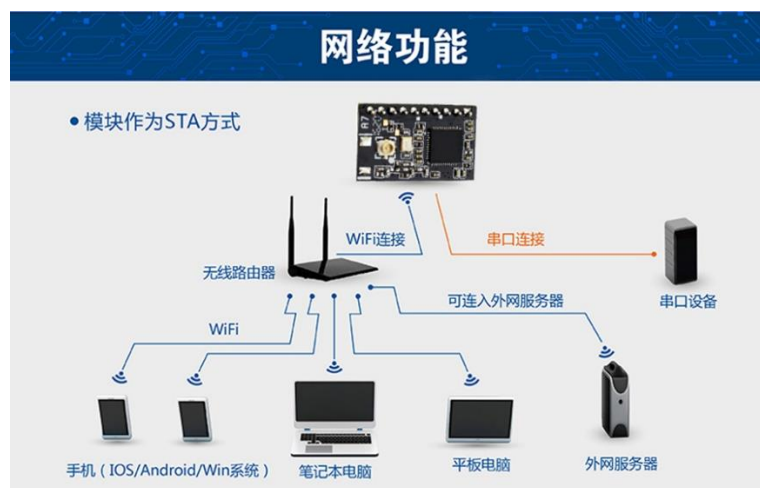


图4-4 STA 模式

3.2.5 实验步骤

步骤1: 编写 WiFi 初始化函数

1. 在“base/iot_hardware/”目录下创建 bsp 目录，然后在 bsp 目录下创建 src 目录和 include 目录，然后在 src 目录下创建“hal_bsp_wifi.c”文件和在 include 目录下创建“hal_bsp_wifi.h”文件。下面为目录格式：



```

├── include
│   ├── hal_bsp_ap3216c.h
│   ├── hal_bsp_aw2013.h
│   ├── hal_bsp_pcf8574.h
│   ├── hal_bsp_sht20.h
│   ├── hal_bsp_ssd1306_fonts.h
│   ├── hal_bsp_ssd1306.h
│   ├── hal_bsp_structAll.h
│   └── hal_bsp_wifi.h
└── src
    ├── hal_bsp_ap3216c.c
    ├── hal_bsp_aw2013.c
    ├── hal_bsp_pcf8574.c
    ├── hal_bsp_sht20.c
    ├── hal_bsp_ssd1306.c
    └── hal_bsp_wifi.c
    
```

2. 在 hal_bsp_wifi.c 文件中，添加驱动程序：

注意：下面显示出主要代码，其余的请查看源码。

3. 在 hal_bsp_ap3216c.h 文件中，编写外部接口：

```

1 #ifndef __HAL_BSP_WIFI_H
2 #define __HAL_BSP_WIFI_H
3
4 #include "cmsis_os2.h"
5 #include "wifi_error_code.h"
6
7 /**
8  * @brief WIFI 创建 wifi 热点
9  * @param ssid WiFi 名称
10 * @param psk WiFi 密码
11 * @return Returns {WIFI_SUCCESS} 成功;
12 *         Returns {other} 失败.
13 */
14 WifiErrorCode createWifihotspots(const char *ssid, const char *psk);
15 /**
16 * @brief WIFI 连接附近的 WiFi
17 * @param ssid WiFi 名称
18 * @param psk WiFi 密码
19 * @return Returns {WIFI_SUCCESS} 成功;
20 *         Returns {other} 失败.
21 */
22 WifiErrorCode connectWiFiHotspots(const char *ssid, const char *psk);
23
24 #endif
    
```



25

步骤2: 编写应用程序

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 network_02_wifi_sta 目录，在此目录中创建 network_wifi_sta_example.c 文件和 BUILD.gn 文件。

2. 编写 network_wifi_sta_example.c 文件：

注意：下面显示出主要代码，其余的请查看源码。

```
1 void Task1(void *argument)
2 {
3     WifiErrorCode result;
4
5     Wifi_connectHotspots("FS_Hi3861", "fs123456");
6
7     while (1)
8     {
9         sleep(1); // 1 s
10    }
11 }
```

3. 编写 BUILD.gn 文件

```
1 static_library("network_wifi_sta_example") {
2     sources = [
3         "network_wifi_sta_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_wifi.c"
5     ]
6     include_dirs = [
7         "//utils/native/lite/include",
8         "//kernel/liteos_m/kal/cmsis",
9         "//base/iot_hardware/peripheral/interfaces/kits",
10        "//foundation/communication/wifi_lite/interfaces/wifiservice",
11        "//base/iot_hardware/bsp/include"
12    ]
13 }
14
```

4. 修改 “/applications/sample/wifi-iot/app/BUILD.gn” 文件，将这句话取消注释，其余的全部注释。

```
# 网络相关实验
# "network_01_wifi_ap:network_wifi_ap_example",
"network_02_wifi_sta:network_wifi_sta_example",
# "network_03_wifi_udp:network_wifi_udp_example",
# "network_04_wifi_tcp:network_wifi_tcp_example",
# "network_05_wifi_mqtt:network_wifi_mqtt_example",
```

图4-5 更改 BUILD.gn 文件



步骤3: 编译工程

按照第1章 开发环境搭建中“编译源码”小节的内容进行编译工程。

步骤4: 烧写程序

按照第1章 开发环境搭建中“烧写程序”小节的内容进行烧写程序。

3.2.6 实验现象

打开串口调试助手，进行查看结果。如下图所示：



图4-6 串口调试助手打印信息

注意：如果自己的手机打开了热点，WiFi 名称和密码都一样，却迟迟连接不成功。这时需要检查一下自己的手机热点配置，将热点的 AP 频段更改为 2.4GHz 频段。

3.3 Wi-Fi 之 UDP 通信实验

3.3.1 实验目的

1. 学习 UDP 通信流程。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

3.3.2 实验要求

在 Hi3861 模块内部建立 UDP 客户端，去连接电脑端的 UDP 服务器端。



3.3.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\network_03_wifi_udp】

3.3.4 实验原理

在 TCP/IP 网络体系结构中，TCP（传输控制协议，Transport Control Protocol）、UDP（用户数据报协议，User Data Protocol）是传输层最重要的两种协议，为上层用户提供级别的通信可靠性。

用户数据报协议（UDP）：UDP（用户数据报协议）是一个简单的面向数据报的传输层协议。提供的是非面向连接的、不可靠的数据流传输。UDP 不提供可靠性，也不提供报文到达确认、排序以及流量控制等功能。它只是把应用程序传给 IP 层的数据报发送出去，但是并不能保证它们能到达目的地。因此报文可能会丢失、重复以及乱序等。但由于 UDP 在传输数据报前不用在客户和服务端之间建立一个连接，且没有超时重发等机制，故而传输速度很快。

- 使用 Socket 接口搭建 UDP 服务器流程
socket → bind → recvfrom → sendto → close
- 使用 Socket 接口搭建 UDP 客户端流程
socket → bind → recvfrom → sendto → close

3.3.5 实验步骤

步骤1: 编写应用程序

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 network_02_wifi_sta 目录，在此目录中创建 network_wifi_sta_example.c 文件和 BUILD.gn 文件。
2. 编写 network_wifi_sta_example.c 文件：

注意：下面显示出主要代码，其余的请查看源码。

```
1 void Task1(void *argument)
2 {
3     int socket_fd = 0;
4     int result;
5     //服务器的地址信息
6     struct sockaddr_in send_addr;
7     socklen_t addr_length = sizeof(send_addr);
8     char recvBuf[512];
9     //连接 Wifi
10    connectWiFiHotspots("HQYJ_OFFICE", "*****");
11    //创建 socket
12    if ((socket_fd = socket(AF_INET, SOCK_DGRAM, 0)) == -1)
13    {
14        printf("[error] create socket failed!\n");
```



```

15     return ;
16 }
17 //初始化预连接的服务端地址
18 send_addr.sin_family = AF_INET; // IPV4
19 send_addr.sin_port = htons(6789); // 远端服务器的端口号
20 send_addr.sin_addr.s_addr = inet_addr("192.168.100.211"); // 远端服务器的 IP 地址
21 addr_length = sizeof(send_addr);
22 //发送数据到服务远端
23 result = sendto(socket_fd, "hello farsight\r\n",
24                 strlen("hello farsight\r\n"),
25                 0,
26                 (struct sockaddr *)&send_addr,
27                 addr_length);
28 if(result)
29 {
30     printf("[info] result: %d, sendData:%s\n", result, "hello farsight\r\n");
31 }
32 while (1)
33 {
34     memset(recvBuf, 0, sizeof(recvBuf));
35     //接收服务端返回的字符串
36     result = recvfrom(socket_fd, recvBuf,
37                      sizeof(recvBuf),
38                      0,
39                      (struct sockaddr *)&send_addr,
40                      &addr_length);
41     if(result > 0)
42     {
43         printf("[info][%s:%d]recvData:%s\n",
44               inet_ntoa(send_addr.sin_addr),
45               ntohs(send_addr.sin_port),
46               recvBuf);
47         //发送数据到服务远端
48         result = sendto(socket_fd, recvBuf,
49                        sizeof(recvBuf), 0,
50                        (struct sockaddr *)&send_addr,
51                        addr_length);
52         if(result)
53         {
54             printf("[info] result: %d, sendData:%s\n", result, recvBuf);
55         }
56     }
57     sleep(1);
58 }

```



```
59
60 //关闭这个 socket
61 closesocket(socket_fd);
62 }
```

3. 编写 BUILD.gn 文件

```
1  static_library("network_wifi_udp_example") {
2      sources = [
3          "network_wifi_udp_example.c",
4          "//base/iot_hardware/bsp/src/hal_bsp_wifi.c"
5      ]
6      include_dirs = [
7          "//utils/native/lite/include",
8          "//kernel/liteos_m/kal/cmsis",
9          "//base/iot_hardware/peripheral/interfaces/kits",
10         "//foundation/communication/wifi_lite/interfaces/wifiservice",
11         "//base/iot_hardware/bsp/include"
12     ]
13 }
14
```

4. 修改 “/applications/sample/wifi-iot/app/BUILD.gn” 文件，将这句话取消注释，其余的全部注释。

```
# 网络相关实验
# "network_01_wifi_ap:network_wifi_ap_example",
# "network_02_wifi_sta:network_wifi_sta_example",
"network_03_wifi_udp:network_wifi_udp_example",
# "network_04_wifi_tcp:network_wifi_tcp_example",
# "network_05_wifi_mqtt:network_wifi_mqtt_example",
```

图4-7 更改 BUILD.gn 文件

步骤2: 编译工程

按照[第 1 章 开发环境搭建](#)中“[编译源码](#)”小节的内容进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建](#)中“[烧写程序](#)”小节的内容进行烧写程序。

3.3.6 实验现象

1. 打开【FS_Hi3861 开发板光盘资料\工具软件\TCP UDP Socket 调试工具.exe】软件，然后修改为如下配置：

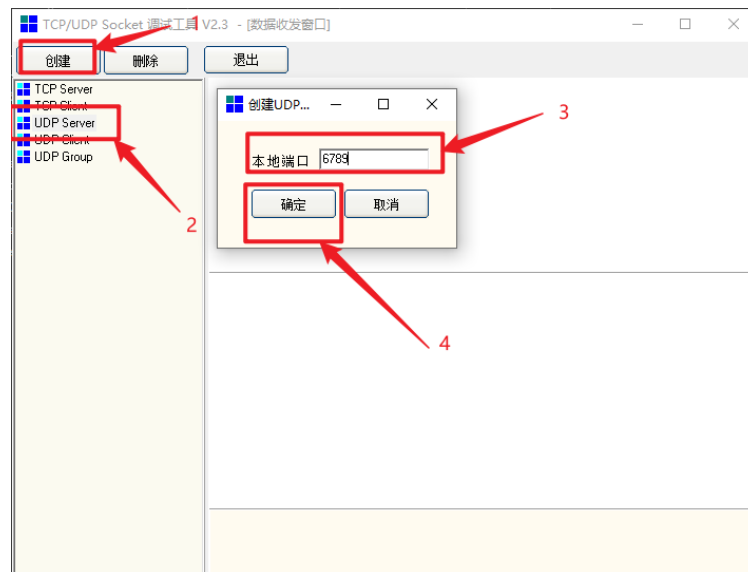


图4-8 创建 UDP 服务器

2. 创建好后，复位开发板，然后在 socket 调试工具中查看收到的内容。

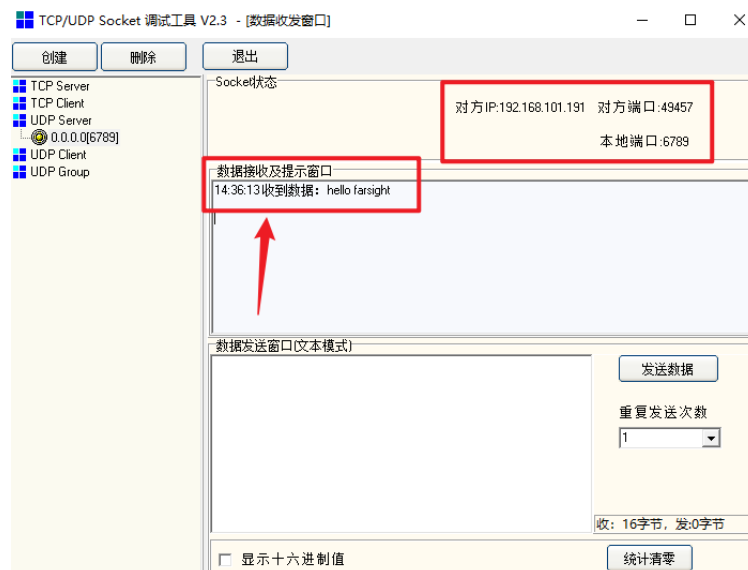


图4-9 在 UDP 服务器中接收到数据

3. 然后在发送窗口发送一串字符串，然后点击发送，此时就会在串口调试助手中看到数据。



图4-10 接收到 UDP 数据

3.4 Wi-Fi 之 TCP 通信实验

3.4.1 实验目的

1. 学习 TCP 通信机制
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

3.4.2 实验要求

在 Hi3861 开发板中创建 TCP 客户端，在 PC 电脑端创建 TCP 服务端，然后可以在两端之间进行传输数据。

3.4.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\network_04_wifi_tcp】

3.4.4 实验原理

在 TCP/IP 网络体系结构中，TCP（传输控制协议，Transport Control Protocol、UDP（用户数据报协议，User Data Protocol）是传输层最重要的两种协议，为上层用户提供级别的通信可靠性。

传输控制协议（TCP）：TCP（传输控制协议）定义了两台计算机之间进行可靠的传输而交换的数据和确认信息的格式，以及计算机为了确保数据的正确到达而采取的措施。协议规定了 TCP 软件怎样识别给定



计算机上的多个目的进程如何对分组重复这类差错进行恢复。协议还规定了两台计算机如何初始化一个 TCP 数据流传输以及如何结束这一传输。TCP 最大的特点就是提供的是面向连接、可靠的字节流服务。

1. 搭建 TCP 服务器端流程

socket → bind → listen → accept → recv/send → close

2. 搭建 TCP 客户端流程

socket → connect → recv/send → close

3.4.5 实验步骤

步骤1: 编写应用程序

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 network_04_wifi_tcp 目录，在此目录中创建 network_wifi_tcp_example.c 文件和 BUILD.gn 文件。

2. 编写 network_wifi_tcp_example.c 文件：

注意：下面显示出主要代码，其余的请查看源码。

```
1 //在 sock_fd 进行监听，在 new_fd 接收新的链接
2 void Task1(void *argument)
3 {
4     int socket_fd = 0;
5     char buff[256];
6     int re = 0;
7
8     //连接 Wifi
9     WiFi_connectHotspots("HQYJ_OFFICE", "HqyjOffice2021*#&!");
10
11     socket_fd = socket(AF_INET, SOCK_STREAM, 0); //创建套接字 (TCP)
12
13     struct sockaddr_in addr;
14     addr.sin_family = AF_INET;
15     addr.sin_port = htons(5555);
16     addr.sin_addr.s_addr = inet_addr("192.168.100.211"); //填写服务器的 IP 地址
17
18     re = connect(socket_fd, (struct sockaddr *)&addr, sizeof(addr)); //连接服务器
19     if (re == -1)
20     {
21         console_log_info("Failed to connect to the server");
22         return ;
23     }
24     console_log_info("Connection to server successful");
25
26     // 发送第一条数据
27     send(socket_fd, "Connection to server successful.",
28         strlen("Connection to server successful."), 0);
```



```

29
30 while (1)
31 {
32     memset(buff, 0, sizeof(buff));
33     re = recv(socket_fd, buff, sizeof(buff), 0); // 接收客户端发送过来的消息
34     if (re <= 0)
35         break;
36     else
37     {
38         console_log_info("Receive data received by the server: %s", buff);
39         send(socket_fd, buff, sizeof(buff), 0);
40     }
41 }
42
43 close(socket_fd);
44 }

```

3. 编写 BUILD.gn 文件

```

1 static_library("network_wifi_tcp_example") {
2     sources = [
3         "network_wifi_tcp_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_wifi.c"
5     ]
6
7     include_dirs = [
8         "//utils/native/lite/include",
9         "//kernel/liteos_m/kal/cmsis",
10        "//base/iot_hardware/peripheral/interfaces/kits",
11        "//foundation/communication/wifi_lite/interfaces/wifiservice",
12        "//base/iot_hardware/bsp/include"
13    ]
14 }

```

4. 修改 “/applications/sample/wifi-iot/app/BUILD.gn” 文件，将这句话取消注释，其余的全部注释。

```

# 网络相关实验
# "network_01_wifi_ap:network_wifi_ap_example",
# "network_02_wifi_sta:network_wifi_sta_example",
# "network_03_wifi_udp:network_wifi_udp_example",
"network_04_wifi_tcp:network_wifi_tcp_example",
# "network_05_wifi_mqtt:network_wifi_mqtt_example",

```

图4-11 更改 BUILD.gn 文件



步骤2: 编译工程

按照[第1章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第1章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

3.4.6 实验现象

1. 打开【FS_Hi3861开发板光盘资料\工具软件\TCP/UDP Socket 调试工具】,按照如下配置进行配置:

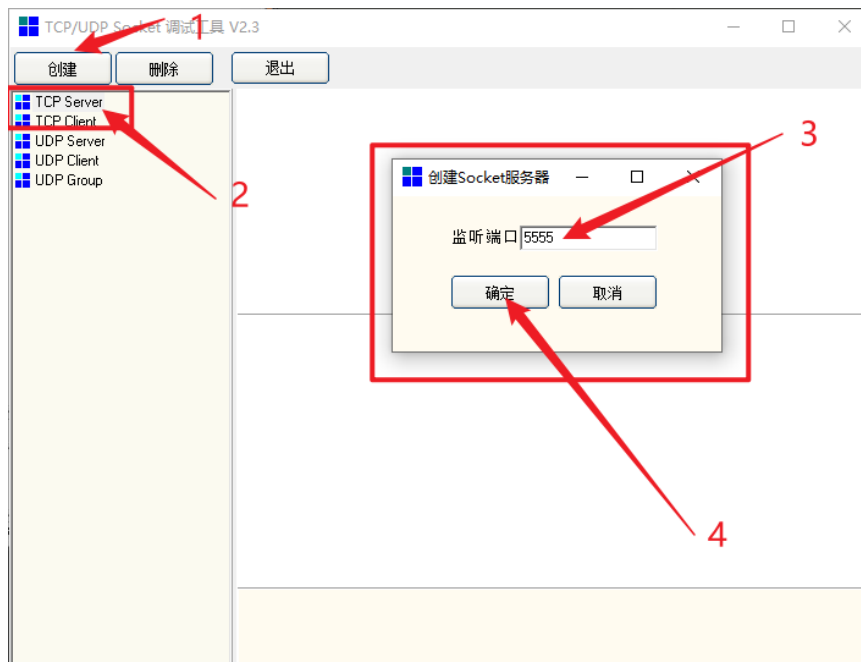


图4-12 创建 TCP 服务器

2. 配置完成之后,按下开发板进行复位,当连接成功之后,在调试工具中会收到一条消息。

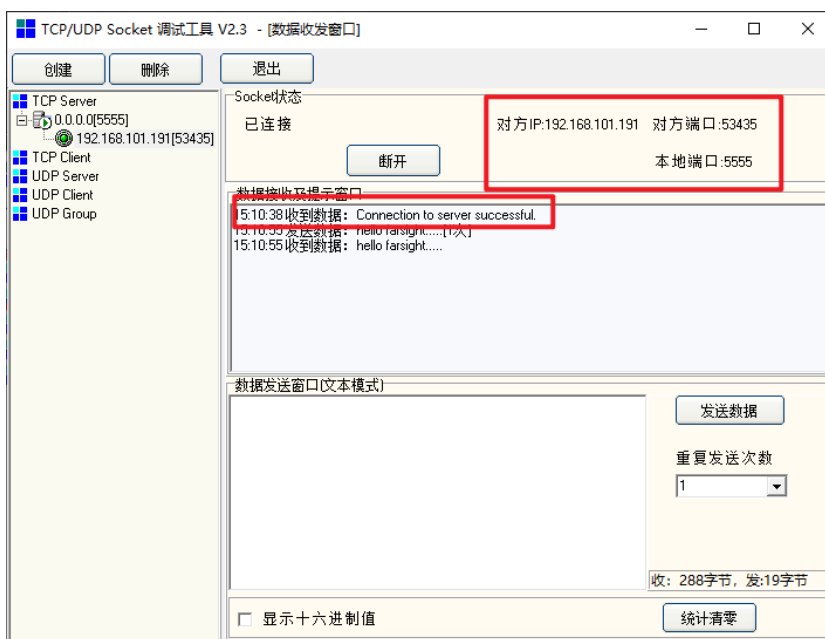


图4-13 TCP 服务器收到 Hi3861 的数据

3. 然后在“数据发送窗口中”，输入一串字符串数据，然后在串口调试助手中可以看到数据。

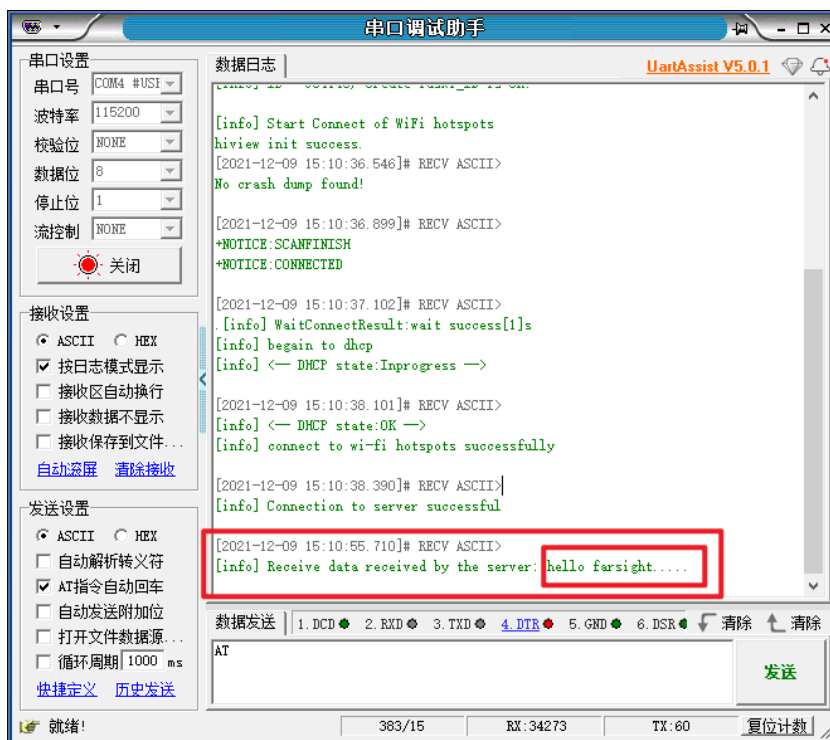


图4-14 Hi3861 TCP 客户端接收到数据



3.5 Wi-Fi 之 MQTT 通信实验

3.5.1 实验目的

1. 学习如何移植第三方开源库。
2. 学习 MQTT 通信流程。
3. 熟悉如何使用 VScode 编辑软件。
4. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
5. 检测搭建的软件及硬件环境是否可正常使用。

3.5.2 实验要求

连接 PC 端的 MQTT 服务器，然后进行订阅主题和发布主题。

3.5.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\network_05_wifi_mqtt】

3.5.4 实验原理

MQTT 是机器对机器(M2M)/物联网(IoT)连接协议。它被设计为一个极其轻量级的发布/订阅消息传输协议。对于需要较小代码占用空间和/或网络带宽非常宝贵的远程连接非常有用，是专为受限设备和低带宽、高延迟或不可靠的网络而设计。这些原则也使该协议成为新兴的“机器到机器”(M2M)或物联网(IoT)世界的连接设备，以及带宽和电池功率非常高的移动应用的理想选择。例如，它已被用于通过卫星链路 with 代理通信的传感器、与医疗服务提供者的拨号连接，以及一系列家庭自动化和小型设备场景。它也是移动应用的理想选择，因为它体积小，功耗低，数据包最小，并且可以有效地将信息分配给一个或多个接收器。MQTT 通信模型如下图所示：

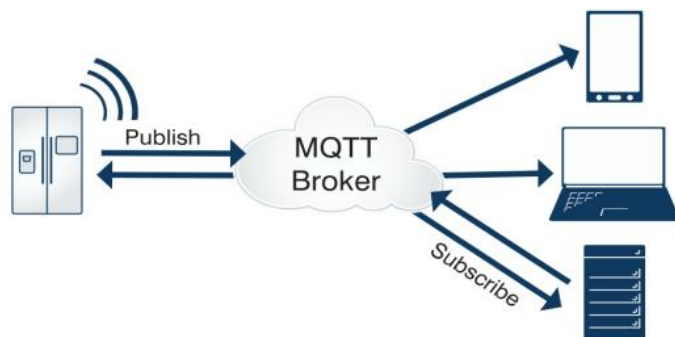


图4-15 MQTT 通信机制

● 特点：

1. 开放消息协议，简单易实现；



2. 发布订阅模式，一对多消息发布；
3. 基于 TCP/IP 网络连接，提供有序，无损，双向连接；
4. 1 字节固定报头，2 字节心跳报文，最小化传输开销和协议交换，有效减少网络流量；
5. 消息 QoS 支持，可靠传输保证；

● 应用：

MQTT 协议广泛应用于物联网、移动互联网、智能硬件、车联网、电力能源等领域。物联网 M2M 通信，物联网大数据采集；Android 消息推送，WEB 消息推送；移动即时消息，例如 Facebook Messenger；智能硬件、智能家居、智能电器；车联网通信，电动车站桩采集；

3.5.5 实验步骤

步骤1: 下载 pahomqtt 库，添加在 HarmonyOS 代码库中

paho mqtt-c 是基于 C 语言实现的 MQTT 客户端，在 GitHub 中下载源码，我们已经下载好了，放在了工具软件中，路径为【FS_Hi3861 开发板光盘资料\工具软件\paho.mqtt.embedded-c-master.zip】。

1. 将这个文件拷贝到 Ubuntu 系统中，然后输入命令“unzip 文件名.zip”进行解压：

```
unzip paho.mqtt.embedded-c-master.zip
```

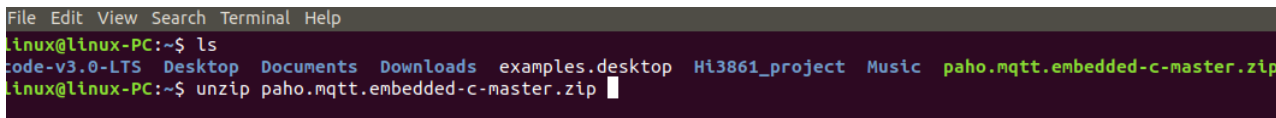


图4-16 解压 pahomqtt 库

2. 将解压完成的目录拷贝到 HarmonyOS 系统目录下的 third_party 目录下：

```
mv paho.mqtt.embedded-c-master/ code-v3.0-LTS/OpenHarmony/third_party/
```

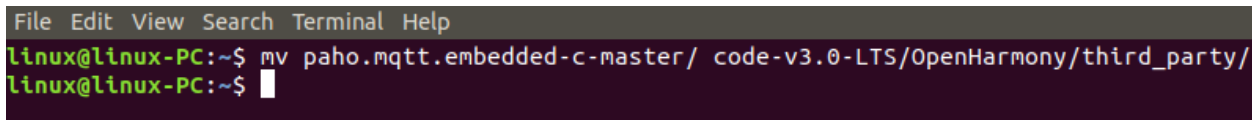


图4-17 移动 pahomqtt 目录到 third_party 目录下

3. 拷贝完成之后，需要进行改名，输入以下命令：

```
mv paho.mqtt.embedded-c-master/ pahomqtt
```

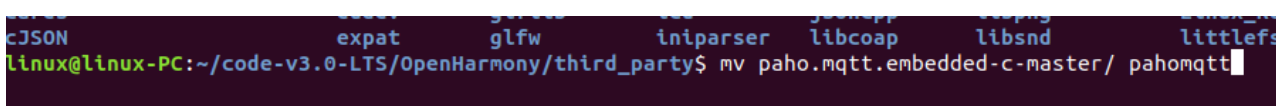


图4-18 将 paho.mqtt.embedded-c-master 改名为 pahomqtt

步骤2: 新建 pahomqtt 的 BUILD.gn 文件

1. 进入到 pahomqtt 目录下，在此目录下新建一个 BUILD.gn 文件，此文件用来将 mqtt 库文件参与构建和编译。在此文件中添加内容：



```

1 if (defined(ohos_lite)) {
2   import("//build/lite/config/component/lite_component.gni")
3   import("//build/lite/ndk/ndk.gni")
4   config("pahomqtt_config") {
5     include_dirs = [
6       "MQTTPacket/src",
7       "MQTTClient-C/src",
8       "//third_party/lwip/src/include",
9       "//kernel/liteos_m/kal/cmsis",
10    ]
11  }
12
13  pahomqtt_sources = [
14    "MQTTPacket/src/MQTTConnectClient.c",
15    "MQTTPacket/src/MQTTConnectServer.c",
16    "MQTTPacket/src/MQTTDeserializePublish.c",
17    "MQTTPacket/src/MQTTFormat.c",
18    "MQTTPacket/src/MQTTPacket.c",
19    "MQTTPacket/src/MQTTSerializePublish.c",
20    "MQTTPacket/src/MQTTSubscribeClient.c",
21    "MQTTPacket/src/MQTTSubscribeServer.c",
22    "MQTTPacket/src/MQTTUnsubscribeClient.c",
23    "MQTTPacket/src/MQTTUnsubscribeServer.c",
24  ]
25
26  lite_library("pahomqtt_static") {
27    target_type = "static_library"
28    sources = pahomqtt_sources
29    public_configs = [ ":pahomqtt_config" ]
30  }
31
32  lite_library("pahomqtt_shared") {
33    target_type = "shared_library"
34    sources = pahomqtt_sources
35    public_configs = [ ":pahomqtt_config" ]
36  }
37
38  ndk_lib("pahomqtt_ndk") {
39    if (board_name != "hi3861v100") {
40      lib_extension = ".so"
41      deps = [
42        ":pahomqtt_shared"
43      ]
44    } else {

```



```

45     deps = [
46         ":pahomqtt_static"
47     ]
48 }
49 head_files = [
50     "//third_party/pahomqtt"
51 ]
52 }
53 }
54

```

2. 打开“device/hisilicon/hispark_pegasus/sdk_liteos/BUILD.gn”文件，在下图部分添加“//third_party/pahomqtt:pahomqtt_static”，第三方库添加完成了，接下来就可以进行使用了。

```

33 lite_component("sdk") {
34     features = []
35
36     deps = [
37         "//build/lite/config/component/cJSON:cjson_static",
38         "//device/hisilicon/hispark_pegasus/hi3861_adapter/kal",
39         "//third_party/a_myparty:a_myparty_static",
40         "//third_party/pahomqtt:pahomqtt_static",
41     ]
42 }
43

```

图4-19 在 sdk 软件包中添加 pahomqtt 静态库

步骤3: 编写 MQTT 函数

1. 在“base/iot_hardware/bsp/include”目录中新建文件 hal_bsp_mqtt.h，然后在该文件中填写内容：

```

1 #ifndef __HAL_BSP_MQTT_H
2 #define __HAL_BSP_MQTT_H
3
4 /**
5  * @brief MQTT 连接 MQTT 服务器
6  * @param ip_addr 服务器的 IP 地址
7  * @param ip_port 服务器的 IP 端口号
8  * @return Returns {0} 成功;
9  *         Returns {-1} 失败.
10 */
11 int mqttClient_connectServer(const char *ip_addr, int ip_port);
12 /**
13  * @brief MQTT 断开连接 MQTT 服务器

```



```

14 * @param 无
15 * @return Returns {0} 成功;
16 *       Returns {-1} 失败.
17 */
18 int mqttClient_unConnectServer(void);
19 /**
20 * @brief MQTT 订阅 MQTT 主题
21 * @param subTopic 具有订阅权限的主题名称
22 * @return Returns {0} 成功;
23 *       Returns {-1} 失败.
24 */
25 int mqttClient_subscribe(char *subTopic);
26 /**
27 * @brief MQTT 客户端的初始化
28 * @param clientID 客户端 ID
29 * @param userName 用户名
30 * @param password 密码
31 * @return Returns {0} 成功;
32 *       Returns {-1} 失败.
33 */
34 int mqttClient_init(char *clientID, char *userName, char *password);
35 /**
36 * @brief MQTT 发布消息
37 * @param pub_Topic 具有发布权限的主题名称
38 * @param payloadData 发布数据
39 * @param payloadLen 发布数据的长度
40 * @return Returns {0} 成功;
41 *       Returns {-1} 失败.
42 */
43 int mqttClient_pub(char *pub_Topic, unsigned char *payloadData, int payloadLen);
44 /**
45 * @brief MQTT 接收消息
46 * @param callback 当接收到消息之后, 将消息传到回调函数中
47 * @return Returns {0} 成功;
48 *       Returns {-1} 失败.
49 */
50 int mqttClient_sub(int (*callback)(unsigned char *payload, int len));
51
52 #endif // !__HAL_BSP_MQTT_H
53

```

2. 在“base/iot hardware/bsp/src”目录中新建文件 hal_bsp_mqtt.c, 然后在该文件中填写内容:

注意: 下面显示出主要代码, 其余的请查看源码。



步骤4: 编写应用程序

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 network_05_wifi_mqtt 目录，在此目录中创建 network_wifi_mqtt_example.c 文件和 BUILD.gn 文件。

2. 编写 network_wifi_mqtt_example.c 文件：

注意：下面显示出主要代码，其余的请查看源码。

3. 编写 BUILD.gn 文件

```
1 static_library("network_wifi_mqtt_example") {
2     sources = [
3         "network_wifi_mqtt_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_wifi.c",
5         "//base/iot_hardware/bsp/src/hal_bsp_mqtt.c"
6     ]
7
8     include_dirs = [
9         "//utils/native/lite/include",
10        "//kernel/liteos_m/kal/cmsis",
11        "//base/iot_hardware/peripheral/interfaces/kits",
12        "//foundation/communication/wifi_lite/interfaces/wifiservice",
13        "//base/iot_hardware/bsp/include",
14        "//third_party/pahomqtt/MQTTpacket/src",
15        "//third_party/cJSON",
16    ]
17 }
18
```

4. 修改 “/applications/sample/wifi-iot/app/BUILD.gn” 文件，将这句话取消注释，其余的全部注释。

```
# 网络相关实验
# "network_01_wifi_ap:network_wifi_ap_example",
# "network_02_wifi_sta:network_wifi_sta_example",
# "network_03_wifi_udp:network_wifi_udp_example",
# "network_04_wifi_tcp:network_wifi_tcp_example",
"network_05_wifi_mqtt:network_wifi_mqtt_example",
```

图4-20 更改 BUILD.gn 文件

步骤5: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤6: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。



3.5.6 实验现象

1. 打开【桌面\MQTT.fx 调试工具软件】，配置如下，需要注意的是，MQTT 调试软件中的 Broker Address 地址与应用程序中的 IP 地址是一样的。然后点击 OK。

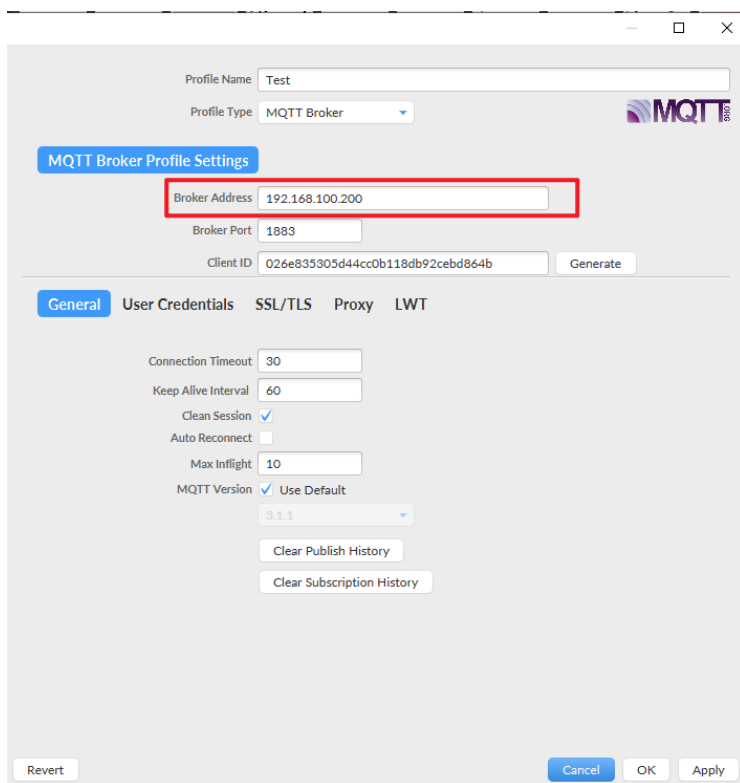


图4-21 更改 Broker Address 地址

2. 然后在 MQTT 调试工具中，订阅一个新的主题：pubTopic，此时复位开发板，然后就可以在 MQTT 调试工具中看到数据。

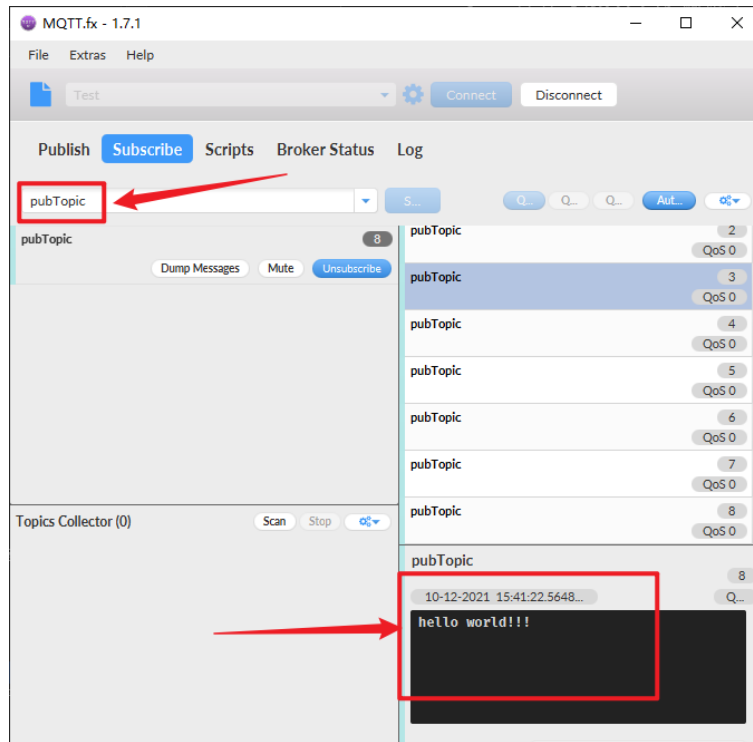


图4-22 订阅主题

3. 然后我们在 Publish 窗口下，输入要发送的数据 “hello world~~~~~”

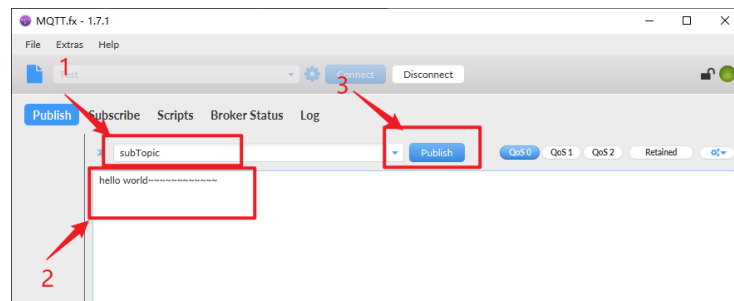


图4-23 发布主题

4. 点击发布 “Publish” 后，会在串口调试助手看到收到的数据：

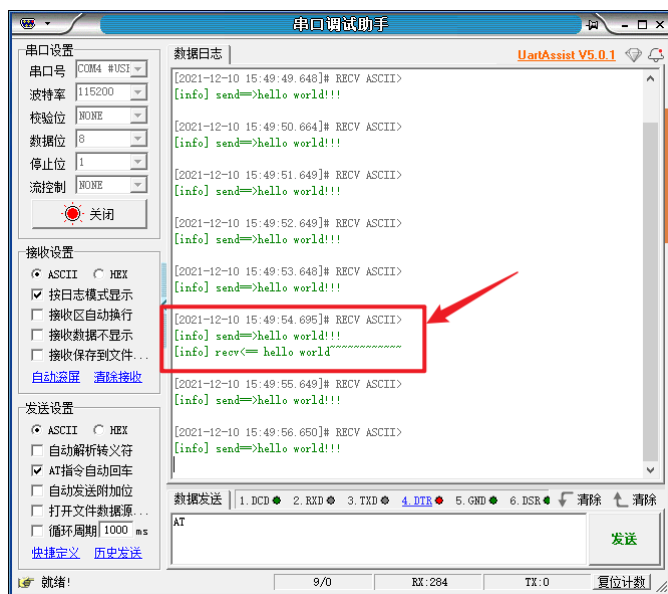


图4-24 在串口调试助手中查看接收数据

3.6 Wi-Fi 之 NFC 配网实验

3.6.1 实验目的

1. 学习如何使用 NFC 芯片进行碰一碰配网。
2. 熟悉如何使用 VScode 编辑软件。
3. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
4. 检测搭建的软件及硬件环境是否可正常使用。

3.6.2 实验要求

使用《智能照明灯》项目中的微信小程序源码，在手机端预览出界面，然后输入正确的 WiFi 名称和密码，进行 NFC 配网实验。

3.6.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\network_06_wifi_nfc_sta】

3.6.4 实验原理

NFC 是一种近场无线通信技术，通信距离理论上可以在 10cm（实际中需要贴的很近），通信频段为 13.56MHz，与现有的非接触式智能卡国际标准相兼容。数据传输速率有 106kbit/s、212kbit/s、424kbit/s。通信原理是基于感应近场，在近场区域内感应场强弱与电磁辐射源以及天线的距离相关，近则强远则弱。

手机端与设备端之间使用了 NFC 的 Ndef 通信协议，通信协议的格式如下图所示：

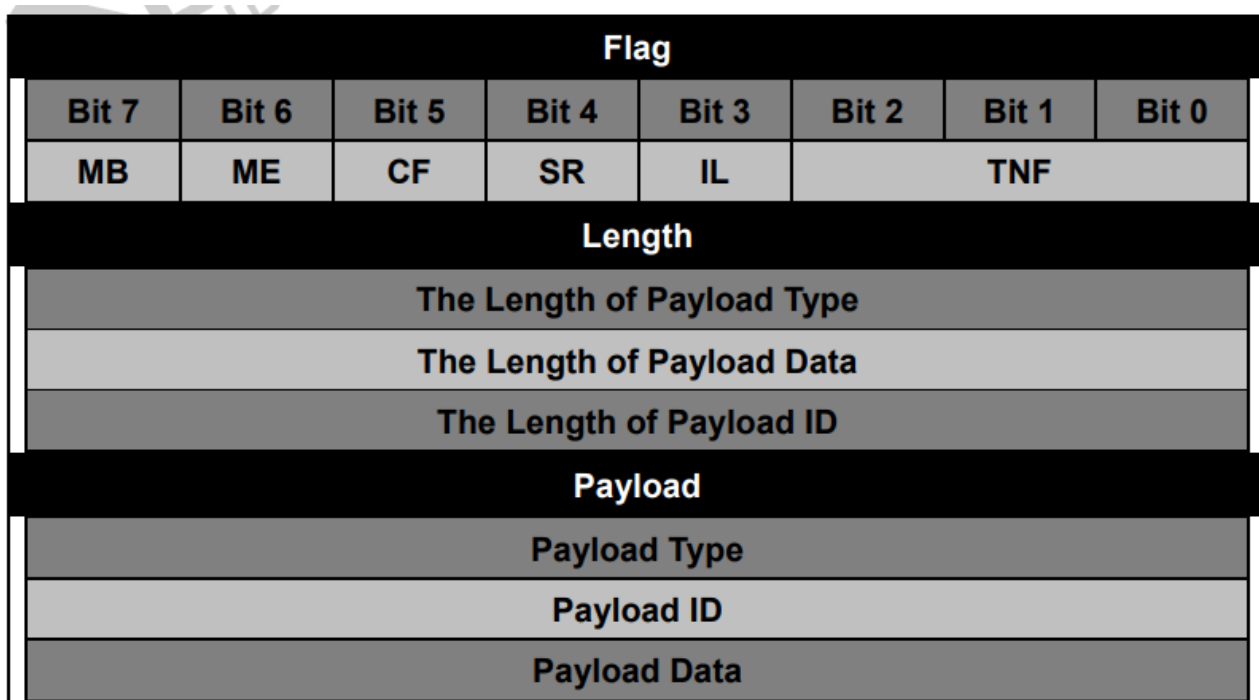


图4-25 NDEF 通信协议格式

- MB 是消息开始的标志，MB=1 表示这是 NDEF 消息的第 1 个记录；
- ME 是消息结束的标志，ME=1 表示这是 NDEF 消息的最后 1 个记录；如果为 0，表示后面还有很多；
- CF 是消息分块的标志，CF=0 表示这不是分块消息，CF=1 表示这是分块消息；
- SR 是短记录标志，SR=1 说明内容长度域是 1 个字节，短记录用来为长度不到 255 个字节的内容做精简封装。
- IL 是 ID 的长度，IL=1，表示 ID 域出现在头部，长度为 1 个字节；IL=0 表示 ID 域不出现在头部；
- TNF 域表示 TYPE 域值的结构

具体通信协议的详情可查看官方文档：【Hi3861 开发板资料\芯片手册\芯片\NFC Data Exchange Format (NDEF).pdf】

3.6.5 实验步骤

步骤1: 编写应用程序

1. 在 “/applications/sample/wifi-iot/app/” 目录中创建 network_06_wifi_nfc_sta 目录，在此目录中创建 network_wifi_nfc_example.c 文件和 BUILD.gn 文件。

2. 编写 network_wifi_nfc_example.c 文件：

注意：下面显示出主要代码，其余的请查看源码。

```
1 osThreadId_t Task1_ID; // 任务 1 设置为低优先级任务
2 void Task1(void *argument)
3 {
```



```

4  uint8_t ret, num = 0;
5  uint8_t ndefLen = 0;           // ndef 包的长度
6  uint8_t ndef_Header = 0;      // ndef 消息开始标志位-用不到
7  uint32_t result_code = 0;    // 函数的返回值
8
9  nfc_Init();
10 console_log_info("I2C Test Start");
11 sleep(3);
12
13 // 读整个数据的包头部分，读出整个数据的长度
14 if (result_code = NT3HReadHeaderNfc(&ndefLen, &ndef_Header) != true)
15 {
16     console_log_error("NT3HReadHeaderNfc is failed. result_code = %d", result_code);
17     return ;
18 }
19
20 ndefLen += NDEF_HEADER_SIZE;    // 加上头部字节
21
22 uint8_t *ndefBuff = (uint8_t*)malloc(ndefLen+1);
23 if(ndefBuff == NULL)
24 {
25     console_log_error("ndefBuff malloc is Falied!");
26     return ;
27 }
28
29 if(result_code = get_NDEFDataPackage(ndefBuff, ndefLen) != HI_ERR_SUCCESS)
30 {
31     console_log_error("get_NDEFDataPackage is failed. result_code = %d", result_code);
32     return ;
33 }
34
35 console_log_info("start print ndefBuff.");
36 for (size_t i = 0; i < ndefLen; i++)
37 {
38     printf("0x%x ", ndefBuff[i]);
39 }
40
41 NFC_configuresWiFiNetwork(ndefBuff);
42
43 while (1)
44 {
45     sleep(1);
46 }
47 }

```



3. 编写 BUILD.gn 文件

```

1 static_library("network_wifi_nfc_example") {
2     sources = [
3         "network_wifi_nfc_example.c",
4         "//base/iot_hardware/bsp/src/hal_bsp_wifi.c",
5         "//base/iot_hardware/bsp/src/hal_bsp_nfc.c",
6         "//base/iot_hardware/bsp/src/hal_bsp_nfc_to_wifi.c",
7         "//base/iot_hardware/bsp/nfc/NT3H.c",
8         "//base/iot_hardware/bsp/nfc/ndef/rtd/nfcForum.c",
9         "//base/iot_hardware/bsp/nfc/ndef/rtd/rtdText.c",
10        "//base/iot_hardware/bsp/nfc/ndef/rtd/rtdUri.c",
11        "//base/iot_hardware/bsp/nfc/ndef/ndef.c",
12    ]
13
14    include_dirs = [
15        "//utils/native/lite/include",
16        "//kernel/liteos_m/kal/cmsis",
17        "//base/iot_hardware/peripheral/interfaces/kits",
18        "//foundation/communication/wifi_lite/interfaces/wifiservice",
19        "//base/iot_hardware/bsp/include",
20        "//third_party/cJSON",
21        "//base/iot_hardware/bsp/nfc/ndef",
22        "//base/iot_hardware/bsp/nfc/ndef/rtd/",
23        "//base/iot_hardware/bsp/nfc"
24    ]
25 }

```

4. 修改 “/applications/sample/wifi-iot/app/BUILD.gn” 文件，将这句话取消注释，其余的全部注释。

```

1 import("//build/lite/config/component/lite_component.gni")
2
3 lite_component("app") {
4     features = [
5
6         "network_06_wifi_sta: network_wifi_nfc_example",
7     ]
8 }

```

步骤2: 编译工程

按照[第 1 章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤3: 烧写程序

按照[第 1 章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。



步骤4: 编译微信小程序

注意：前提需要在电脑上安装“微信开发者工具”。

使用“微信开发者工具”打开《智能照明灯》项目的微信小程序源码，源码目录为【程序源码\综合程序源码\智能照明灯\微信小程序端】，然后双击“app.wxss”文件，即可使用微信开发者工具打开工程，如下图所示：

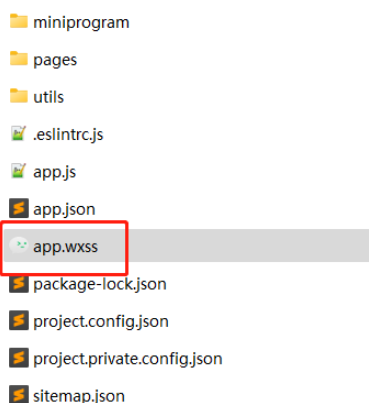


图4-26 微信小程序工程

双击打开之后，我们点击开发者工具的上方，有一个“编译”按钮，点击后进行编译。如下图所示：

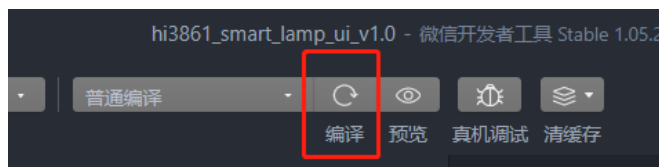


图4-27 编译

编译完成之后，再点击“预览”按钮，然后使用手机，打开微信进行扫码预览，接下来我们开始调试。

3.6.6 实验现象

1. 打开手机上的 NFC 开关，然后点击微信小程序中“NFC 配网”按钮，进入到 NFC 配网界面，在此界面中输入可以上网的 WiFi 名称和密码，如下图所示：



图4-28 输入 WiFi 名称和密码



2. 输入密码完成之后，然后点击“开始配置”按钮，此时会在手机窗口中弹出“请靠近设备”提示框。



图4-29 手机靠近 NFC 设备

3. 手机靠近设备，稍等几秒钟，即可烧写成功，然后可以在手机端看到“写入数据成功”的弹窗。



图4-30 写入数据成功

4. 烧写 Hi3861 开发板的 NFC 配网程序成功之后，我们打开串口调试助手，在串口调试助手中看 hi3861 的运行信息，看看设备是否连接 WiFi 成功。

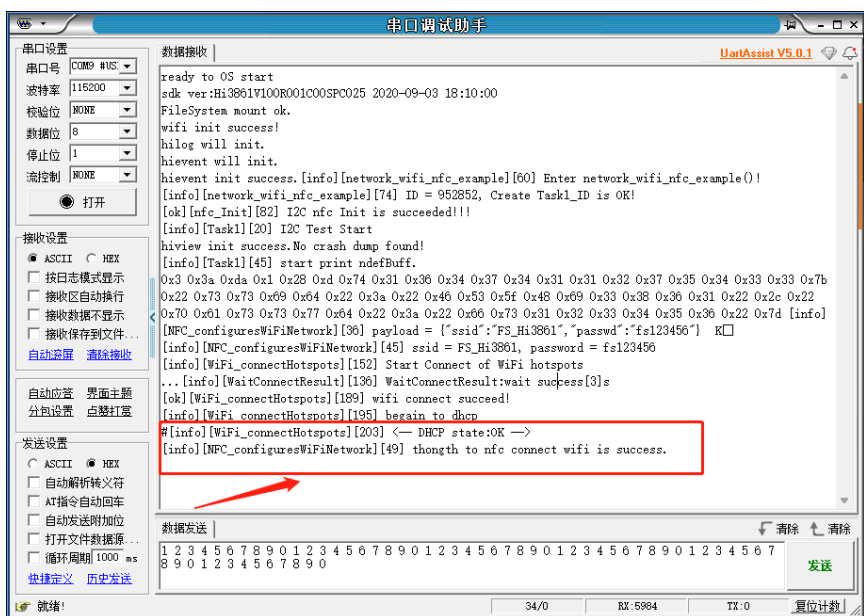


图4-31 连接 WiFi 成功

3.7 Wi-Fi 之连接华为 IoTDA 设备接入

3.7.1 实验目的

1. 学习华为 IoTDA 物联网平台的使用。
2. 学习 JSON 数据的解析和封装。
3. 学习 MQTT 通信协议的通信机制。
4. 熟悉如何使用 VScode 编辑软件。
5. 熟悉 HarmonyOS 系统的编译流程和设计步骤，能够进行设计、编程、调试。
6. 检测搭建的软件及硬件环境是否可正常使用。

3.7.2 实验要求

在华为云 IoTDA 物联网平台中可以看到开发板上传的数据，并且可以控制开发板中的风扇、蜂鸣器、LED 灯和 RGB 灯。

3.7.3 实验环境

1. 硬件环境：FS_Hi3861 开发板、Type-C 串口线。
2. 软件环境：Ubuntu 系统、Vscode 编辑软件。
3. 源码位置：【程序源码\基础实验\cloud_01_HuaWei_IoTDA】

3.7.4 实验原理

华为云物联网平台（IoT 设备接入云服务）提供海量设备的接入和管理能力，可以将自己的 IoT 设备连接到华为云，支撑设备数据采集上云和云端下发命令给设备进行远程控制，配合华为云物联网平台的服务实现设备与设备之间的控制，设备与物联网平台之间的数据信息交互。

在本实验中，将板载资源上的传感器数据传到华为云物联网平台与上并且可以下发指令控制板载资源上风扇、蜂鸣器、LED 灯和 RGB 灯等。查看相关文档可以查看华为云官网：<https://support.huaweicloud.com/iothub/> 下面让我们开始吧！

3.7.5 实验步骤

步骤1: 注册华为云账号并登录

1. 打开华为云物联网平台的网址<https://www.huaweicloud.com/>，打开网址之后，点击右上角的“注册”。



图4-32 注册华为云

2. 在“注册”页面中，填入手机号进行注册，所有信息填写完成之后，点击“注册按钮”进行注册，注册成功之后，进入下一步。



图4-33

3. 注册完成之后，再次进入华为云的官网进行登录。



图4-34 登录华为云

4. 在登录界面，填入刚刚注册的“手机号”、“华为账号”和“密码”，然后点击“登录”。



图4-35 输入自己的账号进行登录

5. 登录成功之后，点击右上角的“控制台”。



图4-36 点击“控制台”

6. 在控制台的左上角，因为“标准版免费试用仅对企业用户开放，个人用户建议使用北京四基础版”，所以选择地区为“北京四”，然后点击左侧“服务列表”。



图4-37 切换为“北京四”

7. 在“服务列表”的搜索框中搜索“IoTDA”，然后点击“设备接入 IoTDA”。



图4-38 搜索找到“IoT 物联网”

8. 点击“平台接入地址”，需要在记事本记录“MQTT 协议设备接入 IP 地址和端口号”。



图4-39 查看平台接入地址

9. 我们只需要使用设备接入的 MQTT 协议就可以，记住这个服务器的地址。

接入信息			
接入类型	接入协议 (端口号)	接入地址	操作
应用接入	HTTPS (443)	a1612c3e69.iotda.cn-north-4.myhuaweicloud.com	📄
	AMQPS (5671)	a1612c3e69.iot-amqps.cn-north-4.myhuaweicloud.com	📄 预置接入凭证 ①
	CoAP (5683) CoAPS (5684)	a1612c3e69.iot-coaps.cn-north-4.myhuaweicloud.com	📄
设备接入	HTTPS (8943)	a1612c3e69.iot-https.cn-north-4.myhuaweicloud.com	📄
	MQTT (1883) MQTTS (8883)	a1612c3e69.iot-mqtt.cn-north-4.myhuaweicloud.com	📄

图4-40 使用 MQTT 的接入方式

步骤2: 创建新产品

1. 点击左侧的产品，点击右上角的创建产品按钮。



图4-41 创建产品

2. 选择所属的资源，产品名称填写 Smart_fan，数据格式默认，厂商名称和设备类型任意，点击确定。



图4-42 填写产品的属性

3. 在左侧菜单栏中，找到所有设备，点击注册设备。



图4-43 注册设备



4. 选择所属资源空间，所属产品，填写设备标识码为 hi3861，密钥由系统自动产生。

The form is titled "单设备注册" (Single Device Registration). It contains the following fields and options:

- 所属资源空间 (Resource Space):** DefaultApp_612cyins
- 所属产品 (Product):** Hi3861_comprehensive
- 设备标识码 (Device ID):** hi3861
- 设备名称 (Device Name):** hi3861
- 设备认证类型 (Device Authentication Type):** 密钥 (Key) [Selected], X.509证书 (X.509 Certificate)
- 密钥 (Key):** 密钥由系统自动生成 (Key generated by system)
- 确认密钥 (Confirm Key):** 密钥由系统自动生成 (Key generated by system)

Buttons at the bottom: 确定 (Confirm), 取消 (Cancel).

图4-44 注册设备的属性

注意：生成好的设备 ID 和设备密钥一定到妥善保管，只生成这一份。

The confirmation screen shows the following information:

- 设备ID (Device ID):** 60765321ce15da02c033a422_Device_Stm32Wifi
- 设备密钥 (Device Key):** b5ff115f32b6dc717e6a4746bce2d984

A warning box states: "为保证安全，密钥在设备详情页内不可见，如遗忘密钥，可通过“概述 -> 重置密钥”操作进行重置。" (For security, the key is not visible in the device details page. If you forget the key, you can reset it via "Overview -> Reset Key").

Button: 保存并关闭 (Save and Close).

图4-45 生成好的密钥信息

步骤3: 创建产品模型

1. 点击左侧“产品”，点击设备新创建好的产品名称“Hi3861_comprehensive”

产品名称	产品ID	资源空间	设备类型	协议类型	操作
Hi3861_comprehensive	61b30c8a4d9b020287fe35c9	DefaultApp_612cyins	Hi3861_WiFi	MQTT	查看 删除
Smart_fan	616e652b88056b027df2dd28	DefaultApp_612cyins	123	MQTT	查看 删除
beapi	612c76842cce4f0286217bc9	DefaultApp_612cyins	1	MQTT	查看 删除

Footer: 10 | 总数: 3 | < 1 >

图4-46 创建好的产品



2. 点击自定义模型。

模型定义 | 在线调试 | Topic 管理



图4-47 创建自定义模型

3. 填写服务 ID 和服务类型。

添加服务

★ 服务ID

服务类型

服务描述

0/128

确定

取消

图4-48 填写服务 ID 和服务类型

4. 在此服务下“添加属性”，用于上传开发板上的数据，以风扇为例，其他的传感器也是类似的方法。



新增属性

* 属性名称

属性描述
5/128

* 数据类型

* 访问权限 ☒ 可读 ☐ 可写

* 长度

枚举值
6/1,024

图4-49 添加属性

5. 展示创建完成的属性列表

☐ 属性名称	数据类型	访问方式	描述	操作
☐ buzzer	string(字符串)	可读	蜂鸣器的状态	复制 修改 删除
☐ fan	string(字符串)	可读	风扇的状态	复制 修改 删除
☐ led	string(字符串)	可读	LED灯的状态	复制 修改 删除
☐ temperature	decimal(小数)	可读	温度	复制 修改 删除
☐ humidity	decimal(小数)	可读	湿度	复制 修改 删除
☐ light	int(整型)	可读	光照强度	复制 修改 删除
☐ proximity	int(整型)	可读	接近传感器	复制 修改 删除
☐ infrared	int(整型)	可读	人体红外	复制 修改 删除

10 ▼ 总条数: 8 < 1 >

图4-50 创建完成的属性列表

6. 在此服务下“添加命令”，用于控制开发板上可以控制的设备，以风扇为例，其他的传感器也是类似的方法。



新增命令

* 命令名称:

1 下发参数:

2 **新增参数**

参数名称:

* 参数名称: value

参数描述:

0/128

响应参数:

参数名称:

* 数据类型:

4

* 长度:

5

6 枚举值:

6/1,024

7

led

RGB

图4-51 添加命令

7. 展示创建完成的命令列表

添加命令			
命令名称	下发参数	响应参数	操作
fan	value		复制 修改 删除
buzzer	value		复制 修改 删除
led	value		复制 修改 删除
RGB	red,green,blue		复制 修改 删除

10 总条数: 4 < 1 >

图4-52 命令列表

步骤4: 根据设备信息, 生成用户名和密码

1. 打开由华为云提供的生成用户名和密码的网址, 进行生成。

<https://iot-tool.obs-website.cn-north-4.myhuaweicloud.com/>

2. 填写生成好的设备 ID 和设备秘钥, 点击按钮, 生成用户名和密码。

注意: 下面的图片提供了方法。



Huaweicloud IoTDA Mqtt ClientId Generator!

这是由华为云设备接入提供的MQTT ClientId生成工具，设备连接鉴权具体生成算法可以点击下方按钮了解更多

[了解更多](#)

DeviceId

60765321ce15da02c033a422 Device Stm32Wifi

DeviceSecret

b5ff115f32b6dc717e6a4746bce2d984

[Generate](#)

ClientId

60765321ce15da02c033a422 Device Stm32Wifi 0 0 2021041403

Username

60765321ce15da02c033a422 Device Stm32Wifi

Password

707bec4112a7ef0a2517eb1ed1851a35cf2a12c12968259c6967f1e1ae0ed784

图4-53 生成设备用户名和密码信息

3. 将生成好的信息（ClientId、Username、Password），修改文件“/applications/sample/wifi-iot/app/ cloud_huawei_iotda_example.c”，下面内容直接替换。

```

22 // 设备ID
23 #define DEVICE_ID "*****"
24 // MQTT客户端ID
25 #define MQTT_CLIENT_ID "*****_0_0_2021121009"
26 // MQTT用户名
27 #define MQTT_USER_NAME "*****"
28 // MQTT密码
29 #define MQTT_PASS_WORD "37a8f2bd3325293d11d646a023bbf9ec19ff717f549db87782749effb81fa271"
    
```

图4-54 替换代码中的 ClientID、UserName、Password

4. 修改华为云平台的 MQTT 服务器的 IP 地址，打开 Windows 下 cmd 控制端，在控制端输入内容。

```

Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

尝试新的跨平台 PowerShell https://aka.ms/pscore6

PS C:\Users\Administrator> ping al612c3e69.iot-mqtt.cn-north-4.myhuaweicloud.com

正在 Ping al612c3e69.iot-mqtt.cn-north-4.myhuaweicloud.com [121.36.42.100] 具有 32 字节的数据:
来自 121.36.42.100 的回复: 字节=32 时间=12ms TTL=1
来自 121.36.42.100 的回复: 字节=32 时间=13ms TTL=1

121.36.42.100 的 Ping 统计信息:
    数据包: 已发送 = 2, 已接收 = 2, 丢失 = 0 (0% 丢失),
    往返行程的估计时间(以毫秒为单位):
        最短 = 12ms, 最长 = 13ms, 平均 = 12ms
Control-C
PS C:\Users\Administrator>
    
```

保存这个IP地址

图4-55 查看华为云 MQTT 服务器的 IP 地址

5. 然后修改 cloud_huawei_iotda_example.c 文件中 IP 地址和端口号。

```

30 // 华为云平台的IP地址
31 #define SERVER_IP_ADDR "121.36.42.100"
32 // 华为云平台的IP端口号
33 #define SERVER_IP_PORT 1883
    
```

图4-56 修改 IP 地址和端口号



6. 向云端设置属性的数据格式如下所示：

```
1{
2
3
4
5
6
7
8
9
10
11
12
13
14
15}
16
```

```
"services": [{
  "service_id": "attribute",
  "properties": {
    "buzzer": "ON",
    "fan": "OFF",
    "led": "OFF",
    "temperature": 36.5,
    "humidity": 78.9,
    "light": 1567,
    "proximity": 125,
    "infrared": 12
  }
}]
```

► 修改“/applications/sample/wifi-iot/app/BUILD.gn”文件，将这句话取消注释，其余的全部注释。

```
# 网络相关实验
# "network_01_wifi_ap:network_wifi_ap_example",
# "network_02_wifi_sta:network_wifi_sta_example",
# "network_03_wifi_udp:network_wifi_udp_example",
# "network_04_wifi_tcp:network_wifi_tcp_example",
"network_05_wifi_mqtt:network_wifi_mqtt_example",
```

图4-57 修改 BUILD.gn 文件

步骤5: 编译工程

按照[第1章 开发环境搭建中“编译源码”小节的内容](#)进行编译工程。

步骤6: 烧写程序

按照[第1章 开发环境搭建中“烧写程序”小节的内容](#)进行烧写程序。

3.7.6 实验现象

1. 按照[第5.2节烧写程序](#)中的步骤进行烧写程序，烧写完成之后，打开串口调试助手，可以查看到当前设备连接 MQTT 服务器的流程，如下图所示：

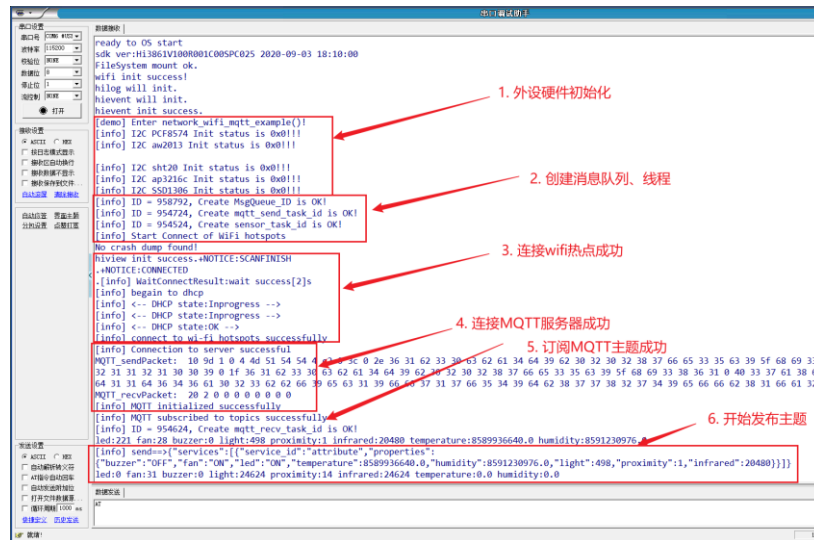


图4-58 串口调试助手的显示数据

2. 此时在华为云 IoTDA 平台中看到数据，点击刷新，可以看到最新的数据。

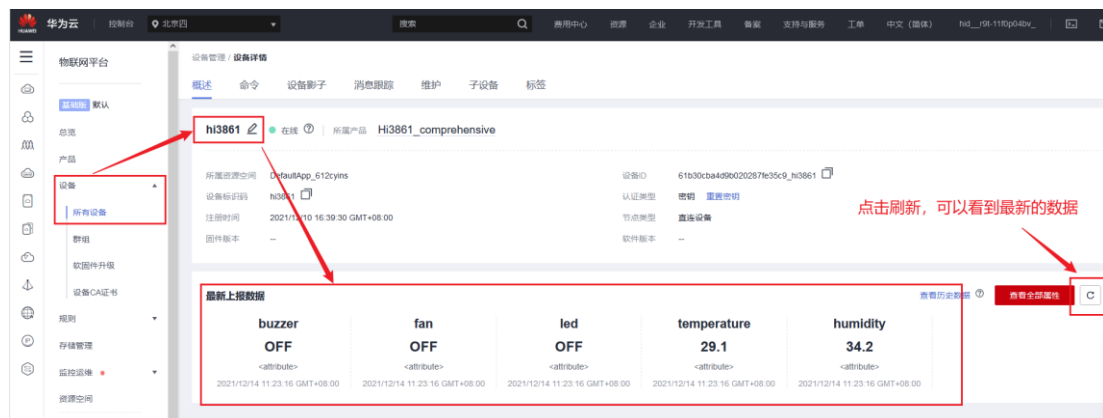


图4-59 在物联网平台查看数据

3. 下面测试命令下发功能，如下图所示步骤，进行测试：



图4-60 测试下发命令



第4章 常见问题

4.1 在添加第三方库完成后，再次编译报错，提示“undefined reference to *****”

● 现象描述：

比如是移植了 MQTT 的库文件，然后再次编译工程的时候，会发现下面的问题：

```
[OHOS ERROR] riscv32-unknown-elf-ld: ohos/libs/libnetwork_wifi_mqtt_example.a(libnetwork_wifi_mqtt_example.hal_bsp_mqtt.o): in
function `L37':
[OHOS ERROR] hal_bsp_mqtt.c:(.text.MQTTClient_init+0x1a2): undefined reference to `MQTTDeserialize_connack'
[OHOS ERROR] riscv32-unknown-elf-ld: ohos/libs/libnetwork_wifi_mqtt_example.a(libnetwork_wifi_mqtt_example.hal_bsp_mqtt.o): in
function `L53':
[OHOS ERROR] hal_bsp_mqtt.c:(.text.MQTTClient_pub+0x7a): undefined reference to `MQTTSerialize_publish'
[OHOS ERROR] riscv32-unknown-elf-ld: ohos/libs/libnetwork_wifi_mqtt_example.a(libnetwork_wifi_mqtt_example.hal_bsp_mqtt.o): in
function `MQTTClient_sub':
[OHOS ERROR] hal_bsp_mqtt.c:(.text.MQTTClient_sub+0x32): undefined reference to `MQTTPacket_read'
[OHOS ERROR] riscv32-unknown-elf-ld: hal_bsp_mqtt.c:(.text.MQTTClient_sub+0x5e): undefined reference to `MQTTDeserialize_publi
sh'
[OHOS ERROR] scons: *** [output/bin/Hi3861_wifiot_app.out] Error 1
[OHOS ERROR] BUILD FAILED!!!!
[OHOS ERROR] Failed building output/bin/Hi3861_wifiot_app.out: Error 1
[OHOS ERROR] you can check build log in /home/book/work/code-v3.0-LTS/OpenHarmony/out/hispanic_pegasus/wifiot_hispanic_pegasus/
build.log
[OHOS ERROR] command: "/home/book/work/code-v3.0-LTS/OpenHarmony/prebuilts/build-tools/linux-x86/bin/ninja -w dupbuild=warn -C
/home/book/work/code-v3.0-LTS/OpenHarmony/out/hispanic_pegasus/wifiot_hispanic_pegasus" failed
[OHOS ERROR] return code: 1
[OHOS ERROR] execution path: /home/book/work/code-v3.0-LTS/OpenHarmony/applications/sample/wifi-iot/app
```

图5-1 报错图片

● 可能原因：

可能是移植完成之后，需要修改“device/hisilicon/hispanic_pegasus/sdk_liteos/BUILD.gn”文件。

● 解决办法：

移植完成之后，需要打开“device/hisilicon/hispanic_pegasus/sdk_liteos/BUILD.gn”文件，在下图部分添加“//third_party/pahomqtt:pahomqtt_static”，第三方库添加完成了，接下来就可以进行使用了。

```
33 lite_component("sdk") {
34     features = []
35
36     deps = [
37         "//build/lite/config/component/cJSON:cjson_static",
38         "//device/hisilicon/hispanic_pegasus/hi3861_adapter/kal",
39         "//third_party/a_myparty:a_myparty_static",
40         "//third_party/pahomqtt:pahomqtt_static",
41     ]
42 }
43
```

图5-2 添加第三方库的文件路径



4.2 编译的时候，提示 BUILD.gn 文件错误。

● 现象描述：

会有以下提示，说在某某路径下的 BUILD.gn 文件有错误。

```
[OHOS INFO] ohos_build_compiler: gcc
[OHOS INFO] ERROR at //build/lite/config/component/lite_component.gni:97:17: Un
nHarmony/applications/sample/wifi-iot/app/base_08_i2c_ssd1360/BUILD.gn".
[OHOS INFO]     deps += [ feature_label ]
[OHOS INFO]           ^-----
[OHOS INFO] hks_config_lite.h
[OHOS ERROR] you can check build log in /home/book/work/code-v3.0-LTS/OpenHarmoni
build.log
[OHOS ERROR] command: "/home/book/work/code-v3.0-LTS/OpenHarmony/prebuilts/build
de-v3.0-LTS/OpenHarmony/out/hisark_pegasus/wifiot_hisark_pegasus --root=/home
=/home/book/work/code-v3.0-LTS/OpenHarmony/build/lite/.gn --script-executable=/
ified="gcc" product_path="/home/book/work/code-v3.0-LTS/OpenHarmony/vendor/hisi
work/code-v3.0-LTS/OpenHarmony/device/hisilicon/hisark_pegasus/sdk_liteos" ohos
false disable_authenticate = false huks_use_lite_storage = true huks_use_hardwa
ig_lite.h" huks_mbedtls_path = "//device/hisilicon/hisark_pegasus/sdk_liteos/tl
ebug" ohos_build_time="1653625094851" ohos_build_datetime="2022-05-26 20:18:14"
[OHOS ERROR] return code: 1
[OHOS ERROR] execution path: /home/book/work/code-v3.0-LTS/OpenHarmony
```

图5-3 工程目录下的 BUILD.gn 文件

● 可能原因：

需要检查一下两个 BUILD.gn 文件。

● 解决办法：

以下面的例子为例，其他的类似。

这时需要仔细检查一下 “//applications/sample/wifi-iot/app/” 目录下的 BUILD.gn 文件，目录和对应的目标文件是否正确。

```
import("//build/lite/config/component/lite_component.gni")

lite_component("app") {
    features = [
        # "base_01_key:base_key_example",
        # "base_02_i2c_pc18574_tan:base_tan_example",
    ]
}
```

目录

目录下的BUILD.gn文件，所对应的目标文件

图5-4 检查 app/BUILD.gn 文件

再检查一下 “base_01_key” 目录下的 BUILD.gn 文件：



```
applications > sample > wifi-iot > app > base_01_key > BUILD.gn
1  static_library("base_key_example") {
2      sources = [
3          "base_key_example.c",
4          "//base/iot_hardware/bsp/src/hal_bsp_pcf8574.c"
5      ]
6
7      include_dirs = [
8          "//utils/native/lite/include",
9          "//kernel/liteos_m/kal/cmsis",
10         "//base/iot_hardware/peripheral/interfaces/kits",
11         "//base/iot_hardware/bsp/include"
12     ]
13 }
```

图5-5 工程下的 BUILD.gn 文件

主要检查几个地方：

1. Sources 源文件的路径是否正确；
2. Include_dirs 头文件的路径是否正确；
3. 检查一下 JSON 的语法是否正确，是否少了“,”逗号；

4.3 编译的时候，提示“undefined reference to ‘hi_i2c_init’”。

● 现象描述：

在工程中使用到了 I2C 的相关接口，然后开始编译时出现下面的问题：

```
-lres_cfg -lsamgr -lsamgr_adapter -lsamgr_source -lsdio -lsec_flash -lspiffs -lsysparam -lsystem -ltoken_stat:
ervice -ltsensor -luart -lupg -lutils_kv_store -lwifi -lwifi_flash -lwifiaware -lwifiot_app -lwifiservice -lw
up
[OHOS ERROR] riscv32-unknown-elf-ld: ohos/libs/libbase_fan_example.a(libbase_fan_example.hal_bsp_pcf8574.o): in
PCF8574_Write':
[OHOS ERROR] hal_bsp_pcf8574.c:(.text.PCF8574_Write+0x30): undefined reference to `hi_i2c_write'
[OHOS ERROR] riscv32-unknown-elf-ld: ohos/libs/libbase_fan_example.a(libbase_fan_example.hal_bsp_pcf8574.o): in
PCF8574_Init':
[OHOS ERROR] hal_bsp_pcf8574.c:(.text.PCF8574_Init+0x38): undefined reference to `hi_i2c_init'
[OHOS ERROR] scons: *** [output/bin/Hi3861_wifiot_app.out] Error 1
[OHOS ERROR] BUILD FAILED!!!!
[OHOS ERROR] Failed building output/bin/Hi3861_wifiot_app.out: Error 1
[OHOS ERROR] you can check build log in /home/book/work/code-v3.0-LTS/OpenHarmony/out/hispark_pegasus/wifiot |
```

图5-6 提示错误

● 可能原因：

可能是 sdk_liteos 源码中的 usr_config.mk 文件中没有把 CONFIG_I2C_SUPPORT 打开。

● 解决办法：

打开“//device/hisilicon/hispark_pegasus/sdk_liteos/build/config/usr_config.mk”文件，在此文件中，将“CONFIG_I2C_SUPPORT”取消注释，并加上“=y”。



```
30  #
31  # BSP Settings
32  #
33  CONFIG_I2C_SUPPORT=y
```

图5-7 打开 I2C 的支持包

注意：比如今后可能还会用到 SPI、PWM、DMA 等，都需要在这个文件中进行修改。

4.4 为什么我编译好之后烧写工具又检测不到板子了？

● 现象描述

在下载程序的时候，发现有的时候连接不上开发板。

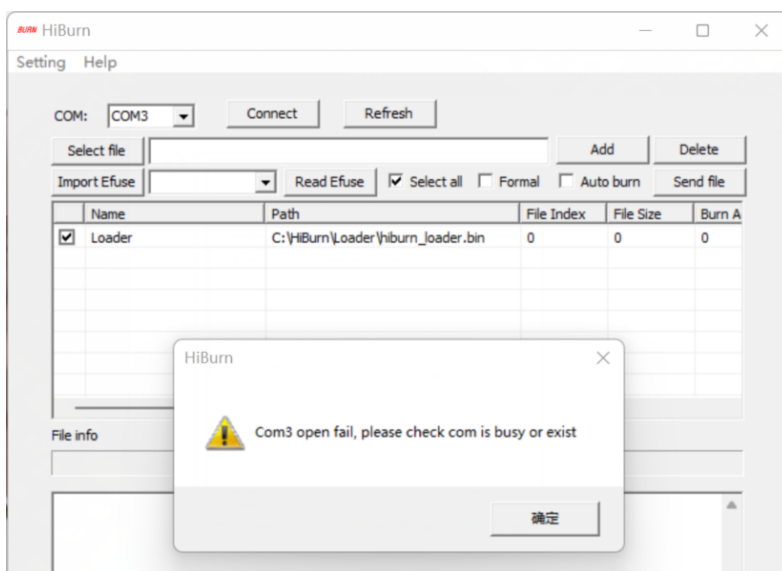


图5-8 烧写软件检测不到芯片

● 解决办法：

1. 检查是否安装 CH340 的串口驱动；
2. 检查开发板是否上电；
3. 检查 USB 转 Type-C 线是否支持数据传输功能；
4. 以上检查好了之后，点击 Refresh 按钮，进行刷新，找到串口的端口号；
5. 接好开发板之后，上电
6. 检查是否有串口端口号
7. 有端口号之后，配置烧写工具的串口波特率
8. 加载要烧写的 bin 文件，
9. 然后点击**connect**按钮
10. 再按下开发板上的复位按键
11. 这时看烧写工具的进度条



12. 直到提示烧写完成，然后点击**disconnect**按钮

13. 然后按下开发板中的复位按键，查看现象。

4.5 在编译开发板程序的时候，提示“/bin/sh: riscv32-unknown-elf-gcc: 未找到命令”？

● 现象描述

在首次编译程序的时候，在终端中，提示“*****: 未找到命令”

```

out > hispark_pegasus > wifiot_hispark_pegasus > build.log
1 phos_build_compiler: gcc
2 hks_config_lite.h
3 Done. Made 74 targets from 64 files in 398ms
4 ninja: Entering directory '/home/linux/work/code-v3.0-LTS/OpenHarmony/out/hispark_pegasus/wifiot_hispark_pegasus'
5 [1/251] gcc cross compiler obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_cache.o
6 FAILED: obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_cache.o
7 riscv32-unknown-elf-gcc -DOUTPUT_OPTION=\ 1 -DHILOG_LITE_SWITCH=\ 1 -DDUMP_LITE_SWITCH=\ 0 -DHIEVENT_LITE_SWITCH=\ 1 -DLOG_OUTPUT_MODULE=\ -1 -DLOG
8 /bin/sh: riscv32-unknown-elf-gcc: 未找到命令
9 [2/251] gcc cross compiler obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_config.o
10 FAILED: obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_config.o
11 riscv32-unknown-elf-gcc -DOUTPUT_OPTION=\ 1 -DHILOG_LITE_SWITCH=\ 1 -DDUMP_LITE_SWITCH=\ 0 -DHIEVENT_LITE_SWITCH=\ 1 -DLOG_OUTPUT_MODULE=\ -1 -DLOG
12 /bin/sh: riscv32-unknown-elf-gcc: 未找到命令
13 [3/251] gcc cross compiler obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_file.o
14 FAILED: obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_file.o
15 riscv32-unknown-elf-gcc -DOUTPUT_OPTION=\ 1 -DHILOG_LITE_SWITCH=\ 1 -DDUMP_LITE_SWITCH=\ 0 -DHIEVENT_LITE_SWITCH=\ 1 -DLOG_OUTPUT_MODULE=\ -1 -DLOG
16 /bin/sh: riscv32-unknown-elf-gcc: 未找到命令
17 [4/251] gcc cross compiler obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_service.o
18 FAILED: obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_service.o
19 riscv32-unknown-elf-gcc -DOUTPUT_OPTION=\ 1 -DHILOG_LITE_SWITCH=\ 1 -DDUMP_LITE_SWITCH=\ 0 -DHIEVENT_LITE_SWITCH=\ 1 -DLOG_OUTPUT_MODULE=\ -1 -DLOG
20 /bin/sh: riscv32-unknown-elf-gcc: 未找到命令
21 [5/251] gcc cross compiler obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_util.o
22 FAILED: obj/base/hiviewdfx/hiview_lite/libhiview_lite.hiview_util.o
23 riscv32-unknown-elf-gcc -DOUTPUT_OPTION=\ 1 -DHILOG_LITE_SWITCH=\ 1 -DDUMP_LITE_SWITCH=\ 0 -DHIEVENT_LITE_SWITCH=\ 1 -DLOG_OUTPUT_MODULE=\ -1 -DLOG
24 /bin/sh: riscv32-unknown-elf-gcc: 未找到命令
25 [6/251] gcc cross compiler obj/base/hiviewdfx/hidumper_lite/mini/libhidumper_mini.hidumper_core.o
26 FAILED: obj/base/hiviewdfx/hidumper_lite/mini/libhidumper_mini.hidumper_core.o
27 riscv32-unknown-elf-gcc -D_XOPEN_SOURCE=700 -DOHOS_DEBUG -D_LITEOS_M -D_LITEOS_M -I../base/hiviewdfx/hidumper_lite/mini/interfaces/native/kits -I..
28 /bin/sh: riscv32-unknown-elf-gcc: 未找到命令
29 ninja: build stopped: subcommand failed.
30

```

图5-9 编译时，提示报错

● 可能原因

在终端里面输入“riscv32-unknown-elf-gcc -v”命令，看看是否有正确的版本号，如果没有的话，大概应该是没有配置好环境。

● 解决办法

1. 在终端中输入“gedit ~/.bashrc”。
2. 在最后一行添加 riscv32-unknown-elf-gcc 工具的 bin 文件路径；

```

118
119 export PATH=/home/linux/.local/bin:$PATH
120 export PATH=/home/linux/work/tools/gcc_riscv32/bin:$PATH
121

```

图5-10 添加 bin 文件的路径

3. 修改完成之后，点击保存，然后在终端中输入命令 source ~/.bashrc
4. 在终端里面输入 riscv32-unknown-elf-gcc -v 命令，看看是否有正确的版本号；
5. 如果有，说明可以解决这个问题。



4.6 为什么我的开发板上蜂鸣器一上电就开始响？

● 可能原因

这是因为开发板上电的时候，蜂鸣器引脚默认为低电平，所以蜂鸣器会响，如下原理图所示（开发板的版本号：FS_Hi3861DB_V3.2）：

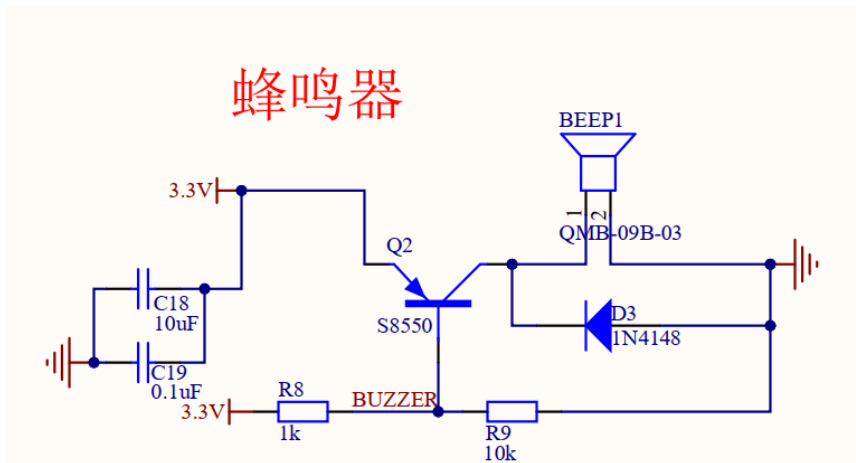


图5-11 蜂鸣器驱动原理图

● 解决办法

打开【//device/hisilicon/hispark_pegasus/sdk_liteos/app/wifiot_app/src/app_main.c】文件，修改 app_main.c 文件，在 294 行的#endif 后面添加：

```

296  #if 1
297      int32_t PCF8574_I2C_ADDR = 0x42;    // 器件的I2C从机地址
298      int32_t PCF8574_I2C_ID = 0;        // 模块的I2C总线号
299      int32_t PCF8574_SPEED = 100000;    // 100KHz
300
301      // gpio 9 复用为 I2C_SCL
302      hi_io_set_pull(HI_IO_NAME_GPIO_9, HI_IO_PULL_UP);
303      hi_io_set_func(HI_IO_NAME_GPIO_9, HI_IO_FUNC_GPIO_9_I2C0_SCL);
304      // gpio 10 复用为 I2C_SDA
305      hi_io_set_pull(HI_IO_NAME_GPIO_10, HI_IO_PULL_UP);
306      hi_io_set_func(HI_IO_NAME_GPIO_10, HI_IO_FUNC_GPIO_10_I2C0_SDA);
307
308      ret = hi_i2c_init(PCF8574_I2C_ID, PCF8574_SPEED);
309      if (ret != HI_ERR_SUCCESS)
310      {
311          printf("[error] I2C PCF8574 Init status is 0x%x!!!", ret);
312          err_info |= PERIPHERAL_INIT_ERR_I2C;
313      }
314
315      uint8_t buffer[] = {0x02};    // 蜂鸣器引脚默认为高电平
316      hi_i2c_data i2c_data = { 0 };
317      i2c_data.send_buf = buffer;
318      i2c_data.send_len = sizeof(buffer);
319
320      ret = hi_i2c_write(PCF8574_I2C_ID, PCF8574_I2C_ADDR, &i2c_data);
321      if (ret != HI_ERR_SUCCESS)
322      {
323          printf("[error] I2C PCF8574 Write ret is 0x%x!!!", ret);
324          err_info |= PERIPHERAL_INIT_ERR_I2C;
325      }
326  #endif
327
328  }
    
```

图5-12 修改 IO 扩展芯片的引脚初始化状态

